

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
UNIVERSITY OF SCIENCE
FACULTY OF ELECTRONICS – TELECOMMUNICATIONS

BUI THANH DAT

**Open-Source RISC-V SoC Implementation
with DNN Acceleration: A Chipyard Frame-
work Study with FPGA Prototyping**

UNDERGRADUATE THESIS
MAJOR OF ELECTRONICS – TELECOMMUNICATIONS
ENGINEERING

Ho Chi Minh City, 08/2025

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
UNIVERSITY OF SCIENCE
FACULTY OF ELECTRONICS – TELECOMMUNICATIONS

Bui Thanh Dat
Student ID: 21207001

**Open-Source RISC-V SoC Implementation
with DNN Acceleration: A Chipyard Frame-
work Study with FPGA Prototyping**

UNDERGRADUATE THESIS
MAJOR OF ELECTRONICS – TELECOMMUNICATIONS
ENGINEERING

SUPERVISOR
Huynh Huu Thuan, Dr.

Ho Chi Minh City, 08/2025

ACKNOWLEDGEMENTS

First and foremost, I wish to express my deepest thanks to all the members of the Computer Embedded Systems Laboratory (CESLAB). Their extensive support, both throughout my academic years and during the intensive work on this thesis, created an environment of collaboration and learning that was essential for my success.

I owe a special debt of gratitude to my supervisor, Dr. Huynh Huu Thuan. His encouragement from my freshman year was pivotal in starting my research journey, and his diligent guidance, invaluable advice, and mentorship were instrumental in the completion of this work.

On a personal note, I extend a heartfelt thanks to my family for their unconditional trust and support, regardless of my choices. I am also deeply thankful for my friends, who stood by me even when I was at the lowest point of my undergraduate life.

I am also immensely grateful to the researchers and professors, both within the Faculty of Electronics - Telecommunications (FETEL) and from universities both within Vietnam and across the USA, Australia, and Denmark, who offered their time and wisdom. Their mentorship and insights into academia were profoundly inspiring and helped shape my decision to pursue a research career.

Furthermore, I wish to acknowledge the global open-source community. For a student from a developing country, access to high-quality tools and shared knowledge is a profound blessing that breaks down barriers to research and innovation. A special thank you is due to Eugene Tarassov, Principal Member of Technical Staff at AMD, whose `vivado-risc-v` open-source repository served as a foundational tool for this study.

Finally, I extend my appreciation to all the instructors and staff at the Faculty of Electronics - Telecommunications (FETEL) for their dedication to education.

TRANG CAM KẾT TÍNH TRUNG THỰC CỦA BÁO CÁO

Tôi xin cam đoan những số liệu và kết quả nghiên cứu được trình bày trong khóa luận tốt nghiệp là trung thực. Tôi cam kết không sao chép nội dung hay kết quả của các nghiên cứu khác đã được công bố. Những tài liệu tham khảo được tôi sử dụng trong báo cáo đã được trích dẫn một cách đầy đủ, rõ ràng về nguồn gốc theo quy định.

Tôi xin chịu trách nhiệm nếu vi phạm các cam kết trên.

Bùi Thành Đạt

TÓM TẮT

Nhu cầu tính toán ngày càng tăng của các Mạng Nơ-ron Sâu (DNN) đã bộc lộ những hạn chế của bộ xử lý đa dụng truyền thống, từ đó đặt ra yêu cầu cấp thiết cho các bộ tăng tốc phần cứng chuyên dụng. Tuy nhiên, việc nghiên cứu và phát triển trong lĩnh vực này thường bị hạn chế bởi các công cụ thiết kế độc quyền và IP phụ thuộc vào nhà cung cấp, với chi phí bản quyền cao và khả năng tùy biến hạn chế. Sự ra đời của kiến trúc tập lệnh (ISA) mã nguồn mở RISC-V đã mang đến một giải pháp thay thế mạnh mẽ và bền vững cho tương lai. Luận văn này trình bày việc thiết kế, triển khai full-stack và đánh giá một Hệ thống trên Vi mạch (SoC) RISC-V có khả năng chạy hệ điều hành Linux, và được tích hợp một bộ tăng tốc DNN chuyên dụng. Mục tiêu chính của luận văn là thực hiện một nghiên cứu toàn diện về hệ sinh thái RISC-V thông qua việc minh chứng một quy trình làm việc mã nguồn mở hoàn chỉnh, từ cấu hình phần cứng mức cao bằng Chisel cho đến việc xây dựng nguyên mẫu trên FPGA. Hệ thống được thiết kế trên nền tảng Chipyard, tích hợp lõi xử lý 64-bit RISC-V Rocket Core với bộ tăng tốc Gemmini có kiến trúc mảng xử lý song song theo nhịp (systolic array) thông qua giao thức kết nối RoCC. Hệ thống SoC đã được triển khai thành công trên bo mạch FPGA Xilinx (AMD) Virtex 7 VC707, khởi chạy thành công hệ điều hành Debian Linux. Kết quả đánh giá hiệu năng cho thấy bộ tăng tốc Gemmini đạt được mức tăng tốc xấp xỉ 1,300 lần trên một tác vụ tính toán đã được tối ưu so với giải pháp phần mềm thuần túy trên Rocket Core. Công trình này đã xác thực thành công nền tảng Chipyard là một nền tảng hoàn thiện để xây dựng các hệ thống RISC-V phức tạp, đồng thời thiết lập một nền tảng full-stack có thể tái tạo cho các nghiên cứu trong tương lai, qua đó cung cấp một lộ trình chuyển đổi từ các quy trình làm việc độc quyền, khép kín sang hệ sinh thái rộng mở cùng tập lệnh RISC-V.

ABSTRACT

The increasing computational demand of Deep Neural Networks (DNNs) has exposed the limitations of traditional general-purpose processors, creating a need for specialized hardware accelerators. However, research and development in this area are often constrained by proprietary design tools and vendor-specific IP, which impose high licensing costs and limit customization. The emergence of the open-standard RISC-V instruction set architecture (ISA) provides a powerful, future-proof alternative. This thesis presents the design, full-stack implementation, and evaluation of a Linux-capable RISC-V System-on-Chip (SoC) featuring a dedicated DNN accelerator. The primary objective is to conduct a comprehensive study of the RISC-V ecosystem by demonstrating a complete open-source workflow, from high-level hardware configuration in Chisel to FPGA prototyping. The system was designed using the Chipyard framework, integrating a 64-bit RISC-V Rocket Core with the Gemmini systolic array accelerator via the RoCC interface. The implemented SoC was successfully deployed on a Xilinx VC707 FPGA board, booting a full Debian Linux operating system. Performance evaluation revealed that the Gemmini accelerator achieved speedups exceeding 1,300x for optimized workloads compared to a software-only implementation on the Rocket Core. This work successfully validates the Chipyard framework as a mature platform for building complex RISC-V systems and establishes a reproducible, full-stack foundation for future research, providing a roadmap for transitioning from closed, proprietary workflows to the open and extensible RISC-V ecosystem.

TABLE OF CONTENTS

ACKNOWLEDGEMENTS	i
TÓM TẮT	iii
ABSTRACT	iv
MỤC LỤC	v
LIST OF ABBREVIATIONS	xi
LIST OF FIGURES	xvi
LIST OF TABLES	1
1 INTRODUCTION	2
1.1 Background and Motivation	2
1.2 The Open-Source Hardware Ecosystem	3
1.2.1 A Survey of Open-Source Soft Cores	3
1.2.2 The Strategic Advantage of RISC-V	5
1.3 A Survey of Modern Open-Source SoC Frameworks	6
1.3.1 LiteX: The Flexible SoC Builder	6
1.3.2 PULP Platform: The Low-Power Parallel Computing Specialist	7
1.3.3 OpenTitan: The Security-First Root of Trust	7
1.4 Chipyard: the framework with Full-Stack Integration Capability	7
1.5 Core Selection within Chipyard: A Comparison of Rocket and BOOM	8
1.6 Research Objectives and Contributions	9
1.7 Significance and Impact	10
1.8 Thesis Structure and Organization	11
2 RISC-V MICROPROCESSOR INSTRUCTION SET ARCHITECTURE	13
2.1 Overview of RISC-V	13
2.1.1 RISC-V 32-bit and the RV32I Base Instruction Set	15
2.1.2 RISC-V 64-bit and the RV64I Base Instruction Set	19
2.2 RISC-V Processor Cores	22

2.2.1	Pipeline Architecture in Microprocessors	22
2.2.2	Out-of-Order Architecture in Microprocessors	24
2.2.3	Open-Source RISC-V Processor Cores	24
2.3	System-on-Chip Design Framework: Chipyard - Rocket Chip	27
3	A SINGLE-CORE RISC-V PROCESSOR SYSTEM WITH A GEMMINI ROCC ACCELERATOR	30
3.1	The Chipyard Compilation and Elaboration Flow	30
3.2	System Design Methodology with Chisel	31
3.2.1	Chisel and Scala	32
3.2.2	Verilator Simulation Tool	33
3.3	The Rocket Processor Core	34
3.4	Rocket Chip SoC Structure	35
3.4.1	Bus and Interconnect	35
3.4.2	Memory and Cache	39
3.4.3	Core-Complex	40
3.4.4	Peripheral Components	41
3.4.5	The RoCC Interface: A Hardware Co-Design Methodology . . .	44
4	ARCHITECTURAL FOUNDATIONS OF DNN ACCELERATION	51
4.1	The Data-Compute Divide: The Memory Wall	51
4.2	A Classic Solution: The Systolic Principle	52
4.3	Case Studies in Modern Systolic Design	54
4.3.1	Google’s TPUv1: A Datacenter-Scale Design	54
4.3.2	NVIDIA NVDLA: An Open-Source Accelerator IP	55
4.4	A Formal Framework: Dataflows	56
4.5	The Gemmini Approach: A Full-Stack Research Platform	58
4.6	Chapter Summary	60
5	THE GEMMINI DNN ACCELERATOR: AN ARCHITECTURAL OVERVIEW	61
5.1	Top-Level Architectural Blueprint	61
5.2	Analysis of Core Architectural Components	61
5.2.1	The Systolic Compute Engine	61
5.2.2	Mapping Convolution to Hardware: The <code>im2col</code> Transformation	63
5.2.3	The On-Chip Memory Subsystem	65

5.3	SoC Integration and System-Level Features	65
5.3.1	The RoCC Coprocessor Interface	65
5.3.2	Virtual Memory Support	66
5.4	The Multi-Level Programming Model	66
5.4.1	Low-Level: Gemmini ISA	66
5.4.2	Mid-Level: Hand-Tuned C Library	67
5.4.3	High-Level: Compiler Support (Conceptual)	67
5.5	Summary	67
6	GEMMINI'S INTERNAL ARCHITECTURE AND PROGRAMMING MODEL	68
6.1	The Command-Unrolling Hierarchy	69
6.1.1	Level 1: Convolution Decomposition (<code>LoopConv</code>)	70
6.1.2	Level 2: Tiled Matrix Multiplication (<code>LoopMatmul</code>)	71
6.2	Decoupled Execution and Resource Management	73
6.2.1	The Reservation Station: A Central Dispatch Hub	73
6.2.2	Back-End Controllers: Load, Store, and Execute	75
6.3	The Physical Hardware: Compute and Memory	75
6.3.1	The Processing Element (PE) and Distributed Arithmetic	75
6.3.2	The Tile and The Mesh: A Tale of Two Hierarchies	77
6.3.3	Architectural Trade-offs in Mesh Configuration	79
6.3.4	The Scratchpad: An Intelligent Memory Subsystem	79
6.4	Chapter Summary	81
7	SYSTEM IMPLEMENTATION AND TESTING	82
7.1	Hardware Implementation: SoC Configuration	82
7.1.1	The <code>Rocket64b1gem16jtag</code> Configuration	84
7.2	Gemmini Accelerator Configuration and Integration	86
7.2.1	Accelerator Instantiation via the RoCC Interface	86
7.2.2	Hardware Configuration Parameters	87
7.3	Peripheral Integration	88
7.3.1	DDR Memory: High-Bandwidth Main Memory	89
7.3.2	SD Card: AXI SD Card Controller and SD Mode	89
7.3.3	UART: AXI UART Controller	90
7.3.4	Ethernet: AXI Ethernet Controller	91
7.3.5	System Monitoring: XADC and Fan Control	92

7.4	Development Environment	92
7.5	Software Stack and Build Process	93
7.5.1	Boot Flow: OpenSBI, U-Boot, and Linux	93
7.5.2	SoC Hardware Generation	93
7.6	Gemmini Testing	94
8	FPGA IMPLEMENTATION RESULTS AND DISCUSSION	95
8.1	Hardware Implementation Analysis	95
8.1.1	System Block Design and Address Map	95
8.1.2	Vivado Implemented Design Results	96
8.2	Software Stack Validation	101
8.2.1	System Boot Sequence	101
8.2.2	Debian Linux Environment	103
8.3	Gemmini Accelerator Performance Evaluation	104
8.3.1	Functional Correctness	104
8.3.2	Performance Speedup and Analysis	104
8.3.3	Real-World Workload Analysis: MobileNetV1	105
8.3.4	Discussion of Results	106
9	MODERN ALGORITHMS IN ASIC PHYSICAL DESIGN FLOW	107
9.1	ASIC Physical Design and Verification Workflow	107
9.1.1	Overview of Physical Design	108
9.1.2	Physical Verification	110
9.2	Open-Source Tools for ASIC Physical Workflow	111
9.2.1	OpenROAD	111
9.2.2	LibreLane	112
9.3	Scale of Modern Design, and the need for Algorithms	112
9.4	Netlist and System Partitioning	113
9.4.1	Partitioning Algorithms, and their Complexity	113
9.4.2	OpenROAD Partitioning Approach: TritonPart	115
9.4.3	TritonPart: the first open-source, constraints-driven partition- ing tool for VLSI physical design	115
9.5	Floorplanning, Pin Assignment and Power Routing: Establishing the Chip's Foundation	116
9.5.1	Floorplanning	116
9.5.2	Pin Assignment	116

9.5.3	Power and Ground Routing	116
9.6	Global and Detailed Placement	116
9.7	Routing	116
9.7.1	Global Routing	116
9.7.2	Detailed Routing	116
9.8	Specialized Routing and Clock Tree Synthesis	116
9.9	Timing Closure	116
9.10	Physical Verification and Signoff	116
10	VLSI PHYSICAL IMPLEMENTATION RESULTS	117
10.1	The Open-Source Physical Design Methodology	119
10.2	Logic Synthesis: From RTL to a Technology-Mapped Netlist	119
10.2.1	Theoretical Foundation	119
10.2.2	OpenROAD Implementation	119
10.2.3	Application to the SoC	120
10.3	Floorplanning and Power Delivery Network (PDN) Generation	120
10.3.1	Theoretical Foundation: Establishing the Chip's Foundation	120
10.3.2	OpenROAD Implementation	120
10.3.3	Application to the SoC	121
10.4	Placement: Assigning Locations to Standard Cells	121
10.4.1	Theoretical Foundation	121
10.4.2	OpenROAD Implementation	121
10.4.3	Application to the SoC	121
10.5	Clock Tree Synthesis and Routing	121
10.5.1	Theoretical Foundation	121
10.5.2	OpenROAD Implementation	121
10.5.3	Application to the SoC	121
10.6	Global and Detailed Routing	121
10.6.1	Theoretical Foundation	121
10.6.2	OpenROAD Implementation	121
10.6.3	Application to the SoC	121
10.7	GDSII Generation and Final Sign-off Verification	121
10.8	Chapter Summary	122
11	CONCLUSION AND FUTURE WORK	126
11.1	Summary of Work and Findings	126

11.2 Limitations and Key Insights	127
11.3 Future Work	127
LIST OF PUBLICATIONS	130
BIBLIOGRAPHY	131
A DETAILED PROCESSING UNITS	139
A.1 ExecuteController Sub-Modules	139
A.2 Scratchpad Integrated Processing Units	140
B ROCC INTERFACE AND GEMMINI INTEGRATION DETAILS	141
B.1 SoC Configuration and Accelerator Instantiation	141
B.2 RoCC Hardware Interface Bundles	142
B.3 Gemmini Custom Instruction Set	143
B.4 CPU-Side Pipeline Integration Details	144
B.4.1 Command Processing Flow	144
B.4.2 Pipeline Stall and Hazard Management	144
B.4.3 Response and Register File Write	145
B.4.4 Status and Interrupt Handling	145
C TIME MEASURING METHODOLOGY	146
D DETAILED PERFORMANCE EVALUATION RESULTS	155
D.1 Performance Benchmark Results	155
D.2 Test Program Documentation and Configurations	156
D.2.1 Matrix Multiplication Tests (bareMetalC/)	156
D.2.2 Convolution Tests (bareMetalC/)	157
D.2.3 Neural Network Models (imagenet/)	159
D.2.4 Multi-Layer Perceptron Models (mlps/)	159
D.2.5 Additional Tests	160

LIST OF ABBREVIATIONS

ABI Application Binary Interface. 15, 16

AI Artificial Intelligence. 7

ALU Arithmetic Logic Unit. 46

ARM Advanced RISC Machine. 13

ASIC Application-Specific Integrated Circuit. 6–8, 11, 14, 27, 30, 54, 79, 128, 129

AXI4 Advanced eXtensible Interface 4. 7, 42, 82, 88, 91, 92

BOOM Berkeley Out-of-Order Machine. 8, 9, 25–28, 35, 41–43

BRAM Block RAM. 96

CESLAB Computer Embedded Systems Laboratory. 7, 8

Chisel Constructing Hardware in a Scala Embedded Language. 7, 11, 25, 27–35, 39, 42, 44, 58, 88, 93, 95

CISC Complex Instruction Set Computer. 13, 69

CNN Convolutional Neural Network. 105

CPU Central Processing Unit. 8, 9, 13, 22–24, 44–49, 51, 58, 61, 63, 65, 66, 68, 69, 80, 82, 88, 91, 105

CSR Control and Status Register. 19, 25

CTS Clear to Send. 91

DDR Double Data Rate. 93, 95

Debian Debian GNU/Linux. 85, 93, 101, 103, 127

DMA Direct Memory Access. 37, 58, 65, 66, 80, 81, 85, 91, 95, 104, 128

DNN Deep Neural Network. 2, 7–9, 11, 12, 45, 49–51, 54, 56, 58, 60, 61, 63, 67, 76, 82, 85, 89, 94, 95, 104, 126

DRAM Dynamic Random Access Memory. 35, 40, 42, 51, 61, 65, 69, 72–75, 79, 85, 101

DSL Domain-Specific Language. 32

DSP Digital Signal Processing. 96

EDA Electronic Design Automation. 10

ETH Zurich Swiss Federal Institute of Technology in Zurich. 7

Ethernet Ethernet Protocol. 83, 88, 91, 94, 95

FIFO First In, First Out. 91, 92

FIRRTL Flexible Intermediate Representation for RTL. 28, 30, 33, 95

FPGA Field-Programmable Gate Array. 6–8, 10–12, 14, 27, 28, 30, 31, 40, 41, 79, 81, 82, 86, 91, 92, 95, 96, 126, 128

FPU Floating Point Unit. 76

GCC GNU Compiler Collection. 22

GEMM General Matrix Multiply. 61, 63, 128

GlobalFoundries GlobalFoundries Inc.. 8

GPIO General Purpose Input/Output. 35, 42

HDL Hardware Description Language. 32, 33, 39, 93, 94

I/O Input/Output. 41, 83, 88, 90, 91, 95, 98

IEEE Institute of Electrical and Electronics Engineers. 14, 76, 92

Intel Intel Corporation. 7

IoT Internet of Things. 7

IP Intellectual Property. 6–8, 40, 55, 58, 82, 83, 88, 89, 91, 92

ISA Instruction Set Architecture. 2, 11, 13–15, 22, 23, 26, 29, 43–45, 67, 82, 103

JTAG Joint Test Action Group. 41, 42

JVM Java Virtual Machine. 32

LAQ Load Address Queue. 27

Linux Linux Operating System. 2, 3, 8–10, 12, 22, 28, 58, 66, 82, 83, 85, 88, 89, 91, 93, 95, 96, 101, 103, 126, 127

LLVM Low Level Virtual Machine. 22

LUT Look-Up Table. 96

MAC Multiply-Accumulate. 53–55, 61, 65, 75, 76, 88, 91

MHz Megahertz. 25, 28, 92

MiB Mebibyte. 54

MIG Memory Interface Generator. 89

MMIO Memory-Mapped Input/Output. 39, 42–44

MMU Memory Management Unit. 25

NVDLA NVIDIA Deep Learning Accelerator. 55, 56, 58, 60

ONNX Open Neural Network Exchange. 67

OOP Object-Oriented Programming. 32

OpenSBI RISC-V Open Source Supervisor Binary Interface. 93

OS Output Stationary. 56, 58, 60, 63, 76, 85, 103

PE Processing Element. 52, 53, 56, 61, 63, 66, 75–79, 81, 85

PTW Page Table Walker. 25, 34, 58, 66

PWM Pulse Width Modulation. 41, 92

Python Python Programming Language. 6

QEMU Quick Emulator. 22

RAM Random Access Memory. 93

RISC Reduced Instruction Set Computer. 13, 23

RISC-V Reduced Instruction Set Computer - Five. 2, 3, 7, 9–11, 13–15, 17, 19, 22–25, 27–29, 34–36, 43–45, 65, 82, 91, 93, 95, 97, 104, 126–129

ROB Re-order Buffer. 24

RoCC Rocket Custom Coprocessor. 2, 9, 27, 30, 35, 39, 43–46, 48–50, 65–69, 81, 82, 86, 87, 94, 104, 126, 128, 129

ROM Read-Only Memory. 41, 85, 93, 94, 101

RTL Register Transfer Level. 28, 30, 32, 33, 58

RTS Request to Send. 91

SAQ Store Address Queue. 27

sbt Simple Build Tool. 32

Scala Scala Programming Language. 11, 25, 30–34, 39, 85, 93

SDQ Store Data Queue. 27

SGMII Serial Gigabit Media Independent Interface. 91, 92

SIMD Single Instruction, Multiple Data. 58

SoC System-on-Chip. 2, 3, 6, 7, 9–12, 22, 27, 28, 30, 34–37, 41, 42, 44, 55, 58, 59, 61, 65, 67, 81–84, 86, 88–96, 98, 103, 104, 126

SPI Serial Peripheral Interface. 42, 90

SRAM Static Random Access Memory. 40, 65

TLB Translation Lookaside Buffer. 25, 58, 66, 85

TPU Tensor Processing Unit. 54–56, 58, 60

TSMC Taiwan Semiconductor Manufacturing Company. 8

UART Universal Asynchronous Receiver/Transmitter. 35, 41, 88, 90, 91, 94, 95

Ubuntu Ubuntu Linux Distribution. 92

UC Berkeley University of California, Berkeley. 2, 7, 8, 10, 14, 25, 27, 28, 32, 58

VC707 Xilinx VC707 Evaluation Board. 10, 31, 82, 83, 86, 89, 91, 92, 94–96, 126

Vivado Xilinx Vivado Design Suite. 31, 95–97

VLIW Very Long Instruction Word. 58

VLSI Very Large Scale Integration. 6–8, 28, 30, 128

WS Weight Stationary. 56, 60, 63, 76, 85

XADC Xilinx Analog-to-Digital Converter. 92

LIST OF FIGURES

2.1	RISC-V is managed by RISC-V International. Source [45]	13
2.2	Instruction encoding formats in RV32I. Source [43]	16
2.3	Instructions in the RV32I base instruction set. [43]	18
2.4	Instructions in the RV64I base instruction set. [43]	20
2.5	Instruction encoding format for ADDIW in 64-bit architecture. [43] . .	20
2.6	Encoding format for 64-bit I-type shift instructions. [43]	21
2.7	Encoding format for 64-bit R-type shift instructions for word operations. [43]	21
2.8	Encoding formats for 64-bit LOAD and STORE instructions. [43] . . .	22
2.9	Basic five-stage pipeline in a RISC machine. The vertical axis is successive instructions; the horizontal axis is time. So in the green column, the earliest instruction is in WB stage, and the latest instruction is undergoing instruction fetch.	23
2.10	Pipeline of Rocket core. [39, 46, 47]	25
2.11	Berkeley Out-of-Order Machine Processor Pipeline Architecture. [41] .	26
2.12	Chipyard, developed by UC Berkeley. [49]	27
2.13	Component structure of Chipyard. [49]	28
3.1	The Chipyard Development and Compilation Flow	31
3.2	Chisel as a library within Scala.	32
3.3	The Chisel to HDL design elaboration flow	33
3.4	Structure of a Rocket Tile.	35
3.5	Proposed structure of a single-core RISC-V processor system with a Gemmini RoCC accelerator.	36
3.6	Connection model in TileLink and its interfaces. [55]	37
3.7	Channels in a TileLink connection between two agents. [55]	38
3.8	TileLink TL-UL connection structure. [55]	40
3.9	Implementation structure of a heterogeneous multi-core processor system.	41
3.10	Peripherals in the Rocket Chip base system.	42
3.11	RoCC co-processor core and MMIO peripheral core in Rocket Chip. . .	43

3.12	Rocket Tile RoCC Interface Architecture with Accelerator Integration. The RoCC interface provides direct communication pathways between the Rocket Core’s datapath and the attached accelerator, including command dispatch, memory requests, and response handling channels.	45
3.13	The RoCC Instruction Format. The <code>opcode</code> field is set to <code>custom3</code> (0b1111011). The <code>funct7</code> field specifies the Gemmini-specific operation, while the <code>xd</code> , <code>xs1</code> , and <code>xs2</code> bits act as control signals for the hardware interface.	46
3.14	Diagram illustrating the RoCC co-design and control flow. (1) A RoCC instruction is recognized at the Decode stage. (2) At the Writeback stage, the <code>RoCCCommand</code> is dispatched to Gemmini over a <code>ready/valid</code> channel. (3) A <code>busy</code> signal provides back-pressure to the CPU’s Hazard Unit, stalling the pipeline. (4) The <code>resp</code> path provides a privileged write-back directly to the Register File.	47
3.15	The asymmetrical RoCC memory interface. The request channel (<code>mem_req</code>) uses a full <code>ready/valid</code> handshake, while the response channel (<code>mem_resp</code>) is a simpler <code>valid-only</code> interface.	49
3.16	Timing diagram for a single RoCC memory request. The diagram demonstrates the two-cycle latency of the memory interface, as the <code>mem_resp_valid</code> signal is asserted two clock cycles after the <code>mem_req_valid</code> signal. Note the absence of a <code>mem_resp_ready</code> signal, confirming the asymmetrical nature of the interface. [2]	50
3.17	Timing diagram for consecutive RoCC memory requests. This illustrates the 50% maximum throughput of the interface. The L1 D-Cache can accept two back-to-back requests but then requires a one-cycle pause before accepting the next two, a critical performance characteristic for data-intensive accelerators. [2]	50
4.1	The Relative Energy Cost of Computation vs. Memory Access	52
4.2	Conventional vs. Systolic Processing	53
4.3	The fundamental principle of a systolic array for matrix multiplication. Input matrix A flows horizontally, while input matrix B flows vertically. This rhythmic dataflow allows for massive parallelism and data reuse.	54
4.4	The Google TPuv1 Block Diagram	55
4.5	The NVDLA High-Level Architecture	56
4.6	Data Reuse Opportunities in DNNs	57

4.7	Taxonomy of Common DNN Dataflows	57
4.8	Architectural Flexibility of the Gemmini Generator	59
4.9	Comparison of OS and WS Dataflows	60
5.1	Gemmini, the DNN Accelerator designed for SoC integration developed by UC Berkeley. [64].	61
5.2	High-level overview of the Gemmini accelerator template integrated with a host CPU and system memory hierarchy. [64].	62
5.3	Microarchitecture of Gemmini’s Spatial Array	62
5.4	Mapping Convolution to Systolic Array Computation via the <code>im2col</code> Transformation. Adapted from Gookyi et al. [66].	64
5.5	Conceptual view of Gemmini’s decoupled access-execute pipelines, managed by separate hardware command queues for Load, Execute, and Store operations. [64].	66
6.1	Overview of the internal modules in the generated Gemmini hardware architecture, showing the flow of commands from the RoCC interface through various queues and controllers.	68
6.2	The <code>LoopConv</code> module. This is the first level of the command-unrolling hierarchy, responsible for decomposing a complex convolution operation into a series of matrix multiplication tasks.	70
6.3	The <code>LoopMatmul</code> module. This second-level unroller translates matrix multiplication tasks into a stream of smaller, tiled micro-operations for the physical hardware.	71
6.4	The primary back-end controllers that receive commands from the Reservation Station.	74
6.5	A single Processing Element (PE), the fundamental compute unit. It contains a Multiply-Accumulate (MAC) unit and the necessary logic to support multiple dataflows, such as Output Stationary and Weight Stationary.	76
6.6	The Multiply-Accumulate (MAC) unit architecture. It is implemented using the <code>Arithmetic.scala</code> typeclass, which provides a suite of operations and supports multiple data types, including integers and IEEE 754 floating-point numbers.	77

6.7	Different Mesh configurations, illustrating the trade-off between the number of tiles and the number of PEs per tile, while maintaining the same total number of 16 PEs.	78
6.8	The internal structure of the Scratchpad module, an intelligent memory subsystem with integrated DMA engines and data processing units that serves the systolic array.	80
7.1	The implemented Rocket SoC with Gemmini on the VC707 FPGA board. The Rocket Core is connected to a 16x16 Gemmini systolic array, and various peripherals are integrated via AXI interfaces.	83
7.2	Gemmini Configuration on <code>Rocket64b1gem16jtag</code> SoC.	86
7.3	SD Mode: Multi-Block Read Operation. [68]	89
7.4	SD Mode: Multi-Block Write Operation. [68]	90
7.5	SPI Mode: Multi-Block Read Operation. [68]	90
7.6	SPI Mode: Multi-Block Write Operation. [68]	90
8.1	Block Design of the System, showing the top-level integration of the RocketChip core, DDR, and IO subsystems.	96
8.2	Block Design of the IO Subsystem, illustrating the AXI interconnect for peripherals such as Ethernet, SD Card, and UART.	97
8.3	Block Design of the DDR Subsystem, detailing the Memory Interface Generator (MIG) and its AXI connection.	98
8.4	Address Editor view, confirming the memory map for the integrated peripherals.	98
8.5	Detailed Utilization Summary for the entire SoC on the VC707 FPGA.	99
8.6	Hierarchical Utilization of the Gemmini accelerator module.	99
8.7	Post-implementation on-chip power summary.	100
8.8	Hierarchical power report for the Gemmini accelerator.	100
9.1	The major steps in the VLSI circuit design flow, with a focus on the physical design steps: partitioning, chip planning, placement, clock tree synthesis and routing, and finally timing closure. [71]	108
9.2	An example of a partitioned circuit, where the original circuit is divided into smaller subcircuits or modules. Each module can be designed or analyzed individually, allowing for more efficient design and verification processes.	113

9.3	An illustration of multilevel partitioning. (a) The original graph is first coarsened through several levels. (b) The graph after coarsening. (c) After coarsening, a heuristic partition is found of the most coarsened graph. (d) That partition is then projected onto the next coarsest graph (dotted line) and then refined (solid line). Projection and refinement continue until a partitioning solution for the original graph is found.	114
10.1	The major steps in the VLSI physical design flow, from logic synthesis to final sign-off. This chapter will follow this sequence to analyze the implementation of the RISC-V SoC. Adapted from [71].	118
10.2	The LibreLane flow, showing the sequence of OpenROAD and various tools orchestrated to transform an RTL netlist into a final GDSII layout.	123
10.3	Floorplan of the RISC-V SoC showing the core area (center), I/O pads (periphery), and the power delivery network (grid). The target core utilization was 60%.	124
10.4	Final GDSII layout of the RISC-V SoC, generated using the open-source LibreLane and OpenROAD toolchain with the SkyWater 130nm PDK. The design passed all sign-off DRC and LVS checks.	125

LIST OF TABLES

1.1	A Survey of Major Open-Source Soft-Core Families	3
1.2	Comparative Analysis of Major Open-Source SoC Frameworks	6
2.1	RISC-V Standard 32-bit ABI Register Conventions (RV32I). [43] . . .	16
2.2	Common RISC-V Pseudo-instructions. [43]	17
2.3	Development status of extended RISC-V instruction sets. [43]	19
3.1	TileLink operational configurations. [55]	37
8.1	Summary of Gemmini performance speedup across key operational categories. Detailed results for each test case are provided in Appendix D.	105
8.2	Cycle breakdown for a single inference pass of MobileNetV1 on the Gemmini accelerator.	105
10.1	Post-Synthesis Cell Utilization for the RISC-V SoC.	120
D.1	Detailed Matrix Multiplication Performance	155
D.2	Detailed Convolution Performance	155
D.3	Detailed Convolution with Pooling Performance	156
D.4	Detailed MLP Model Performance	156

CHAPTER 1: INTRODUCTION

This thesis presents a study of the open-source RISC-V ecosystem, investigated through the deep-dive implementation and analysis of a specialized Deep Neural Network (DNN) accelerator. The work examines the capabilities of modern open-source hardware by focusing on a state-of-the-art systolic array, Gemmini, and its high-performance integration with a processor core via the Rocket Custom Coprocessor (RoCC) interface. The primary outcome is a complete, Linux-capable System-on-Chip (SoC) that serves as a tangible case study on advanced accelerator co-design and establishes a foundational platform for future research within the Gemmini DNN Accelerator.

1.1 Background and Motivation

The advancement of artificial intelligence has created an unprecedented demand for specialized computing architectures. However, this field exists within a processor landscape long dominated by proprietary Instruction Set Architectures. This paradigm imposes significant barriers to research, including high licensing costs and a lack of freedom to deeply customize core architectures. The emergence of the open-standard RISC-V ISA represents a fundamental shift, enabling a new era of permissionless innovation [1].

A key feature of RISC-V is its inherent extensibility, which reserves opcode space for custom instructions. This has catalyzed the development of sophisticated open-source ecosystems like the Chipyard framework, which not only provide processor core generators but also a library of advanced, research-grade accelerators. Among these, the Gemmini project from UC Berkeley stands out. Gemmini is not just an accelerator; it is a generator for high-performance systolic arrays, embodying years of research into efficient DNN computation.

Furthermore, Chipyard provides a unique mechanism for integrating such components: the Rocket Custom Coprocessor (RoCC) interface. Unlike standard, loosely-coupled memory-mapped peripherals, the RoCC interface is a profound hardware co-design methodology that tightly couples an accelerator to the CPU's own pipeline [2]. This creates a privileged, low-latency communication channel that is essential for high performance.

This combination presents a compelling opportunity for study. The central ques-

tion of this thesis is: How capable is the modern open-source RISC-V ecosystem at implementing and analyzing a state-of-the-art, tightly-coupled hardware accelerator? This work answers that question by conducting an end-to-end implementation of the Gemmini accelerator on a Linux-capable SoC, providing a deep analysis of its architecture, integration, and performance.

1.2 The Open-Source Hardware Ecosystem

The processor landscape has historically been dominated by proprietary soft cores, such as the Xilinx MicroBlaze and Intel Nios II. These architectures, while well-established, operate within closed ecosystems, tightly coupling designs to a specific vendor’s toolchain and FPGA family [3, 4]. In parallel, the open-source hardware movement has now matured into a powerful and credible force, providing a spectrum of permissively licensed alternatives [5]. This paradigm shift unlocks new levels of flexibility and control, freeing designs from vendor dependency and enabling a more transparent and extensible path for hardware development [6]. For academic institutions, this signals a strategic moment: it is time to engage with these emerging platforms to build the foundation for the next generation of custom computing research.

1.2.1 A Survey of Open-Source Soft Cores

The open-source landscape includes a diverse range of soft-core processors spanning multiple Instruction Set Architectures (ISAs). While legacy ISAs like OpenRISC and SPARC have established communities, the RISC-V ISA has seen the most rapid growth and diversification, offering solutions from ultra-low-power microcontrollers to Linux-capable, out-of-order application processors.

Table 1.1: A Survey of Major Open-Source Soft-Core Families

Family	ISA	Key Traits	Notable Users/ Boards
PicoRV32 [7]	RISC-V RV32IMC	Ultra-small footprint (750 - 2,000 LUTs); configurable for size/performance; used as auxiliary processor	Lattice iCE40 boards, Digilent Cmod A7

Table 1.1 continued from previous page

Family	ISA	Key Traits	Notable Users/Boards
VexRiscv [8] (VexiRiscv is in-progress)	RISC-V RV32I[M][A][F[D]][C]	Highly scalable Scala/Spinal-HDL generator; from tiny MCUs to Linux-capable SMPs; plug-in based configurable pipeline	LiteX SoC Builder, various Linux demos [9], Nordic nRF Connect SDK, RIOT OS [10]
Ibex [11]	RISC-V RV32IMCB	2-stage in-order, area-optimized, production-quality, formally-verified core from lowRISC [12]	OpenTitan Root of Trust, Google Open Source Silicon [13]
CV32E40P [14]	RISC-V RV32IM [F Zfinx]C	4-stage in-order, high-performance core with PULP extensions for embedded AI/DSP; OBI memory interface	PULP Platform, GreenWaves GAP-8, CORE-V MCU
Rocket [15]/BOOM [16]	RISC-V 32/64-bit	High-performance application processor generators; Rocket (5-stage in-order), BOOM (out-of-order); Linux-capable	SiFive Commercial Cores, Chipyard, many academic/industrial projects; BROOM Chip [17]
OpenRISC 1200 [18]	OpenRISC [19]	Legacy 5-stage RISC with MMU, caches, DSP features; established community; Verilog implementation	Samsung Set-Top Boxes [20], various FPGA implementations (e.g., ORPSoC)

Table 1.1 continued from previous page

Family	ISA	Key Traits	Notable Users/Boards
LEON3 [21]	SPARC V8	7-stage, configurable, fault-tolerant (FT versions), space-grade	ESA/NASA Satellites (e.g., GR712RC), various space missions
LEON5 [22]	SPARC V8	Advanced dual-issue pipeline; high-performance; fault-tolerant (commercial version); enhanced branch prediction	In future GR765 SoC; Xilinx KCU105, Microchip PolarFire Splash Kit (evaluation)
NOEL-V [23]	RISC-V RV64GC	Dual-issue, 64-bit, high-performance, fault-tolerant (commercial version), targets space applications	Microchip PolarFire Splash Kit, Xilinx KCU105, Digilent Arty-A7 (evaluation); gaining adoption in space ASICs

1.2.2 The Strategic Advantage of RISC-V

Among the available open-source ISAs, RISC-V stands out due to its permissive, royalty-free licensing, modular extension scheme, and rapidly expanding commercial and academic ecosystem. This momentum is confirmed by market analysis, which forecasts RISC-V CPU IP revenue to grow from \$156 million in 2023 to \$1.6 billion by 2030 [24]. This strong industry adoption justifies the selection of RISC-V as a stable and future-proof foundation for this thesis and for CESLAB’s future research. However, the ecosystem is not without challenges, including a risk of fragmentation and the need for robust verification methodologies, which shifts the validation burden to the implementer [25, 26, 27, 28].

1.3 A Survey of Modern Open-Source SoC Frameworks

Having selected RISC-V as the target ISA, the next step is to choose a framework for building a complete System-on-Chip. We conducted a survey of prominent open-source SoC design frameworks, with the key differences summarized in Table 1.2.

Table 1.2: Comparative Analysis of Major Open-Source SoC Frameworks

	Chipyard	LiteX	PULP Platform	OpenTitan
Primary Goal	Research-class heterogeneous Linux-capable SoCs	Fast, flexible FPGA SoC prototyping	Energy-efficient MCU/AI cluster ASICs	High-assurance, transparent silicon Root of Trust (RoT)
Integration Style	Parameterized generators with Diplomacy bus negotiation	Python DSL for wiring Wishbone/AXI cores	Pre-built, optimized subsystems (e.g., cluster, uDMA)	Generation of secure subsystems from hardened, "com-portable" IPs
ASIC Flow	Mature, integrated flow via Hammer + OpenROAD [29]	SKY130 Backends in active development [30]	Multiple proven tape-outs on GF12/TSMC16/Intel 16 [31]	Silicon-proven RoT design ('Earl Grey' Chip [32]); commercially produced [33]
Accelerator Hooks	Tightly-coupled (RoCC) and loosely-coupled (TileLink MMIO)	Standard bus bridges (Wishbone/AXI)	Native Tightly-Coupled Data Memory (TCDM) ports for HWPEs	Integrated cryptographic co-processors (AES, KMAC, OTBN) as core IP
Learning Curve	Steep (Scala/Chisel)	Moderate (Python)	Moderate (SystemVerilog)	High (SystemVerilog + complex security principles)

1.3.1 LiteX: The Flexible SoC Builder

LiteX is a versatile framework that utilizes Migen, a Python-based hardware description language, to facilitate the rapid construction of custom SoCs [34, 35]. Its primary strength lies in its flexibility, offering a vast library of general-purpose IP and support for over 150 FPGA boards from a wide range of vendors [36]. While LiteX provides an exceptional environment for rapid FPGA prototyping and experimentation, its toolchain is not inherently optimized for a direct, end-to-end VLSI design flow in the same way as other frameworks. For our laboratory's specific goal of a seamless transition from FPGA prototyping to ASIC implementation, LiteX presented a less direct path.

1.3.2 PULP Platform: The Low-Power Parallel Computing Specialist

The PULP (Parallel Ultra-Low-Power) Platform, developed at ETH Zurich, is a leader in energy-efficient, parallel computing for edge AI and IoT applications [37]. The platform is built around a family of highly-optimized RISC-V cores and has been validated through an impressive 51 silicon tape-outs, demonstrating its maturity and industrial relevance [31]. However, the entire PULP architecture is meticulously optimized for its specific domain of ultra-low-power, multi-core data processing. Our project required a general-purpose SoC architecture capable of integrating a large, tightly-coupled, memory-intensive DNN accelerator, a different architectural paradigm than PULP’s primary focus.

1.3.3 OpenTitan: The Security-First Root of Trust

OpenTitan is a high-profile, open-source project managed by lowRISC to create a transparent and verifiable silicon "root of trust" (RoT) [13]. The framework, backed by a coalition including Google, provides a comprehensive suite of security-hardened IP, such as cryptographic accelerators and secure boot managers, built around the Ibex RISC-V core [38]. While OpenTitan represents a landmark achievement in open-source hardware security, its purpose is highly specialized. It is engineered to provide verifiable security, not high-performance computation. As our thesis is focused on DNN acceleration research, the OpenTitan framework was not a suitable foundation.

1.4 Chipyard: the framework with Full-Stack Integration Capability

Among the various RISC-V SoC design frameworks, Chipyard has emerged as the most comprehensive and mature solution for academic research and industrial VLSI development. Developed at UC Berkeley, Chipyard provides a complete, open-source SoC design environment that spans from high-level hardware generators written in Chisel to full VLSI implementation flows optimized for ASIC fabrication. This end-to-end capability is precisely what CESLAB requires to transition from FPGA-based prototyping to actual chip development.

The framework’s advantages over traditional FPGA design flows is evident in several key areas. Unlike the proprietary Avalon Bus used in Intel’s ecosystem, Chipyard leverages industry-standard interconnects including TileLink and AXI4. AXI4, in particular, has become the de facto standard for modern SoC design due to its superior performance characteristics, extensive tool support, and widespread adop-

tion across the semiconductor industry. This adherence to industry standards ensures that designs developed using Chipyard can seamlessly integrate with commercial IP cores and industry-standard VLSI tools.

Most critically, Chipyard includes a fully integrated, open-source VLSI flow that encompasses synthesis, place-and-route, and timing closure using industry-standard tools such as Cadence Genus and Innovus. This VLSI-optimized design flow enables the direct translation of high-level hardware descriptions into fabrication-ready layouts, supporting multiple process technologies including advanced nodes from foundries like TSMC and GlobalFoundries. This capability is essential for CESLAB’s strategic goal of developing chip prototypes and eventual commercial silicon products.

A key innovation of Chipyard is its emphasis on *full-stack integration*. Unlike traditional approaches that design and evaluate accelerators in isolation, Chipyard enables the creation of complete, Linux-capable systems that capture real-world effects such as operating system overhead, memory hierarchy interactions, and system-level resource contention. This holistic approach is crucial for understanding the true performance characteristics of specialized accelerators when deployed in practical computing environments—insights that are invaluable when transitioning from FPGA prototypes to optimized ASIC implementations.

The framework includes the Gemmini accelerator generator, an open-source DNN accelerator specifically designed for systolic array-based matrix computations. Gemmini embodies the principles of efficient DNN acceleration through its configurable systolic array architecture, which maximizes data reuse and minimizes the costly movement of data between computational units and external memory—a fundamental challenge known as the "memory wall" in computer architecture.

1.5 Core Selection within Chipyard: A Comparison of Rocket and BOOM

A key advantage of the Chipyard framework is that it is a generator capable of instantiating different processor cores. This presented a critical design choice for our system: selecting the CPU core that best aligned with the project’s objectives. The two primary candidates developed at UC Berkeley and available within Chipyard are the Rocket Core and the Berkeley Out-of-Order Machine (BOOM).

- The Rocket Core is a mature, stable, and widely-used 5-stage, single-issue, in-order processor core. It is highly configurable and provides a solid foundation for general-purpose computing, capable of booting full operating systems like

Linux. Its relative simplicity and extensive documentation make it a reliable and robust choice for systems where the primary focus is on integrating other components, such as custom accelerators [39].

- The Berkeley Out-of-Order Machine (BOOM) is a synthesizable, parameterized, out-of-order RISC-V core designed for high-performance applications [40, 41]. It features a much more complex microarchitecture than Rocket, with a deeper pipeline and sophisticated hardware for dynamic scheduling, register renaming, and speculative execution. BOOM is designed to compete with high-performance commercial cores, targeting performance levels suitable for desktop or server-class applications [42].

For this thesis, the Rocket Core was undoubtedly the correct choice. While BOOM offers superior standalone CPU performance, our project’s primary goal was to design an SoC, integrate, and evaluate a specialized DNN accelerator (Gemmini). The vast majority of the performance uplift for our target workload was expected to come from the accelerator, not the host CPU. Therefore, the selection criteria for the CPU core prioritized stability, reliability, and ease of integration over raw CPU performance.

1.6 Research Objectives and Contributions

This thesis aims to investigate the open-source RISC-V ecosystem by focusing on the implementation and analysis of the Gemmini DNN accelerator and its RoCC integration. The specific objectives are:

1. **Study of Advanced Accelerator Architecture:** To conduct a detailed study of the Gemmini systolic array generator, including its internal microarchitecture, command-unrolling hierarchy, decoupled execution model, and flexible dataflow strategies.
2. **Investigation of Co-Design Methodology:** To analyze the Rocket Custom Coprocessor (RoCC) interface as a high-performance integration methodology, contrasting it with standard memory-mapped approaches and detailing its impact on the CPU pipeline.
3. **Full-Stack System Implementation:** To implement a complete SoC featuring a 64-bit Rocket core tightly coupled with a 16×16 Gemmini accelerator,

demonstrating the practical application of the Chipyard framework for complex, heterogeneous systems.

4. **System Verification and Performance Analysis:** To verify the complete system by deploying it on a Xilinx VC707 FPGA and booting a Linux OS. Subsequently, to conduct a quantitative performance analysis to measure the speedup provided by the Gemmini accelerator.
5. **Documentation and Knowledge Transfer:** To produce comprehensive documentation and a reproducible platform for future research into advanced hardware accelerator design and co-design techniques within the RISC-V ecosystem.

1.7 Significance and Impact

The primary significance of this work is its contribution as a comprehensive, practical case study validating the maturity of the open-source RISC-V ecosystem for designing complex, high-performance systems. By successfully implementing a complete, functioning SoC using a fully open-source toolchain, this thesis provides tangible evidence that modern open-source hardware methodologies are a viable and powerful alternative to traditional, proprietary design flows.

Furthermore, this work provides one of the first detailed, public-facing analyses of the Gemmini DNN accelerator’s internal microarchitecture. While foundational papers from its creators at UC Berkeley describe its high-level design and performance, this thesis elucidates the step-by-step operational flow of a command through its internal hardware. This includes a detailed breakdown of the two-level command unrolling hierarchy, the decoupled execution model managed by the Reservation Station, and the physical Tile-and-Mesh structure. This in-depth documentation of a state-of-the-art, open-source accelerator is a significant contribution to the research community, making this complex and powerful IP more accessible for future adoption, modification, and study.

This thesis also serves as a detailed implementation roadmap for researchers and engineers considering the adoption of open-source hardware. The successful demonstration of the Chipyard framework, with its integrated support for modern interconnects and industry-standard EDA tools, confirms that the ecosystem is capable of producing sophisticated designs suitable for both academic research and serious engineering development.

The comprehensive documentation and the reproducible platform generated by this thesis—now including the detailed Gemmini analysis—serve as an invaluable knowledge base for the research community. This material addresses the practical challenges of implementing complex SoC designs with open-source tools, offering detailed solutions and establishing best practices for future projects in this domain.

Most critically, the full-stack nature of this implementation provides valuable insights into the complex interactions between hardware accelerators, operating systems, and application software. These system-level insights—often overlooked in studies that focus on isolated hardware components—are crucial for understanding the real-world performance of specialized hardware. The findings from this work can inform the design of future, highly-optimized systems, whether they target FPGA platforms or custom ASIC implementations.

1.8 Thesis Structure and Organization

This thesis is organized into nine chapters that systematically present the foundational argument, technical background, design, implementation, and evaluation of the RISC-V SoC with a Gemmini DNN accelerator.

CHAPTER 2: RISC-V MICROPROCESSOR INSTRUCTION SET ARCHITECTURE provides the necessary technical background for this thesis. It begins with a comprehensive overview of the RISC-V ISA and its key architectural features. It then presents a detailed description of the Chipyard SoC design framework, the central development platform used in this work.

CHAPTER 3: A SINGLE-CORE RISC-V PROCESSOR SYSTEM WITH A GEMMINI ROCC ACCELERATOR introduces the specific architecture of the system designed and implemented in this project. It details the high-level system structure, the role of Chisel/Scala in hardware design, and the overall methodology for constructing our custom SoC within the Chipyard framework.

CHAPTER 4: ARCHITECTURAL FOUNDATIONS OF DNN ACCELERATION explores the fundamental challenges of Deep Neural Network acceleration, including the memory wall problem and the principles of systolic array architectures. This chapter provides the theoretical context for understanding why specialized accelerators like Gemmini are necessary for efficient DNN computation.

CHAPTER 5: THE GEMMINI DNN ACCELERATOR: AN ARCHITECTURAL OVERVIEW presents a detailed analysis of the Gemmini accelera-

tor’s high-level architecture, including its systolic compute engine, memory hierarchy, and integration model within the SoC framework.

CHAPTER 6: GEMMINI’S INTERNAL ARCHITECTURE AND PROGRAMMING MODEL performs a deeper dive into the microarchitectural details of Gemmini. It examines the internal command and control flow, the instruction processing pipeline that enables efficient DNN computation.

CHAPTER 7: SYSTEM IMPLEMENTATION AND TESTING documents the complete implementation process, from SoC configuration, integration of all system components, and hardware generation to FPGA deployment and system validation. This chapter provides practical insights into the entire development workflow and demonstrates the full-stack, open-source design process.

CHAPTER 8: FPGA IMPLEMENTATION RESULTS AND DISCUSSION presents the results of the project. It includes an analysis of hardware resource utilization, validates the successful boot of a full Linux operating system, and provides a quantitative performance evaluation of the Gemmini accelerator compared to a software-only implementation.

CHAPTER 11: CONCLUSION AND FUTURE WORK summarizes the key achievements of this thesis, evaluates the success of the implementation against the stated objectives, acknowledges the limitations of this work, and outlines potential directions for future research and development.

This structured approach ensures a comprehensive understanding of both the theoretical principles underlying modern SoC design and the practical challenges involved in implementing complex, multi-component systems using open-source tools and methodologies.

CHAPTER 2: RISC-V MICROPROCESSOR INSTRUCTION SET ARCHITECTURE

2.1 Overview of RISC-V

Designing and developing a microprocessor, or Central Processing Unit (CPU), is a complex process requiring the collaboration of many experts in fields such as digital logic design, compiler development, and operating systems, along with significant financial resources. Major vendors like ARM, with prominent processor lines such as Cortex-M and Cortex-A, often impose high licensing fees and strict confidentiality requirements on their designs. This creates considerable barriers to hardware and software research and customization, particularly for academic institutions or small businesses. In response to these limitations, the RISC-V architecture [43, 44, 1] was developed and introduced as an open-source solution, providing a flexible platform that eliminates licensing costs and allows for need-based customization. By fostering innovation and opening up a freer research ecosystem, RISC-V is reshaping the traditional approach to microprocessor development, aiming for more efficient and comprehensive solutions.



Figure 2.1: RISC-V is managed by RISC-V International. Source [45]

Essentially, RISC-V is an open-source microprocessor instruction set architecture (ISA), designed and developed based on the Reduced Instruction Set Computer (RISC) methodology, and it represents the fifth generation of this design approach. For a program executing on a microprocessor, the RISC design methodology stipulates that the microprocessor executes only simple assembly (machine code) instructions, which have a fixed length and are executed within a single, uniform clock cycle. A complex task is performed by multiple assembly instructions, making the instruction set simple, compact, and easy to develop hardware for rapidly. This is entirely different from the Complex Instruction Set Computer (CISC) architecture, which aims to complete a task with the fewest possible assembly instructions. To achieve this, CISC microprocessors must be designed to execute structurally complex assembly instructions, capable of performing multiple specialized functions sequentially and executing over multiple clock cycles.

Originating as a project developed by researchers and graduate students at the University of California, Berkeley in 2010, the initial purpose of RISC-V was to create a practical ISA to support research and teaching in computer architecture, deployable on both hardware and software without licensing requirements. The original authors of RISC-V were individuals with considerable experience in computer design. Consequently, for stable development, RISC-V aimed for several goals, most of which have been achieved, including:

- A complete, fully open-source ISA, free for academic, educational, and industrial use, with practical applicability suitable not only for experimental simulation but also for hardware implementation.
- An ISA not oriented towards complex microarchitectures or fixed by any specific implementation technology (FPGA, ASIC, full-custom, etc.) but still allowing for efficient implementation in any of these directions.
- An ISA divided into base ISAs, usable for developing hardware accelerators or for educational purposes, and optional standard extensions to support general software development.
- An ISA supporting extensive instruction set extensions with specialized functionalities and numerous variants.
- Support for the IEEE 754-2008 floating-point standard.
- Support for parallel, multi-core, and heterogeneous many-core implementations.
- Both 32-bit and 64-bit address space variants developed for applications, operating system kernels, and hardware implementations.

Due to its open-source nature, RISC-V allows developers to freely use the design architecture without paying intellectual property and licensing fees. This creates development opportunities for RISC-V itself as it is adopted by the research community and educational institutions, gradually becoming popular in industry, and attracting significant attention from many large companies, corporations, and even governments of various countries. Furthermore, designers using the RISC-V ISA can utilize it without limitations in both hardware and software. This means developers are encouraged, but not required, to publicly disclose their designs and projects developed using RISC-V.

Currently, the development process and rights regarding the publication, release, and maintenance of intellectual property and official documentation related to RISC-V are managed by RISC-V International, based in Switzerland. This meets the standardization requirements for designers and users for commercial purposes, who demand a system using the RISC-V ISA that can operate and exist for many years.

To suit various implementers and ensure flexibility in designs with specialized characteristics, the RISC-V instruction set is divided and standardized into two main parts: the base instruction set and standard extensions. The RISC-V instruction set supports 32-bit, 64-bit, and 128-bit architectures with the respective base instruction sets RV32I, RV32E, RV64I, and RV128I. Among these, RV32I is the most important and common set for 32-bit microprocessors, featuring 40 integer manipulation instructions. The fact that RISC-V only standardizes and provides the instruction set allows hardware designers the freedom to choose how to implement their designs.

2.1.1 RISC-V 32-bit and the RV32I Base Instruction Set

RV32I [43] is the base instruction set present in almost all 32-bit RISC-V microprocessor architectures. The assembly instructions in this set use 32 32-bit registers, named `x0` - `x31`, to store integers. Each register, in addition to being numbered from 0 to 31, is also referred to by its Application Binary Interface (ABI) name, which reflects its specific function. For example, register `x0` is a read-only register containing the constant value "0", and register `x1` is used as the return address register, storing the return address when a function is executed. Table 2.1 lists these registers and their functions in detail.

The assembly instructions in RV32I use six different 32-bit instruction formats, depending on the function of the instruction. These include R, I, S, U formats, as well as two additional formats for immediate values: B and J. These formats are clearly shown in Figure 2.2. Notably, the source registers (`rs1` and `rs2`) and the destination register (`rd`) are kept in the same position in most formats, simplifying instruction decoding. The only discernible difference lies in how `rs2`, `rd`, and immediate values are organized within the instruction.

Figure 2.3 details all instructions in the RV32I base instruction set with their respective formats, specific opcode values, and function codes (`funct3` and `funct7`). Functionally, the RV32I set is divided into five distinct groups:

- 21 integer computation instructions: Perform arithmetic (addition, subtraction, bit shifts), logic (AND, OR, XOR, NOT, etc.), and comparison operations

Table 2.1: RISC-V Standard 32-bit ABI Register Conventions (RV32I). [43]

Register	ABI Name	Description
x0	zero	Constant "0"
x1	ra	Return address
x2	sp	Stack pointer
x3	gp	Global pointer
x4	tp	Thread pointer
x5 $\rightarrow$ x7	t0 $\rightarrow$ t2	Temporary registers
x8	s0/fp	Memory/frame pointer
x9	s1	Saved register
x10, x11	a0, a1	Function parameter/return value
x12 $\rightarrow$ x17	a2 $\rightarrow$ a7	Function parameter
x18 $\rightarrow$ x27	s2 $\rightarrow$ s11	Saved register
x28 $\rightarrow$ x31	t3 $\rightarrow$ t6	Temporary registers

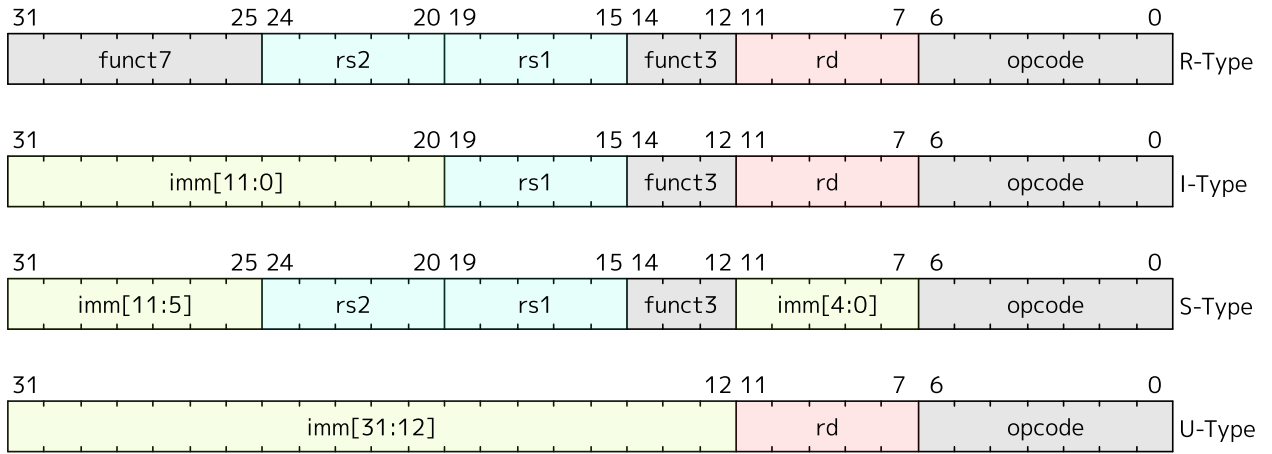


Figure 2.2: Instruction encoding formats in RV32I. Source [43]

on signed and unsigned numbers. Typically use R-type and I-type formats.

- 8 load-store instructions: LOAD and STORE byte, half-word, word (signed and unsigned), typically using I-type and S-type formats.
- 8 control-flow/branch instructions: Conditional branches, performing comparisons between two registers to decide whether to branch (BEQ/BNE, BLT, BGE), typically using I-type format.
- 2 system access instructions for the execution environment - ECALL and breakpoint - EBREAK, using I-type format.
- 1 memory access ordering instruction - FENCE, used by other processing cores

(in multi-core scenarios) or other peripheral devices in the system. The I-type format is used for this instruction with some specific bit encoding rules and opcodes.

With these 40 instructions, the microprocessor can perform most basic operations on integers. Additionally, several standard pseudo-instructions are defined to add tasks for which there are no dedicated instructions in the set. For example, the MOVE instruction - "mv rd, rs" - performs the function of moving/copying data between two registers. In a program, this instruction might be generated in the assembly code by a RISC-V compiler but is actually converted by assemblers into a standard instruction when translating to machine code, namely ADDI - Add Immediate "addi rd, rs, 0", which adds the source register to the constant "0" and assigns it to the destination register, achieving a similar function. Some other pseudo-instructions and their equivalent standard instructions are shown in Table 2.2.

Table 2.2: Common RISC-V Pseudo-instructions. [43]

Pseudo-instruction	Corresponding Base Instruction	Meaning
nop	addi x0, x0, 0	No operation
li rd, imm	addi rd, x0, imm	Load immediate
mv rd, rs	addi rd, rs, 0	Move register (copy)
not rd, rs	xori rd, rs, -1	One's complement (bitwise NOT)
neg rd, rs	sub rd, x0, rs	Two's complement (negate)
seqz rd, rs	sltiu rd, rs, 1	Set if equal to zero
snez rd, rs	sltu rd, x0, rs	Set if not equal to zero
sltz rd, rs	slt rd, rs, x0	Set if less than zero
sgtz rd, rs	slt rd, x0, rs	Set if greater than zero
beqz rs, offset	beq rs, x0, offset	Branch if equal to zero
bnez rs, offset	bne rs, x0, offset	Branch if not equal to zero
blez rs, offset	bge x0, rs, offset	Branch if less than or equal to zero
bgez rs, offset	bge rs, x0, offset	Branch if greater than or equal to zero
bltz rs, offset	blt rs, x0, offset	Branch if less than zero
bgtz rs, offset	blt x0, rs, offset	Branch if greater than zero
j offset	jal x0, offset	Jump to label (offset)
jal offset	jal x1, offset	Jump and link (call)
ret	jalr x0, 0(x1)	Return from subroutine

As mentioned, a goal of RISC-V is to support implementations requiring more

RV32I Base Instruction Set							
imm[31:12]					rd	0110111	LUI
imm[31:12]					rd	0010111	AUIPC
imm[20 10:1 11 19:12]					rd	1101111	JAL
imm[11:0]			rs1	000	rd	1100111	JALR
imm[12 10:5]	rs2	rs1	000	imm[4:1 11]	1100011	BEQ	
imm[12 10:5]	rs2	rs1	001	imm[4:1 11]	1100011	BNE	
imm[12 10:5]	rs2	rs1	100	imm[4:1 11]	1100011	BLT	
imm[12 10:5]	rs2	rs1	101	imm[4:1 11]	1100011	BGE	
imm[12 10:5]	rs2	rs1	110	imm[4:1 11]	1100011	BLTU	
imm[12 10:5]	rs2	rs1	111	imm[4:1 11]	1100011	BGEU	
imm[11:0]			rs1	000	rd	0000011	LB
imm[11:0]			rs1	001	rd	0000011	LH
imm[11:0]			rs1	010	rd	0000011	LW
imm[11:0]			rs1	100	rd	0000011	LBU
imm[11:0]			rs1	101	rd	0000011	LHU
imm[11:5]	rs2	rs1	000	imm[4:0]	0100011	SB	
imm[11:5]	rs2	rs1	001	imm[4:0]	0100011	SH	
imm[11:5]	rs2	rs1	010	imm[4:0]	0100011	SW	
imm[11:0]			rs1	000	rd	0010011	ADDI
imm[11:0]			rs1	010	rd	0010011	SLTI
imm[11:0]			rs1	011	rd	0010011	SLTIU
imm[11:0]			rs1	100	rd	0010011	XORI
imm[11:0]			rs1	110	rd	0010011	ORI
imm[11:0]			rs1	111	rd	0010011	ANDI
0000000	shamt	rs1	001	rd	0010011	SLLI	
0000000	shamt	rs1	101	rd	0010011	SRLI	
0100000	shamt	rs1	101	rd	0010011	SRAI	
0000000	rs2	rs1	000	rd	0110011	ADD	
0100000	rs2	rs1	000	rd	0110011	SUB	
0000000	rs2	rs1	001	rd	0110011	SLL	
0000000	rs2	rs1	010	rd	0110011	SLT	
0000000	rs2	rs1	011	rd	0110011	SLTU	
0000000	rs2	rs1	100	rd	0110011	XOR	
0000000	rs2	rs1	101	rd	0110011	SRL	
0100000	rs2	rs1	101	rd	0110011	SRA	
0000000	rs2	rs1	110	rd	0110011	OR	
0000000	rs2	rs1	111	rd	0110011	AND	
fm	pred	succ	rs1	000	rd	0001111	FENCE
1000	0011	0011	00000	000	00000	0001111	FENCE.TSO
0000	0001	0000	00000	000	00000	0001111	PAUSE
000000000000			00000	000	00000	1110011	ECALL
000000000001			00000	000	00000	1110011	EBREAK

Figure 2.3: Instructions in the RV32I base instruction set. [43]

features beyond the base instruction set. This is reflected in RISC-V defining extensions [43] for its instruction set architecture. These extensions define additional instruction encodings for various functionalities. For example, the standard M extension adds instructions for integer multiplication and division, while the standard F extension defines instructions for single-precision floating-point computation and manipulation.

Table 2.3: Development status of extended RISC-V instruction sets. [43]

Instruction Set	Function Description	Version	Status
M	Integer multiplication and division instruction set	2.0	Approved
A	Atomic memory operation instruction set	2.1	Approved
F	Single-precision floating-point computation	2.2	Approved
D	Double-precision floating-point computation	2.2	Approved
Q	Quad-precision floating-point computation	2.2	Approved
C	Compressed instruction set	2.0	Approved
Zifence	Instruction-Fetch Fence instruction set	2.0	Approved
Zicsr	Control and Status Registers (CSRs) instruction set	2.0	Approved
Ztso	Total Store Ordering instruction set	1.0	Approved

Table 2.3 above clearly lists the standard extension sets developed and published by RISC-V International. It can be seen that many instruction sets have been ratified and can be used in designs. When implemented, architectures supporting the RV32I base instruction set (or RV64I depending on the architecture) combined with the standard extensions M, A, F, D, Zicsr, and Zifencei are often summarized and referred to as the RV32G (or RV64G) instruction set, named for its general-purpose utility.

2.1.2 RISC-V 64-bit and the RV64I Base Instruction Set

Similar to RV32I, RV64I is the base instruction set for 64-bit microprocessor architectures, featuring 64-bit wide registers and supporting 64-bit addressing (XLEN=64), though instructions remain 32-bits wide. Architectures compliant with RV64I are compatible with RV32 instructions and include additional instructions to support operations between 32-bit and 64-bit data. For example, the LW instruction loads a 32-bit value and sign-extends it into a 64-bit register, while the new LD instruction loads a 64-bit word. The ADD instruction now performs 64-bit addition, while the ADDW instruction handles 32-bit addition, ignoring the upper 32 bits and returning a 32-bit signed value, sign-extended to 64 bits. These instructions enabling backward compatibility typically have a "W" suffix to denote 32-bit wide operations. These

instructions, along with new 64-bit load and store operations, are summarized in Figure 2.4.

RV64I Base Instruction Set (in addition to RV32I)						
imm[11:0]		rs1	110	rd	0000011	LWU
imm[11:0]		rs1	011	rd	0000011	LD
imm[11:5]	rs2	rs1	011	imm[4:0]	0100011	SD
000000	shamt	rs1	001	rd	0010011	SLLI
000000	shamt	rs1	101	rd	0010011	SRLI
010000	shamt	rs1	101	rd	0010011	SRAI
imm[11:0]		rs1	000	rd	0011011	ADDIW
0000000	shamt	rs1	001	rd	0011011	SLLIW
0000000	shamt	rs1	101	rd	0011011	SRLIW
0100000	shamt	rs1	101	rd	0011011	SRAIW
0000000	rs2	rs1	000	rd	0111011	ADDW
0100000	rs2	rs1	000	rd	0111011	SUBW
0000000	rs2	rs1	001	rd	0111011	SLLW
0000000	rs2	rs1	101	rd	0111011	SRLW
0100000	rs2	rs1	101	rd	0111011	SRAW

Figure 2.4: Instructions in the RV64I base instruction set. [43]

The compiler and calling conventions maintain the invariant that all 32-bit values are stored in sign-extended format within 64-bit registers. Even 32-bit unsigned integers will have bit 31 extended into bits 63 through 32. Consequently, converting between 32-bit signed and unsigned integers is a no-operation, as is converting from a 32-bit signed integer to a 64-bit signed integer. Existing 64-bit instructions like SLTU (set less than unsigned) and branch comparisons continue to operate correctly on 32-bit unsigned integers under this rule. Similarly, 64-bit logical operations on 32-bit sign-extended integers preserve the sign-extension property. A few new instructions (ADD[I]W/SUBW/SxxW) are necessary for addition and shifts to ensure reasonable performance for 32-bit values.

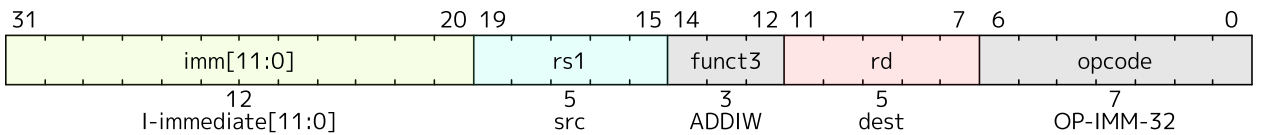


Figure 2.5: Instruction encoding format for ADDIW in 64-bit architecture. [43]

Some distinctly different instructions found only in RV64I include the ADDIW instruction, as shown in Figure 2.5. ADDIW performs an addition between a 12-bit sign-

extended immediate value and register **rs1**, producing a proper 32-bit sign-extended result in **rd**. Overflows are ignored, and the result consists of the lower 32 bits of the sum, sign-extended to 64 bits. Note, the instruction "**addiw rd, rs1, 0**" writes the sign-extension of the lower 32 bits of register **rs1** into register **rd** (pseudo-instruction **SEXT.W**).

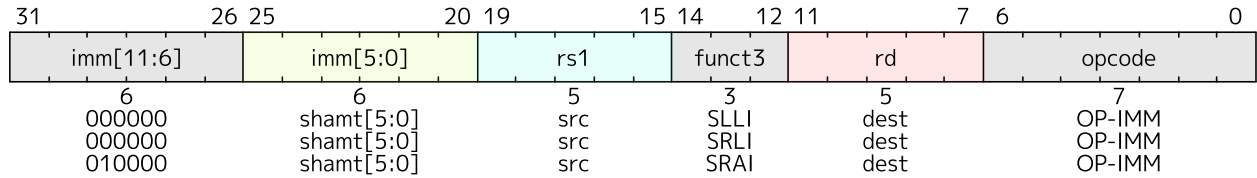


Figure 2.6: Encoding format for 64-bit I-type shift instructions. [43]

I-type shift operations in RV64I (Figure 2.6) use the same opcode as RV32I with some modifications. Register **rs1** contains the operand, and the shift amount is encoded in the lower 6 bits of the I-immediate field. Bit 30 determines the shift type: **SLLI** (logical left shift, zeros shifted into lower bits), **SRLI** (logical right shift, zeros shifted into upper bits), and **SRAI** (arithmetic right shift, sign bit copied into vacated upper bits).

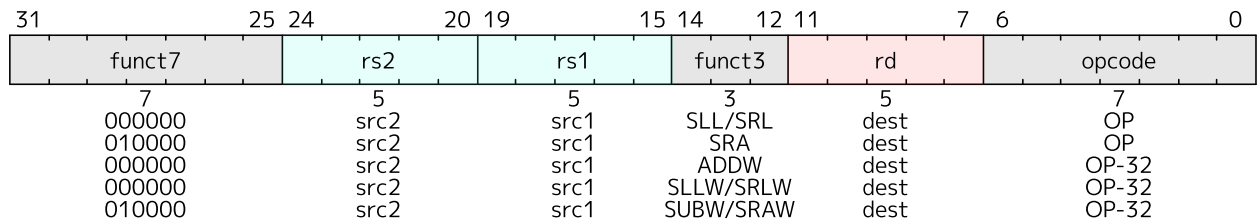


Figure 2.7: Encoding format for 64-bit R-type shift instructions for word operations. [43]

The **ADDW** and **SUBW** instructions in RV64I, similar to **ADD** and **SUB** for 64-bit operations, operate on 32-bit data and produce a 32-bit signed result, which is then sign-extended to 64-bits before being written to the destination register. Overflows are also ignored. The **SLL**, **SRL**, and **SRA** instructions perform logical left, logical right, and arithmetic right shifts on the value in **rs1**, with the shift amount determined by the lower 6 bits of **rs2**. The **SLLW**, **SRLW**, and **SRAW** instructions (Figure 2.7), unique to RV64I, operate on 32-bit values and use the lower 5 bits of **rs2** to determine the shift amount.

Finally, load-store instructions can be mentioned. RV64I extends the address space to 64 bits, but the number of accessible addresses may vary depending on the execution environment.

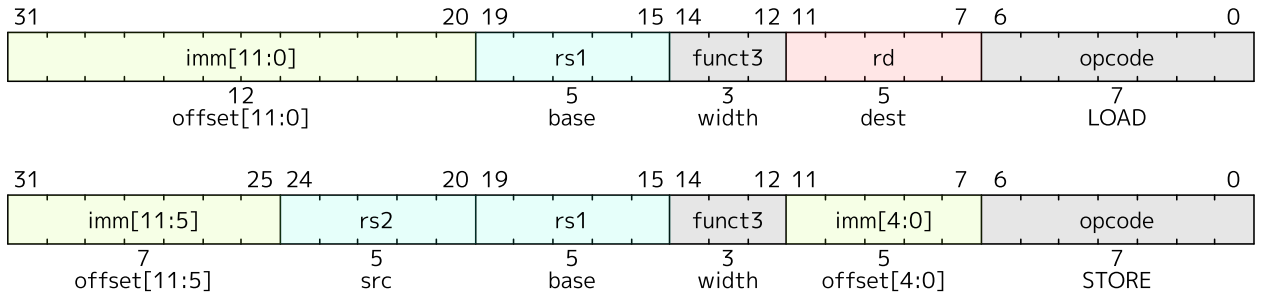


Figure 2.8: Encoding formats for 64-bit LOAD and STORE instructions. [43]

The instruction formats are shown in Figure 2.8. The LD instruction here will load a 64-bit value, LW loads a 32-bit value (sign-extended to 64-bits) from memory into a register, and LWU zero-extends a 32-bit value from memory into register `rd`. Similarly, LH/LHU and LB/LBU handle 16-bit and 8-bit values. The SD, SW, SH, and SB instructions store 64-bit, 32-bit, 16-bit, and 8-bit values from register `rs2` to memory, respectively.

2.2 RISC-V Processor Cores

With its open-source nature, system developers can freely implement designs based on the RISC-V architecture, supported by an increasingly robust and growing ecosystem. The RISC-V ISA ecosystem mentioned here includes a diverse range of components from hardware design tools, low-level firmware development, bootloaders, to fully functional operating system kernels. Currently, development tools such as GCC/LLVM compilers, QEMU and Verilator simulators, and even the Linux operating system have incorporated RISC-V architecture and are under active development. As of the time of writing this thesis, there are over 100 open-source hardware processor cores and more than 40 open-source SoC platforms using the RISC-V ISA that have been and continue to be developed.

This section will provide a general overview of pipeline, in-order, and out-of-order architectures commonly implemented in microprocessors. Subsequently, a brief survey of open-source RISC-V processor cores will be conducted to select the most suitable design for implementation.

2.2.1 Pipeline Architecture in Microprocessors

A modern general-purpose CPU is often designed using a pipeline architecture. This technique draws inspiration from industrial assembly lines, dividing the process-

ing of an assembly instruction into multiple stages or segments, each performed by different specialized hardware units connected sequentially throughout the process and capable of continuous operation. This architecture is commonly found in many microprocessor designs using RISC in general and RISC-V ISA in particular, due to its higher performance compared to processing instructions sequentially and waiting for results.

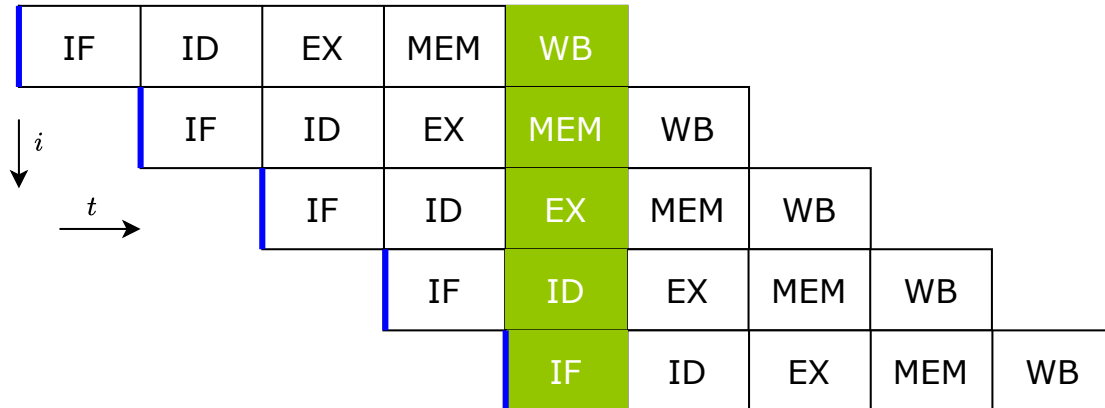


Figure 2.9: Basic five-stage pipeline in a RISC machine. The vertical axis is successive instructions; the horizontal axis is time. So in the green column, the earliest instruction is in WB stage, and the latest instruction is undergoing instruction fetch.

The concept of pipelining is illustrated in Figure 2.9. While one assembly instruction is being processed in later stages, another assembly instruction begins processing in earlier stages, in parallel with previous instructions. It can be seen that at the fourth clock cycle, the pipeline is processing four different instructions in parallel, each in a distinct stage. A basic microprocessor pipeline consists of five stages. However, depending on the implementation and design, a microprocessor may use more or fewer stages, but still follows the sequence of stages explained below:

- **Instruction Fetch (IF):** Retrieves instructions from the instruction memory, also known as program memory. Each instruction code that the CPU must execute is stored in the program memory. These instruction codes are binary sequences compiled from C/C++ code or assembly code.
- **Instruction Decode (ID):** The binary of each instruction is decoded, and corresponding control signals are activated.
- **Execute (EX):** Performs calculations and processing corresponding to the decoded instruction, such as addition, subtraction, multiplication, division, shifts, assignments, branches, etc.

- **Memory Access (MEM):** Accesses memory and performs read/write operations of data from/to memory.
- **Writeback (WB):** Saves the results after instruction execution to the location specified by the instruction, such as registers.

2.2.2 Out-of-Order Architecture in Microprocessors

Alternatively, out-of-order execution architecture is a crucial paradigm in modern CPU design, delivering significant performance gains by allowing instructions to be executed as soon as their input operands are ready, rather than strictly adhering to the program order as in a pipeline. This architecture is also a core element in high-performance systems used in scientific computing and artificial intelligence research. This flexible execution model addresses performance bottlenecks caused by data dependencies, cache latencies, and pipeline stalls, ensuring more efficient utilization of computational resources.

Key components of this architecture include the instruction window, which stores multiple instructions for dependency analysis, and dynamic scheduling mechanisms such as Tomasulo's algorithm or scoreboarding, which track dependencies and resource availability. Another critical feature is the Re-order Buffer (ROB), which helps maintain program order during instruction retirement, ensuring correct architectural state updates and precise exception handling. By decoupling instruction fetch and execution stages, out-of-order processors achieve higher instruction throughput and sustain the performance of execution units, optimizing efficiency across various applications.

Despite its many strengths and implementation benefits, out-of-order processors also present significant design challenges. The hardware blocks required for components like the ROB and branch prediction increase complexity, chip area, and power consumption, making energy efficiency optimization an important research area. Furthermore, extending out-of-order architecture to meet the demands of heterogeneous processing and large-scale parallel computing requires innovative approaches in dynamic scheduling and resource management.

2.2.3 Open-Source RISC-V Processor Cores

Rocket [39], or Rocket Core, is a RISC-V instruction set architecture microprocessor core designed by the Berkeley Architecture Research group. Rocket features

a 5-stage, in-order architecture and implements the RISC-V RV32G and RV64G instruction sets.

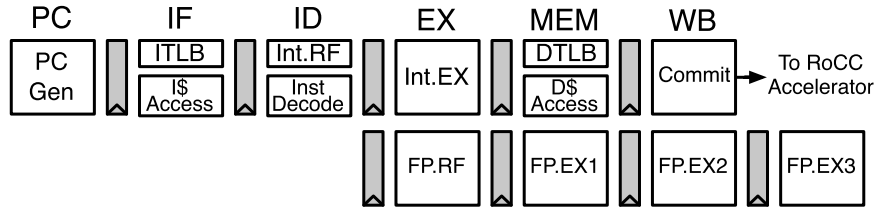


Figure 2.10: Pipeline of Rocket core. [39, 46, 47]

The design of this microprocessor integrates many advanced features, including a Memory Management Unit (MMU) that supports virtual memory, enabling efficient memory access and address translation. Besides its role as a general-purpose microprocessor, Rocket also serves as a library of reusable components, including functional units, caches, Translation Lookaside Buffer (TLB), Page Table Walker (PTW), and Control and Status Registers (CSRs), which are reused in many other microprocessor designs. This makes Rocket Core a foundational component in Chipyard - RocketChip, allowing for the rapid development and creation of experimental systems ranging from simple to complex.

Rocket is considered the most prominent microprocessor core in the open-source RISC-V development community due to its well-updated and stably supported development documentation. It is built using Chisel, a relatively new hardware construction language based on Scala, employing an object-oriented approach similar to software development. Therefore, the flexibility of the system development process and the applicability of Rocket Core are very high, making it suitable for implementing multi-core systems.

Along with Rocket Core, the Berkeley Out-of-Order Machine (BOOM) [48, 40, 41] is also a prominent open-source RISC-V core, designed for high performance and out-of-order execution. Also developed within the microprocessor architecture research group at the University of California, Berkeley, BOOM is aimed at applications in advanced academic research and industry, featuring a flexible design and high operational performance. BOOM is also written in the Chisel language as a generator, like Rocket Core, allowing for rapid deployment and customization of the core according to the RV64GC RISC-V architecture. The latest version, SonicBOOM [42] (also known as BOOMv3), achieves a performance of 6.2 CoreMarks/MHz, capable of competing directly with high-performance commercial processor cores, setting a

high standard for open-source processor design.

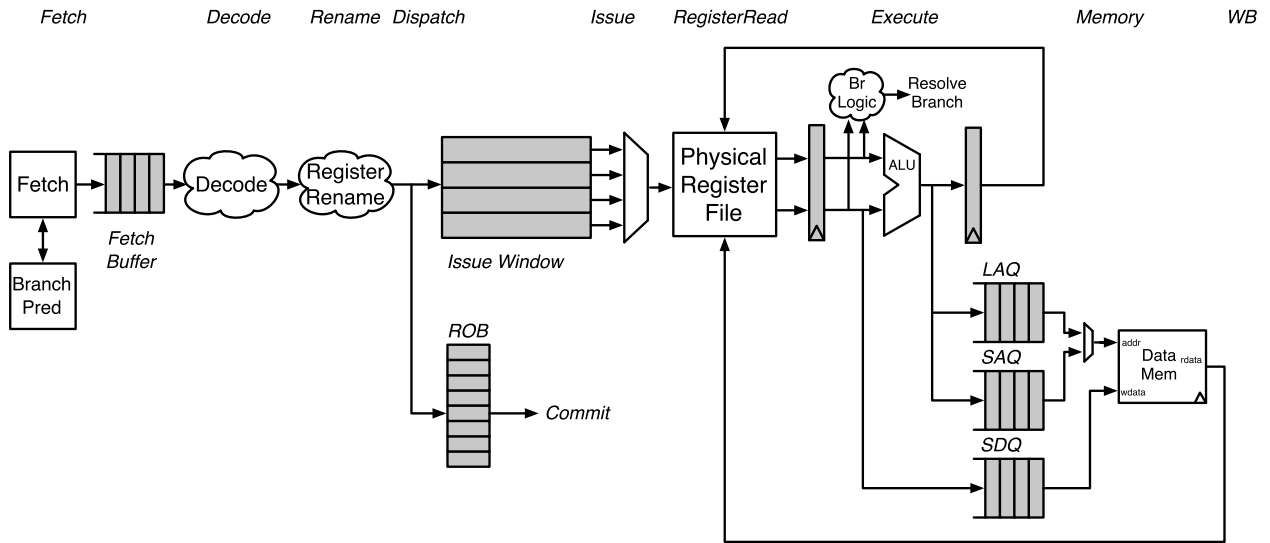


Figure 2.11: Berkeley Out-of-Order Machine Processor Pipeline Architecture. [41]

Structurally, BOOM features a 10-stage pipeline architecture including Fetch, Decode, Register Rename, Dispatch, Issue, Register Read, Execute, Memory, Writeback, and Commit. These stages allow for detailed control and optimization of the instruction flow, maintaining the "out-of-order" functionality at the high speeds necessary for modern processors. Specifically, each stage performs as follows:

- **Fetch:** Instructions are fetched from instruction memory and stored in a fetch buffer to prepare for processing.
- **Decode:** Instructions are converted into simpler micro-ops to optimize execution within the pipeline.
- **Register Rename:** Logical registers in the ISA are renamed to independent physical registers, reducing data conflicts and increasing processing efficiency.
- **Dispatch:** Micro-ops from the decode stage are sent to the Issue window, awaiting necessary operands.
- **Issue:** Instructions in the Issue window are checked for operand readiness, and when conditions are met, micro-ops are sent to an execution unit.
- **Register File (RF) Read:** Micro-ops read operands from the physical register file or bypass network, preparing data for the next stage.

- **Execute:** Micro-ops are processed by functional units, performing computational tasks and memory address calculations.
- **Memory:** Manages memory operations via Load Address Queue (LAQ), Store Address Queue (SAQ), and Store Data Queue (SDQ), performing loads when addresses are ready and stores when both address and data are available.
- **Writeback:** Results from operations are written back to the physical register file, ensuring updated values are stored for future instructions.
- **Commit:** The Reorder Buffer tracks the status of instructions in the pipeline and commits results when instructions complete, including storing data to memory.

Similar to Rocket Core, this is also considered one of the most prominent open-source microprocessor cores. However, BOOM’s design is more geared towards ASIC implementation, and its numerous features along with the use of Chisel for implementation might pose difficulties in creating a multi-core design in Verilog and deploying it on FPGAs.

2.3 System-on-Chip Design Framework: Chipyard - Rocket Chip

The Rocket Chip project, also developed by the Berkeley Architecture Research (BAR) group at the University of California, Berkeley, marked a significant advancement in open-source processor design. This project provides a flexible, scalable, and configurable platform, enabling the creation of custom RISC-V microprocessors tailored to diverse performance and application requirements. Rocket Chip is essentially a library of hardware generators that can be configured and interconnected in various ways, producing diverse SoC designs. This library can generate an in-order core (Rocket) or an out-of-order core (BOOM), and these cores can be connected to co-processors via an interface named Rocket Custom Coprocessor (RoCC) [39].



Figure 2.12: Chipyard, developed by UC Berkeley. [49]

Subsequently, to develop and expand the capabilities of Rocket Chip, Chipyard was created. Chipyard is an open-source framework that integrates system design, simulation, and implementation for SoC development, and it has become one of the most actively embraced and developed frameworks by the community. Chipyard, also originating from UC Berkeley, is the result of continuous development and integration of Chisel, Rocket Chip, and the Rocket Core.

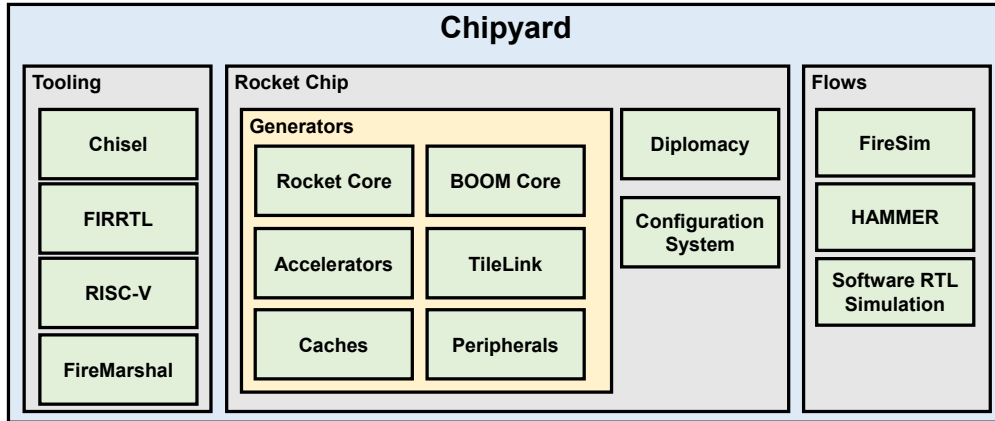


Figure 2.13: Component structure of Chipyard. [49]

By utilizing Chisel, the FIRRTL compiler, the Rocket Chip generator, and other tools, Chipyard facilitates the development of highly customizable RISC-V SoCs. Chipyard includes a diverse set of processor cores (Rocket Core, BOOM, Ariane), accelerators (Hwacha, Gemmini, SHA3), along with comprehensive memory systems, peripheral devices, and tools to support the construction of a full-featured SoC. Chipyard supports multiple development flows, allowing for flexible hardware extension and experimentation. These development flows include software-based Register Transfer Level (RTL) simulation, FPGA-accelerated simulation using FireSim [50], automated VLSI flows with Hammer, and FireMarshal, which enables the creation and loading of software on both bare-metal systems (similar to microcontrollers) and Linux-based systems.

Rocket Chip supports deployment on FPGAs, with operational frequencies reaching 25-100 MHz depending on the configuration and the type of FPGA used. Currently, Rocket Chip is being utilized by SiFive [51], a company founded by some of the authors of RISC-V. SiFive produces RISC-V processors on silicon based on Rocket and provides tools for developing, testing, and evaluating their designs.

Using Chipyard (or Rocket Chip) as a foundation for system construction can pose several challenges due to its complexity in terms of the number of components and the

design language used—Chisel. However, overcoming this barrier yields numerous benefits due to Rocket Chip’s high flexibility and system customization capabilities. Its components are developed independently, have detailed development documentation, and are easily extensible. Furthermore, the Rocket Chip support community is very active, with more frequent updates and improvements compared to other frameworks.

Having established the theoretical underpinnings of the RISC-V ISA and identified the Chipyard framework as our chosen development platform, the next step is to detail the architecture of our specific implementation. The following chapter will describe the design of the single-core RISC-V processor system, its integration with the Gemmini accelerator, and the high-level compilation and co-design flow that makes such a complex system possible.

CHAPTER 3: A SINGLE-CORE RISC-V PROCESSOR SYSTEM WITH A GEMMINI ROCC ACCELERATOR

From the surveys and analyses, it is evident that Chipyard effectively meets current requirements, but it also holds the potential for expansion to support more complex systems in the future.

This chapter will present a more detailed exposition of the component structure and the process of constructing an SoC system, while also delving into the architecture of the Rocket processor core. The thesis will utilize high-level design languages Chisel/Scala to implement a single-core processor system integrated with the Gemini RoCC accelerator, focusing on leveraging the components within an SoC system, and building the hardware and software design based on the Chipyard framework.

3.1 The Chipyard Compilation and Elaboration Flow

A key advantage of the Chipyard framework is its unified compilation and elaboration flow, which decouples high-level hardware design from low-level simulation and implementation targets [52]. The process begins with a **Custom SoC Configuration**, a set of Scala classes that define the parameters for the entire system. These parameters are consumed by the **RTL Generators**—hardware libraries written in Chisel—which programmatically construct the SoC design. Rather than directly emitting Verilog, the Chisel compiler generates a circuit description in the **FIRRTL Intermediate Representation**. This crucial IR acts as a pivot point for the entire framework, allowing a single design description to be consumed by various backend "transforms" tailored for specific goals, as illustrated in Figure 3.1.

For instance, the **FireSim Transforms** can apply modifications necessary for mapping the design onto cloud FPGAs for fast, cycle-exact simulation. The **VLSI Transforms** prepare the design for an ASIC implementation flow by replacing logical memories with technology-specific macros and preparing the hierarchy for place-and-route tools.

For the purposes of this thesis, this flow is primarily used to generate synthesizable Verilog for two key targets:

1. **Software RTL Simulation:** The FIRRTL is compiled into C++ code for cycle-accurate simulation using Verilator, enabling functional verification and initial performance analysis.

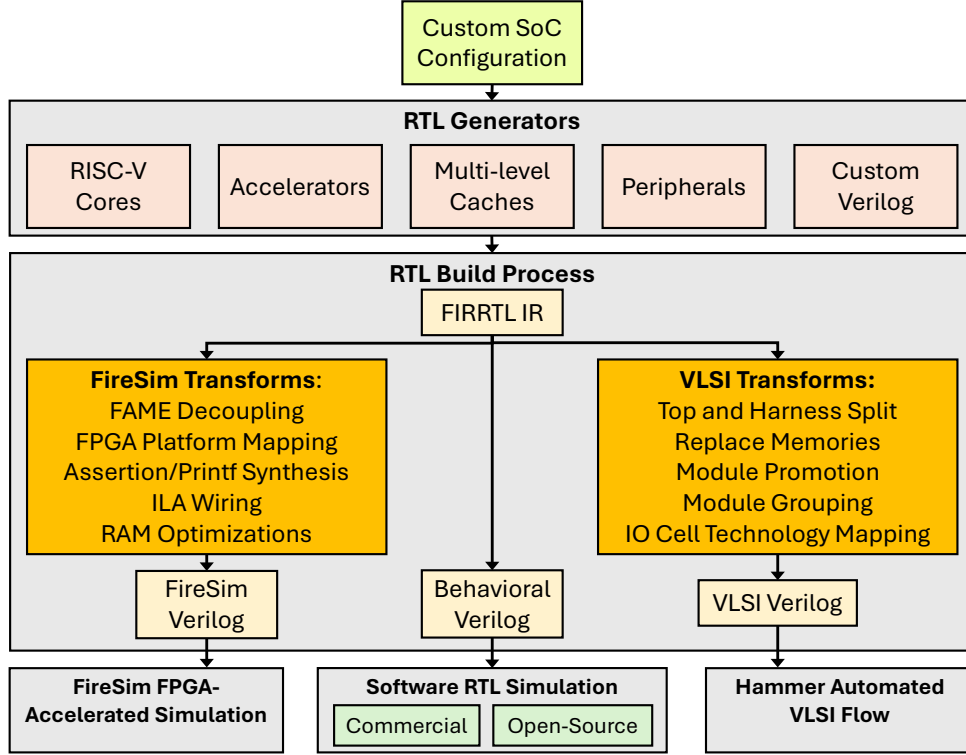


Figure 3.1: The overall development flow within Chipyard. Design begins with high-level configuration and Chisel generators, which produce FIRRTL. The FIRRTL IR acts as a pivot point, enabling different transformation paths for software simulation, FPGA-accelerated simulation, or VLSI layout. [52].

2. **FPGA Implementation:** The generated Verilog is consumed by the Xilinx Vivado toolchain to be synthesized, placed, and routed for deployment on the target VC707 FPGA board.

This structured flow demonstrates the power of a generator-based approach, allowing a single, high-level design description to target multiple platforms while maintaining design consistency.

3.2 System Design Methodology with Chisel

As previously introduced, this thesis undertakes system design based on the components and tools within Chipyard. This framework and its library components are constructed from a diverse range of elements, all based on Chisel/Scala.



Figure 3.2: Chisel as a library within Scala.

3.2.1 Chisel and Scala

Chisel is a Hardware Construction Language (HCL) developed at UC Berkeley, initially intended to enable hardware design through highly parameterized and hierarchically specific code generators [53]. Chisel is used to define these generators, which then produce designs in Verilog, suitable for industry-standard RTL design flows. Chisel is classified as a Domain-Specific Language (DSL); more precisely, it is implemented as a library package for Scala, a high-level, multi-paradigm programming language that integrates features of both Object-Oriented Programming (OOP) and functional programming on the Java Virtual Machine (JVM). Furthermore, Scala’s functional programming capabilities support higher-order functions and immutable data structures, facilitating more maintainable code, which is crucial for reproducibility and collaboration in scientific research.

Chisel’s design aims to address the lack of object-oriented programming and pre-processing structures in existing Hardware Description Languages (HDLs), supporting the construction of more flexible generators. By being based on Scala, Chisel gains the flexibility to describe hardware with high levels of abstraction and modularity. This helps hardware designers describe circuits in a way that is easy to maintain, modify, and extend, especially for large and complex designs such as microprocessors and specialized digital signal processing cores. Scala’s robust type system, diverse numerical representations and variables, immutability, and support for functional programming make it an ideal foundation for Chisel, aiding in error checking during hardware and circuit description and encouraging reusable design patterns.

In the context of hardware design based on Chisel and Scala, **sbt** (Simple Build Tool) plays a crucial role, optimizing the design management and development process. Built specifically for Scala and Java projects, **sbt** is the build tool used by the majority of Scala developers. Essentially, **sbt** acts as the compiler when working with Chisel and Scala. Using **sbt** simplifies the process of structuring projects for hardware design with Chisel in Chipyard, as well as enabling integration with many other open-source projects.

The design elaboration process using Chisel is illustrated in Figure 3.3. A de-

sign written in Chisel is first executed by a Scala program, which invokes the Chisel compiler to generate a circuit representation in FIRRTL. This intermediate representation is then processed by a series of FIRRTL transformers and optimizers, which ultimately produce synthesizable Verilog or VHDL code. This final HDL can then be consumed by standard industry tools for synthesis and physical design, leading to a final chip layout. This flow abstracts away many tedious aspects of hardware design, allowing architects to focus on the high-level structure and parameterization of their designs.

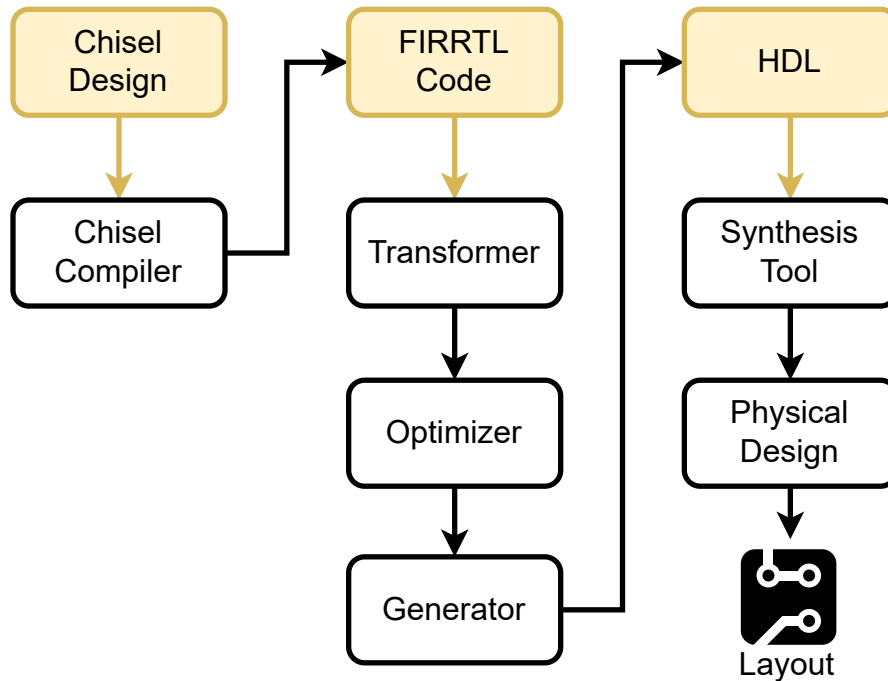


Figure 3.3: The Chisel to HDL design elaboration flow. A design is described in Chisel, compiled to FIRRTL, and finally transformed into a standard HDL for synthesis and physical design. [54]

3.2.2 Verilator Simulation Tool

In the hardware design process, evaluating the functionality of the design is critically important. With Chipyard, system designs, from simple to complex structures, can be effectively realized and tested thanks to integrated tools and RTL simulators like Verilator, which are readily available within the project framework.

Verilator is an open-source program, prominent in the field of RTL simulation due

to its unique method of converting ("Verilating") `Verilog` or `SystemVerilog` into `C++` or `SystemC`, creating a highly optimized executable program. Unlike traditional simulators, Verilator does not directly interpret `Verilog` during simulation. Instead, it generates `C++` or `SystemC` program files by parsing the hardware code, performing syntax checking, linting, and optionally inserting debug points and coverage analysis to enhance the correctness and accuracy of evaluation and verification. This "Verilated" output file can be configured to run in single-threaded or multi-threaded mode on a development server, offering an advantage for simulating complex or computationally demanding designs. After the "Verilated" code is generated, it is compiled by a standard `C++` compiler such as `GCC`, `Clang`, or `MSVC++`, and is typically combined with a user-provided wrapper file to manage the initialization of the Verilated model.

Verilator offers high simulation performance, especially for designs requiring substantial resources, due to its high speed and cycle accuracy. Verilator's compilation approach makes it up to 100 times faster than interpretive simulators like Icarus Verilog (which is also open-source), and with multi-threading, performance can increase by an additional 2-10 times, achieving a total improvement of up to 1000 times. This makes Verilator suitable for RISC-V multi-core processor designs ranging from simple to complex, and competitive even with commercial simulators such as Synopsys VCS, Cadence Incisive, or Mentor ModelSim.

3.3 The Rocket Processor Core

As introduced in the previous chapter, Rocket is a 5-stage, in-order processor core compliant with the RISC-V instruction set architecture, with RV32G and RV64G variants. Currently, the development of the Rocket core is maintained by the CHIPS Alliance. Rocket is written in the Chisel hardware design language, based on the Scala programming language, and is a key component of the Rocket Chip SoC Generator toolchain.

When implemented, each Rocket core is combined with L1 data and instruction caches. This memory operates locally for each core and, in a multi-core system, is responsible for storing temporary variables generated during the execution of the pipeline, helping to reduce congestion when multiple cores attempt to store data. Together with the Page Table Walker (PTW), a Tile within the SoC is formed. This Rocket Tile (Figure 3.4) is considered a fundamental unit in the Rocket Chip SoC system and can be replicated to create multi-core processor systems, meeting the

demands for performance and flexibility in modern SoC design.

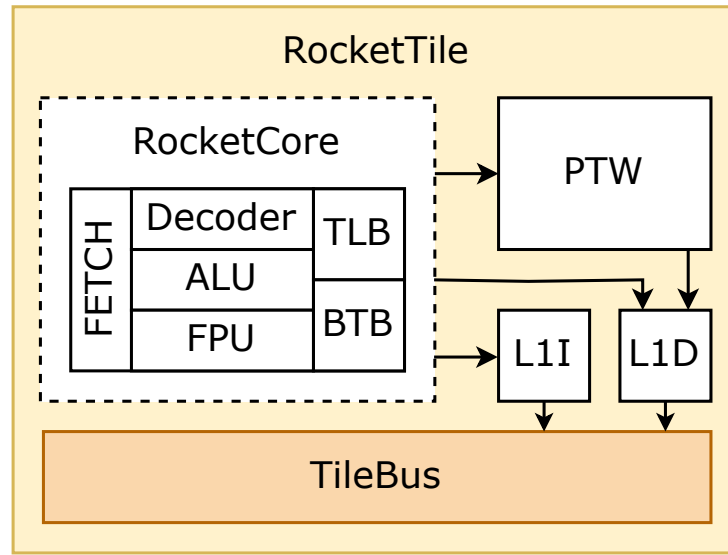


Figure 3.4: Structure of a Rocket Tile.

Due to the object-oriented nature of Chisel, the language used for its design, Rocket itself is written from object classes that can be defined and reconfigured according to implementation needs, such as `WithNBigCore()`, `WithNMedCore()`, `WithNSmallCore()`, and `WithNTinyCore()`. Additionally, instead of implementing the Rocket core, the BOOM core can also be integrated in its place, and many other peripherals can be integrated into a Tile unit.

3.4 Rocket Chip SoC Structure

This system generator includes multiple components, not just the processor core but also the on-chip system bus (SoC bus), specifically TileLink. TileLink comprises a peripheral bus, control bus, and memory bus, along with arbiters and system controllers. Additionally, the system integrates peripheral components such as DRAM, GPIO, UART, and, critically for this thesis, supports tightly-coupled accelerators like Gemmini via the RoCC interface. Figure 3.5 below depicts the generalized structure of the single-core RISC-V processor system with a Gemmini RoCC accelerator, which forms the basis for the development in this thesis.

3.4.1 Bus and Interconnect

In the design of System-on-Chips (SoCs), bus architecture plays a backbone role, connecting and coordinating the operation of all components within the system. Par-

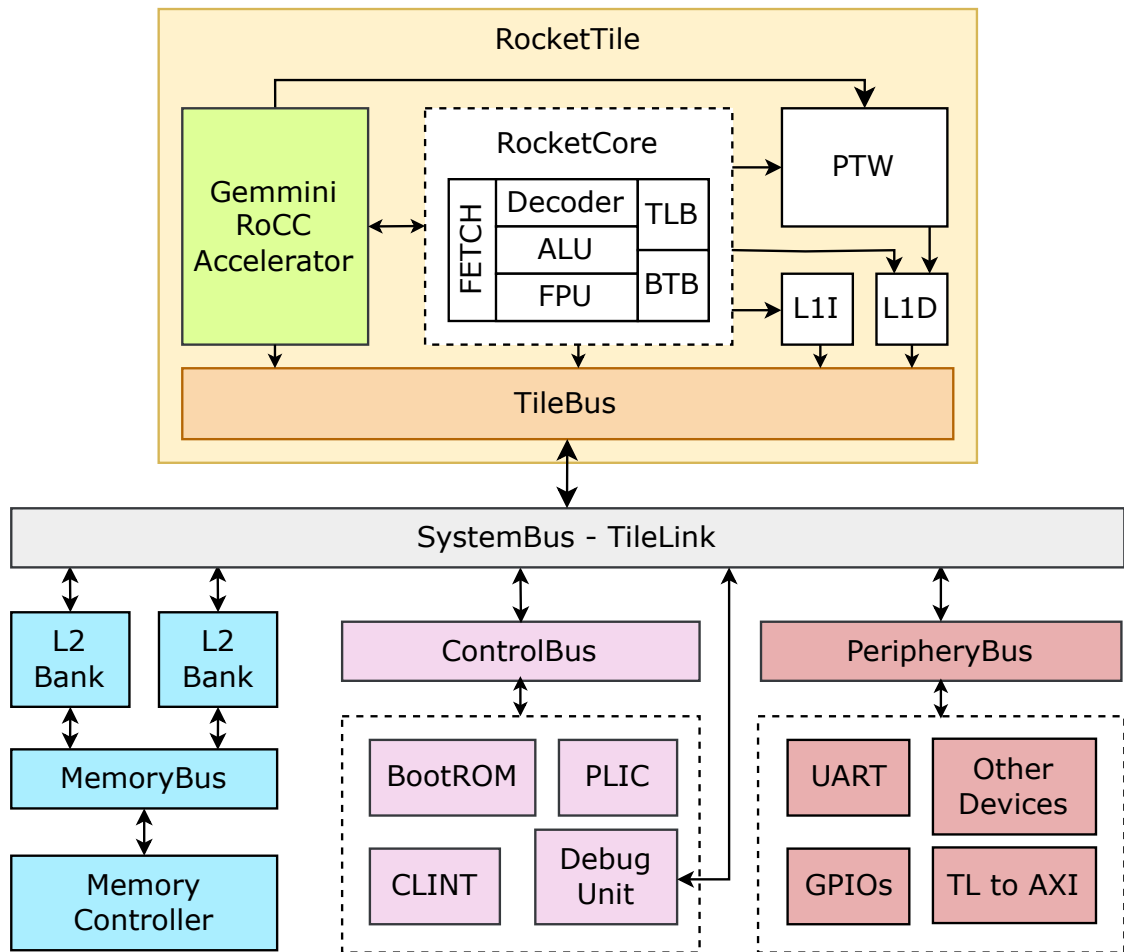


Figure 3.5: Proposed structure of a single-core RISC-V processor system with a Gemini RoCC accelerator.

ticularly for modern multi-core processor systems, development trends focus on using crossbar-type bus architectures instead of Time Shared Buses. A crossbar, specifically a fully-connected crossbar structure, is considered a form of hardware switching fabric, allowing any input port to connect directly to any output port simultaneously. The outstanding advantage of this architecture lies in its ability to provide high bandwidth and low latency, effectively meeting performance requirements in applications such as networking equipment, multi-core processors, and high-performance storage systems. The optimization in data transmission offered by this architecture is a key factor in improving the overall performance of modern SoC systems.

For Rocket Chip, the TileLink [55] system communication bus was developed concurrently with the project and the RISC-V instruction set, and is widely used in open-source system research at present.

TileLink is a highly parameterizable, open-source, shared memory interconnect

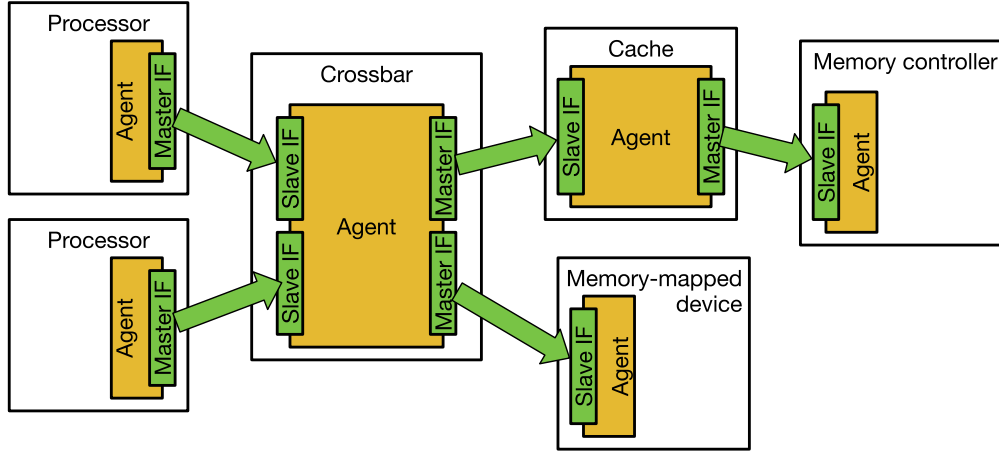


Figure 3.6: Connection model in TileLink and its interfaces. [55]

protocol based on the aforementioned crossbar structure, designed to connect diverse modules for SoCs. TileLink is implemented via a hierarchical, point-to-point network that can be easily scaled. This protocol provides memory-mapped access for multiple controllers, ensuring memory coherency while supporting efficient communication between processors, DMAs, and peripheral devices.

A TileLink bus system can support a combination of multiple communicating agents, each capable of supporting different configured subsets of the protocol. The TileLink specification includes three levels of operational configuration for agents, presented in Table 3.1.

Table 3.1: TileLink operational configurations. [55]

	TL-UL	TL-UH	TL-C
Read/Write operations	y	y	y
Multibeat messages	.	y	y
Atomic operations	.	y	y
Hint operations	.	y	y
Cache block transfers	.	.	y
Channels B+C+E	.	.	y

TileLink clearly distinguishes between cached and uncached communications. The simplest level is TileLink Uncached Lightweight (TL-UL), which only supports basic read and write operations to memory, for individual words. The next, more complex level is TileLink Uncached Heavyweight (TL-UH), which adds features such as "hints," "atomic" operations, and burst accesses but does not support coherent caching. Here, "hints" refers to a feature where a request command seeks data in-

formation external to the executing command from masters, and "atomic" refers to operations that lock access rights in a system with multiple masters, which could be microprocessors. Finally, TileLink Cached (TL-C) is the full protocol, supporting the use of coherent caches within the system.

For integrating peripheral hardware, all three bus configuration levels can be used. The communication connection diagram, at its most complete level, between a custom peripheral hardware and an agent is shown in Figure 3.7.

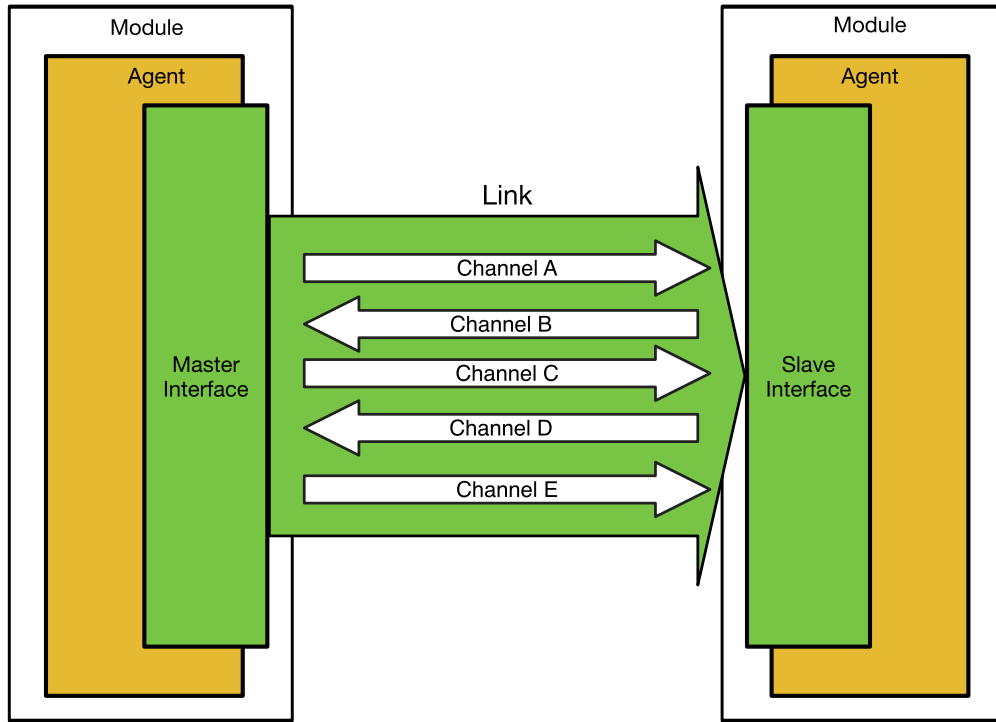


Figure 3.7: Channels in a TileLink connection between two agents. [55]

In this TileLink protocol, the defined communication channels include A, B, C, D, and E, each undertaking specific roles in the interaction process between a master (M) and a slave (S), and all have a defined direction of transmission as depicted in the figure above.

- **Channel A:** This is the initiation channel, where M sends requests to S, establishing the initial step of the communication process.
- **Channel B:** This channel is used when M requests a cache block from S, ensuring necessary data is queried.
- **Channel C:** Used for S to respond to M's request for a cache block, playing a crucial role in providing necessary data to M.

- **Channel D:** This is the general response channel, where S sends the final response to M after processing requests.
- **Channel E:** This channel ensures transaction completion by performing the final handshake for cache block transfer between the two parties.

The priority order for these channels is as follows: $A < B < C < D < E$, where channel A has the lowest priority and E has the highest. Prioritization in the TileLink bus system ensures that messages transmitted through the system do not fall into a hold-and-wait loop. In other words, the message flow through all channels between agents is maintained as a directed acyclic graph, enabling the TileLink protocol to operate without encountering deadlocks, ensuring continuous and uninterrupted data flow within the system.

The choice of TileLink bus configuration is relevant for connecting various memory-mapped peripherals within the system. For general-purpose MMIO peripherals, which might include cryptographic algorithm accelerators designed in Verilog HDL (if they were to be integrated as MMIO devices), the TL-UL configuration, comprising only channels A and D between M and S (as shown in Figure 3.8), is often sufficient. However, for the primary accelerator in this thesis, the Gemmini core, integration is achieved through the RoCC interface, which provides a more tightly-coupled path to the processor core, rather than as a TileLink-attached MMIO peripheral. Details on the RoCC interface are discussed further in Section 3.4.4. For other standard MMIO peripherals, a wrapper from the Rocket Chip library, using Chisel/Scala, can be inherited and customized to connect their signals to the appropriate system bus, typically the peripheral bus.

At the system configuration level, a wrapper from the Rocket Chip library, using Chisel/Scala, can be inherited and customized to convert the signals of the accelerator core to the system bus, in this case, the peripheral bus. Details for this interface will be further elaborated in subsequent sections of this thesis.

3.4.2 Memory and Cache

Each Tile in the system is integrated with a processor core and an L1 cache that is flexibly configurable. By default design with Chisel, the L1 cache has a size of 16 KiB, organized multi-dimensionally with sets and ways. The L1 cache uses a set-associative cache structure, dividing the memory region into sets, each set containing a fixed number of data blocks (ways). When a memory address access request occurs,

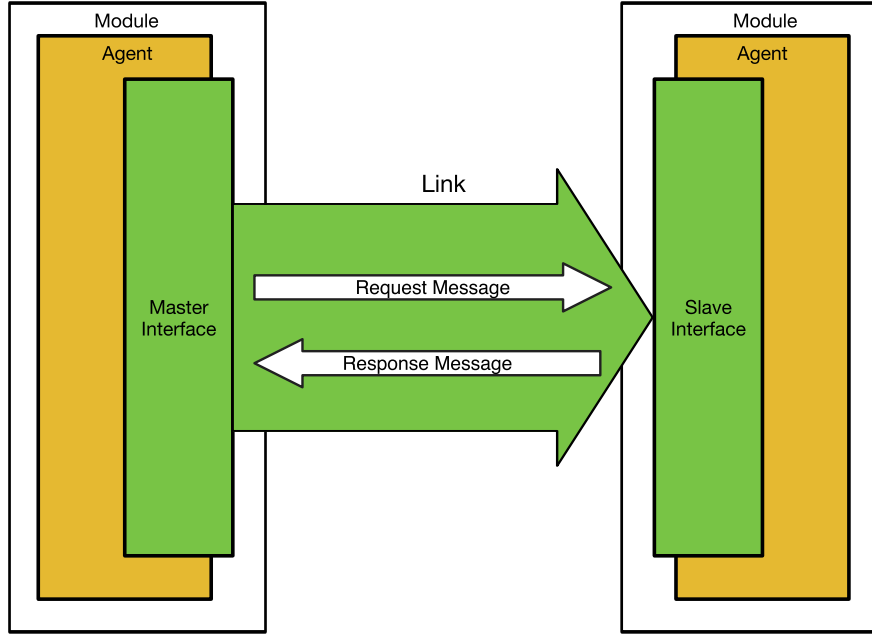


Figure 3.8: TileLink TL-UL connection structure. [55]

the index derived from the address maps it to a specific set. The cache controller then searches for the data block within the ways of that set. This design supports both instruction and data storage, optimizing access performance and increasing the hit rate compared to caches with lower associativity.

In addition to the L1 cache, the system also integrates a shared L2 cache, connected to all cores via the TileLink bus system. The L2 cache acts as an intermediate storage repository, allowing processor cores and other components in the system to perform read/write data requests. This is a coordination mechanism that helps reduce pressure on the main memory and improves overall processing performance. The L2 cache controller also ensures fairness in resource allocation, using a queuing system to manage access requests.

At a higher level, the system also supports the integration of external memory, such as DRAM or SRAM, through open-source controllers or specialized IPs from FPGA implementation toolkits. Additionally, an interface with DRAMSim is supported for integration when performing full-system simulation. This provides flexibility and scalability, meeting various requirements in practical applications.

3.4.3 Core-Complex

As previously mentioned, the Chipyard framework and library consist of many component definition classes that can form a Rocket Chip system. This allows for the

formation of a complex system with diverse processor cores and peripherals, specifically heterogeneous architecture systems comprising multiple different processors like Rocket and BOOM coexisting on the system.

In these multi-core processor structures, each individual core will be uniquely identified with a different hart ID, ordered according to the system configuration. An example can be referenced below with a configuration comprising one Rocket core and one BOOM core. The corresponding block diagram configuration when generating the system will be as shown in Figure 3.9 below.

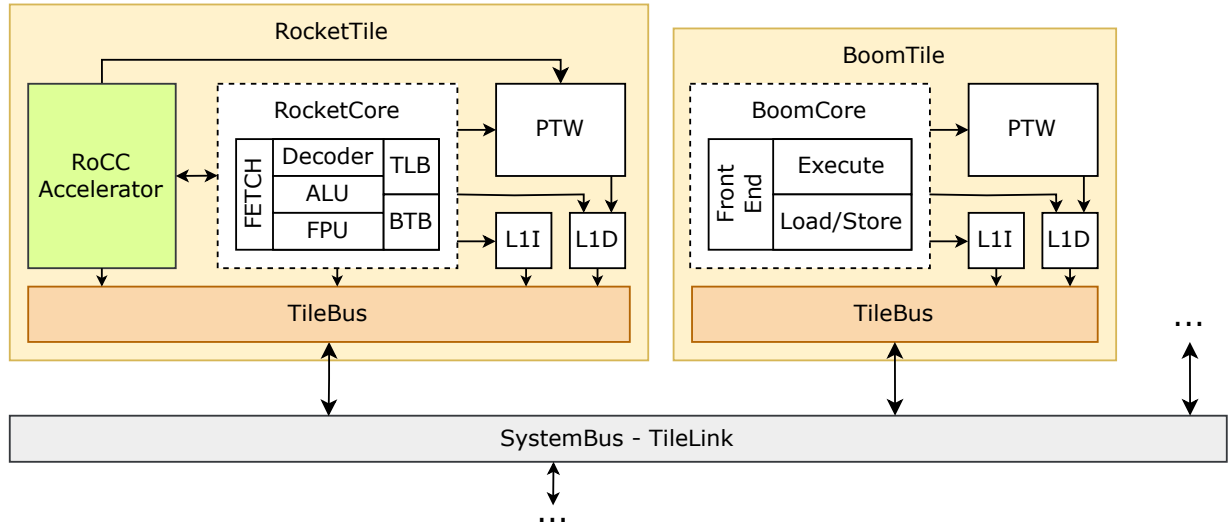


Figure 3.9: Implementation structure of a heterogeneous multi-core processor system.

A heterogeneous SoC system is an implementation approach also considered in this thesis to combine different processor cores to optimize performance and energy efficiency. At the application scale, lightweight cores can handle simple tasks with low power consumption, while high-performance cores support hyper-threading and out-of-order execution to process complex tasks. This design ensures flexibility, resource optimization, and superior performance for diverse applications.

3.4.4 Peripheral Components

In the base system configuration class of Rocket Chip, many peripheral components are initialized and integrated by default to support both simulation and deployment on FPGA hardware for the SoC system. This configuration includes basic I/O devices such as PWM, interrupt controllers, JTAG, ROM, external memory, and standard peripheral interfaces, ensuring that basic communication, storage, and control requirements are met. In particular, common peripheral devices like UART,

SPI Flash, and GPIO are implemented to support serial communication, storage management, and flexible control within the system.

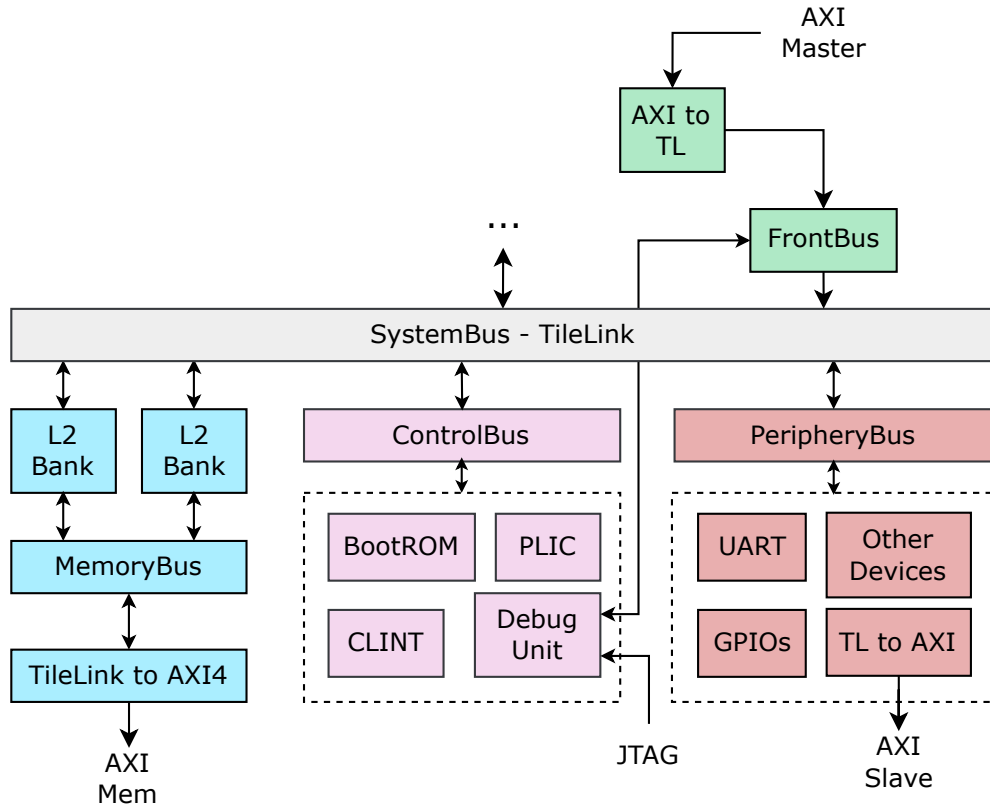


Figure 3.10: Peripherals in the Rocket Chip base system.

All these components are developed based on the open-source Rocket Chip library, with hardware designs like Rocket and BOOM realized on the Chisel platform. When external memory is needed in design and simulation, the Advanced eXtensible Interface 4 (AXI4) interface can be integrated as an intermediate layer, allowing effective connection with DRAM or SPI Flash models. Concurrently, serial protocols like JTAG are integrated to ensure accurate debugging capabilities and efficient serial communication, configured to connect with the TileLink bus. The Rocket Chip system also supports flexible configuration through special control ports, including custom boot pins and chip identification ports. Furthermore, the Rocket Chip system provides the capability to directly map bus interfaces such as AXI4 Memory and Memory-Mapped Input/Output (MMIO), allowing for expanded compatibility and optimized performance in complex SoC applications.

To expand the capability of integrating components and peripheral cores to form highly parameterized custom systems suitable for specific applications, the Rocket Chip architecture provides two flexible methods for peripheral integration. Specif-

ically, the system can integrate a peripheral core as a Rocket Custom Coprocessor (RoCC), located within each tile, or integrate at the system communication level via the MMIO mechanism. These methods facilitate in-depth customization, meeting various requirements for each specific application.

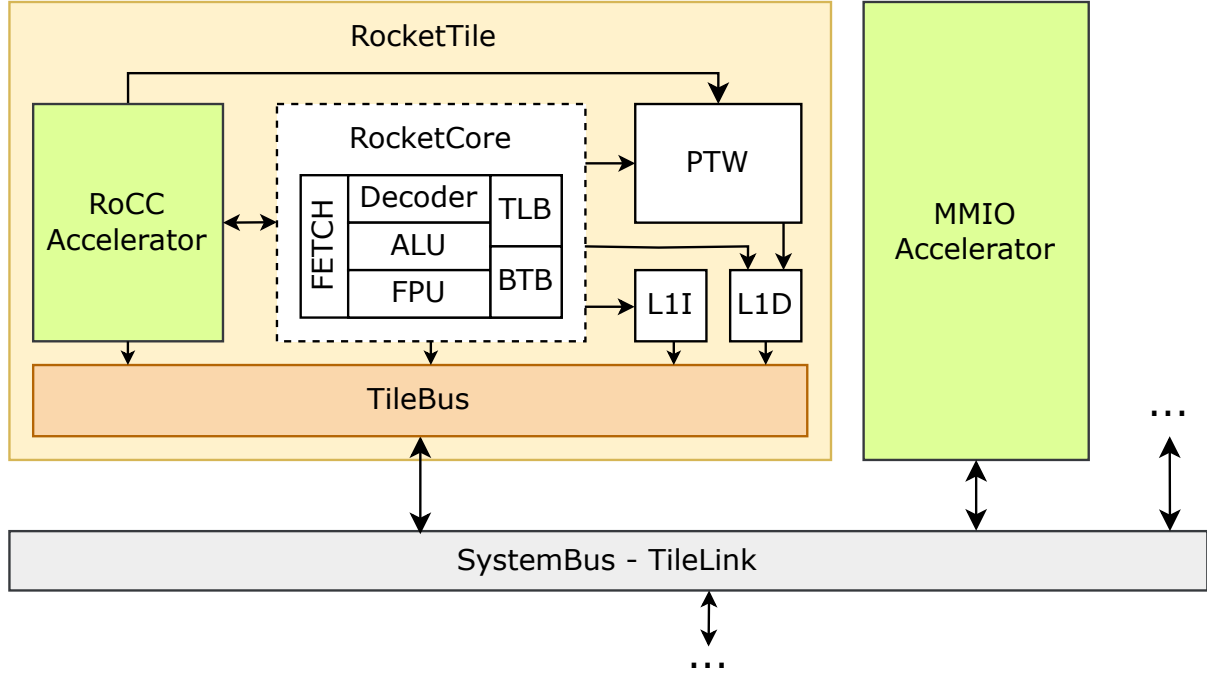


Figure 3.11: RoCC co-processor core and MMIO peripheral core in Rocket Chip.

MMIO Peripherals are custom peripherals attached directly to the TileLink bus, while Tightly-Coupled RoCC Accelerators are custom peripherals attached directly to the arbiter and processor within each Tile, in parallel with the processor core. With the TileLink-Attached MMIO method, the processor communicates with MMIO devices through registers mapped into memory at addresses on the system. Through these registers, control mechanisms or data transfer can be customized to suit the application of the peripheral core. Conversely, in the RoCC Accelerator method, communication is performed via a custom protocol and non-standard ISA instructions, which are configured and defined separately within the encoding space of the RISC-V ISA. For Rocket and BOOM, each core can control up to four accelerators through custom opcodes, sharing resources with the microprocessor. Instructions executed with RoCC cores have the format `customX rd, rs1, rs2, funct`. Here, X is a number from 0-3, determining the opcode that controls the routing of the instruction to a specific accelerator. The fields `rd`, `rs1`, `rs2` are the register numbers for the destination register and two source registers, and the `funct` field is a 7-bit integer

that the accelerator can use to differentiate various instructions.

Peripherals connected as RoCCs have a distinct advantage due to their high customizability and direct connection and control via custom ISA instructions. This allows for specialized performance optimization, particularly useful for cores requiring large, complex computational workloads or applications needing extremely fast response times. Communication via a custom protocol and leveraging the RISC-V ISA helps RoCC utilize CPU resources efficiently, while also offering flexibility in designing specialized accelerator processors. The Gemmini accelerator, central to this thesis, is a prime example of a RoCC-based component, designed for high-throughput matrix multiplication and tightly integrated with the Rocket core.

While RoCC implementation can require a custom software toolchain for its specialized instructions, the performance benefits for demanding applications like those targeted by Gemmini often justify this investment. For this thesis, the integration of the Gemmini accelerator via the RoCC interface is deemed the most appropriate approach to achieve the desired performance for deep learning workloads.

Conversely, the MMIO TileLink-Attached method is widely used for integrating general-purpose memory-mapped peripheral devices. In this model, peripheral devices provide a set of registers (e.g., Chisel Registers) to communicate with microprocessors or other masters on the system bus. This method is suitable for peripherals where the tight coupling and custom instruction set of RoCC are not necessary.

To minimize design complexity for MMIO peripherals, the Rocket Chip architecture provides the `regmap` interface, which allows for the automatic generation of much of the necessary linking logic. Designers can use `TLRegisterRouter` or create a `LazyModule` and initialize `TLRegisterNode` to integrate such devices. Since TileLink is the official protocol used in SoCs based on Rocket Chip and Chippyard, designing MMIO peripheral devices (other than the primary RoCC accelerator) would typically focus on integration with this protocol.

3.4.5 The RoCC Interface: A Hardware Co-Design Methodology

The primary conduit for communication between the Rocket Core and the Gemmini accelerator is the Rocket Custom Coprocessor (RoCC) interface. This interface, however, is not merely an external bus; it is a profound hardware **co-design methodology** that fundamentally alters the CPU’s own pipeline. When the `WithGemmini` trait is active in the SoC configuration, the Chippyard generator does more than just instantiate the accelerator; it injects specific logic directly into the Rocket Core’s

decoder, hazard unit, and writeback stage.

The architectural philosophy of the RoCC interface is designed for high-throughput, decoupled accelerators. As was found in prior work by Pala, the interface’s high-latency profile for CPU-to-accelerator register reads makes it best suited for offloading large, autonomous tasks rather than for frequent, small operations that require constant communication between CPU and accelerator [2]. Gemmini, which operates on large data streams for DNN computation, is a canonical example of this intended use case.

This section will trace the lifecycle of a RoCC instruction to demonstrate how these code-level modifications create a tightly-coupled, low-latency communication channel that is essential for high-performance acceleration.

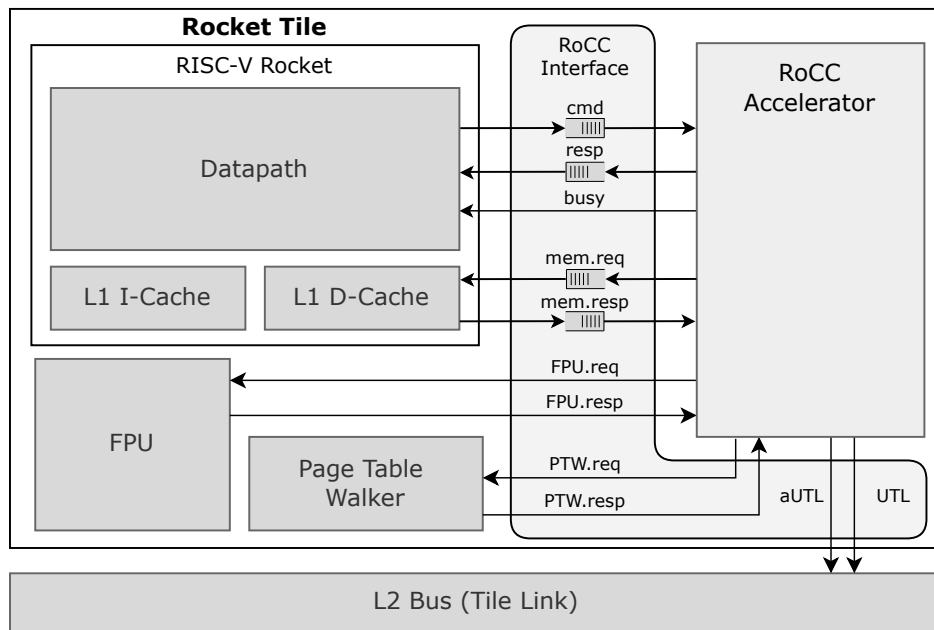


Figure 3.12: Rocket Tile RoCC Interface Architecture with Accelerator Integration. The RoCC interface provides direct communication pathways between the Rocket Core’s datapath and the attached accelerator, including command dispatch, memory requests, and response handling channels.

3.4.5.1 The RoCC Instruction Format

The entire co-design process is initiated by a specific instruction format recognized by the CPU. The RISC-V ISA reserves four major opcodes for custom extensions, and the Gemmini accelerator is assigned the `custom3` opcode space. Any instruction using this opcode follows the R-type format, modified for RoCC communication as shown in Figure 3.13.

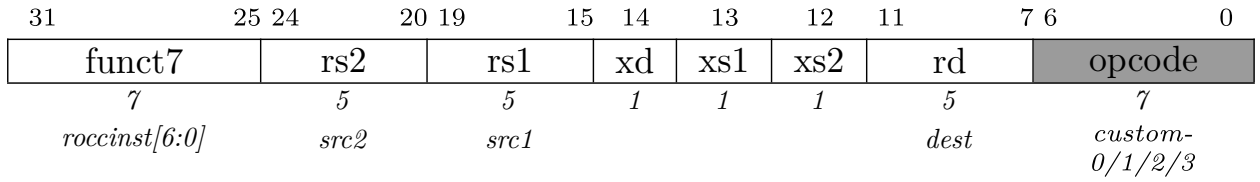


Figure 3.13: The RoCC Instruction Format. The `opcode` field is set to `custom3` (0b1111011). The `funct7` field specifies the Gemmini-specific operation, while the `xd`, `xs1`, and `xs2` bits act as control signals for the hardware interface.

The fields in this instruction format are critical for orchestrating the hardware interaction:

- **opcode:** Identifies the instruction as a RoCC command, directing it away from the standard execution units.
- **funct7:** This 7-bit field is passed directly to Gemmini, which uses it to select the specific operation to perform (e.g., `LOOP_CONV_WS`, `CONFIG_LOAD`).
- **xd, xs1, xs2:** These single-bit fields are essential hardware control signals. They function as "valid" bits for the register operands. `xs1` and `xs2` signal that valid data is present on the `rs1` and `rs2` buses, while `xd` signals that the CPU expects a result for the destination register `rd` and must stall until a response arrives.

3.4.5.2 Modifying the CPU's Instruction Decoder

For the co-design to function, the CPU must first be taught to recognize the new instruction format. The Chipyard generator achieves this by programmatically modifying the main decoder table in the Rocket Core, as shown in Listing 3.1.

```
1 // From rocket-chip/src/main/scala/rocket/IDecode.scala
2 CUSTOM3 -> List(Y,N,Y,N,N,N,N,N,A2_ZERO,A1_RS1,IMM_X,DW_XPR,...) ,
```

Listing 3.1: Modification of the Rocket Core's instruction decode table.

This modification is the first and most critical step. It instructs the decoder that the `custom3` opcode is a valid instruction that uses the register file (as specified by `A1_RS1`) but not the standard ALU. This prevents the CPU from triggering an illegal instruction trap and instead flags the instruction internally as a RoCC command, allowing it to propagate down the pipeline for specialized handling, as illustrated in Figure 3.14.

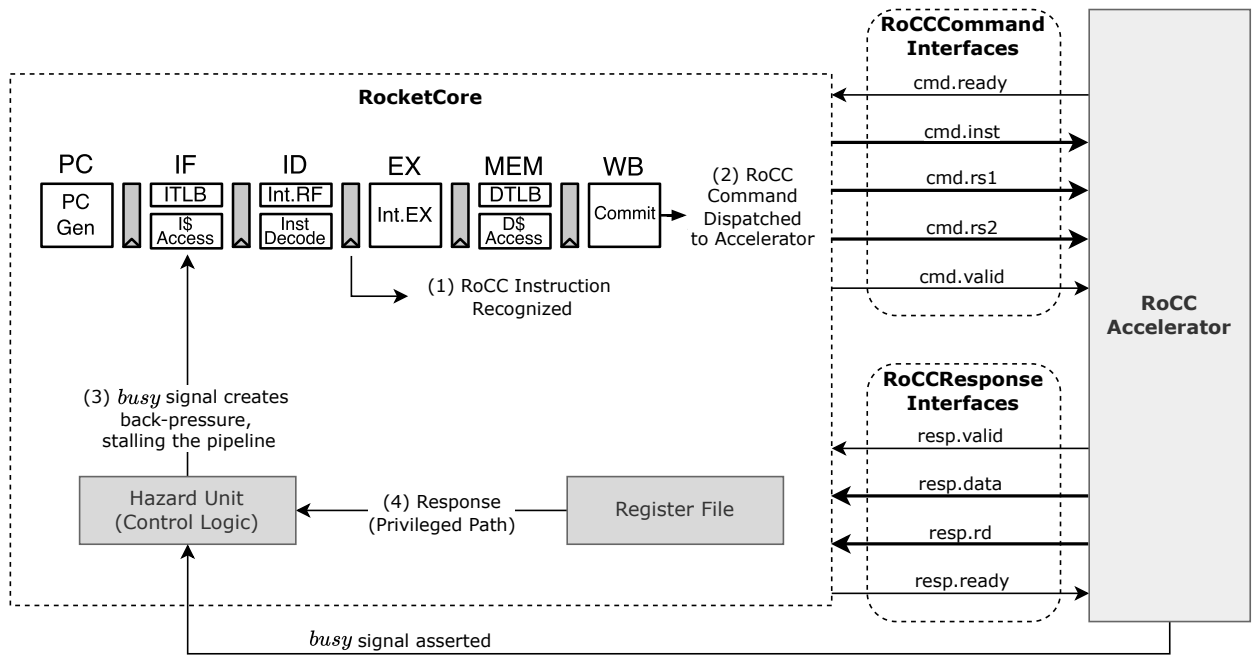


Figure 3.14: Diagram illustrating the RoCC co-design and control flow. (1) A RoCC instruction is recognized at the Decode stage. (2) At the Writeback stage, the RoCCCommand is dispatched to Gemini over a **ready/valid** channel. (3) A **busy** signal provides back-pressure to the CPU’s Hazard Unit, stalling the pipeline. (4) The **resp** path provides a privileged write-back directly to the Register File.

3.4.5.3 Pipeline Integration and Hazard Management

The tight coupling between the CPU and Gemini necessitates new hardware interlocks to manage pipeline hazards. These interlocks are injected directly into the CPU’s control logic. If Gemini is busy and cannot accept a new command, it exerts **back-pressure** on the CPU. The hardware logic shown in Listing 3.2 generates a stall signal that freezes the CPU pipeline until the accelerator is ready.

```

1 // Source: rocket-chip/src/main/scala/rocket/RocketCore.scala
2
3 // Stall decode if RoCC is not ready for a new command
4 val rocc_blocked = Reg(Bool())
5 rocc_blocked := !wb_xcpt && !io.rocc.cmd.ready &&
6               (io.rocc.cmd.valid || rocc_blocked)
7
8 // Replay the instruction in the writeback stage
9 val replay_wb_rocc = wb_reg_valid && wb_ctrl.rocc &&
10                    !io.rocc.cmd.ready

```

Listing 3.2: RoCC-specific stall logic in the Rocket Core.

This logic provides definitive proof of a hardware co-design. The `replay_wb_rocc` signal physically stalls the CPU pipeline if a command is in the writeback stage and Gemini is not ready (`!io.rocc.cmd.ready`). Furthermore, the `rocc_blocked` register freezes the pipeline at the decode stage if a new RoCC instruction arrives while the accelerator is already busy. The CPU’s progression is now physically dependent on the state of the accelerator.

3.4.5.4 The Privileged Response Path

Finally, the RoCC interface provides a privileged, low-latency path for the accelerator to write results back into the CPU’s register file. This is arbitrated with other internal units, as shown in Listing 3.3.

```

1 // Source: rocket-chip/src/main/scala/rocket/RocketCore.scala
2 when (io.rocc.resp.fire) {
3   div.io.resp.ready := false.B // Disable div response
4   ll_wdata := io.rocc.resp.bits.data
5   ll_waddr := io.rocc.resp.bits.rd
6   ll_wen := true.B
7 }

```

Listing 3.3: RoCC response handling in the Rocket Core.

When the Gemini accelerator has a result, the hardware routes the response data (`io.rocc.resp.bits.data`) directly to the register file’s write port (`ll_wdata`) and asserts the write enable signal (`ll_wen`). This is a privileged write path that completely bypasses the memory system, a feature impossible to achieve with a standard peripheral.

3.4.5.5 The RoCC Memory Interface

Beyond the command interface, the RoCC protocol provides a privileged, high-bandwidth connection directly to the CPU’s L1 Data Cache, shown in Figure 3.15. This is a key advantage over a standard memory-mapped peripheral, which would suffer from the higher latency of the L2 cache and system bus.

Critically, this interface is asymmetrical. The request channel from Gemini to the L1 D-Cache (`mem_req`) is a full `Decoupled` interface, allowing Gemini to wait if the cache is busy. However, the response channel from the L1 D-Cache to Gemini (`mem_resp`) has only a `valid` signal. This is a deliberate design choice: the

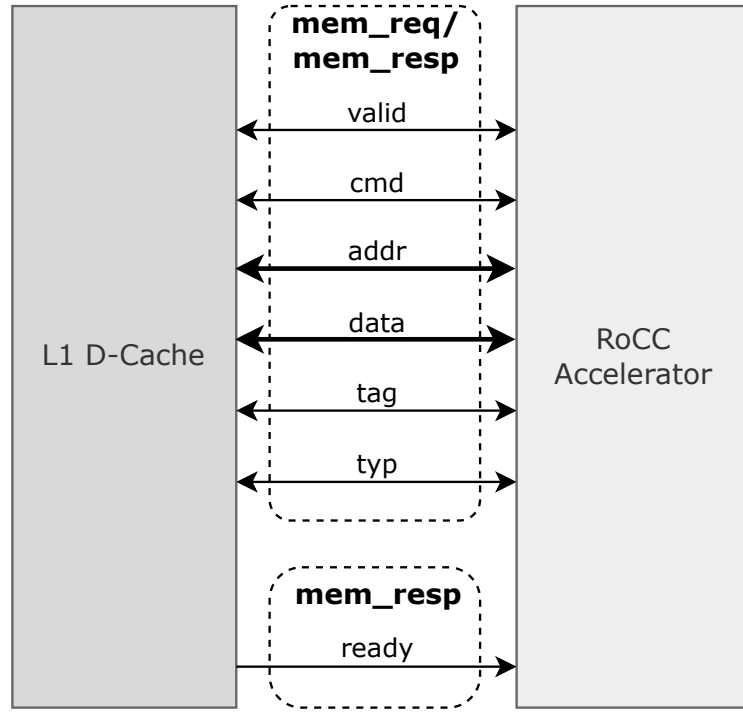


Figure 3.15: The asymmetrical RoCC memory interface. The request channel (**mem_req**) uses a full **ready/valid** handshake, while the response channel (**mem_resp**) is a simpler **valid**-only interface.

accelerator cannot apply back-pressure to the L1 D-Cache and must accept memory responses immediately to prevent stalling the main CPU pipeline. This nuance highlights the deeply integrated nature of the memory co-design. As confirmed by the timing analysis in Figures 3.16 and 3.17, this interface exhibits a two-cycle latency and a maximum throughput of 50%, performance characteristics that must be considered when designing and scheduling operations for the accelerator

3.4.5.6 Conclusion

In summary, the RoCC integration is a profound act of hardware co-design, proven by the evidence from the hardware generator. The process modifies the CPU’s instruction decoder, injects new stall conditions into the core hazard unit, provides a privileged response path to the register file, and grants direct, arbitrated access to the L1 cache. These code-level modifications create the tightly-coupled, high-performance link necessary to realize the performance uplift for DNN workloads, which will be quantified in Chapter 8.

With the hardware co-design methodology now established as the mechanism for integrating our accelerator, the next chapter will explore the architectural foundations

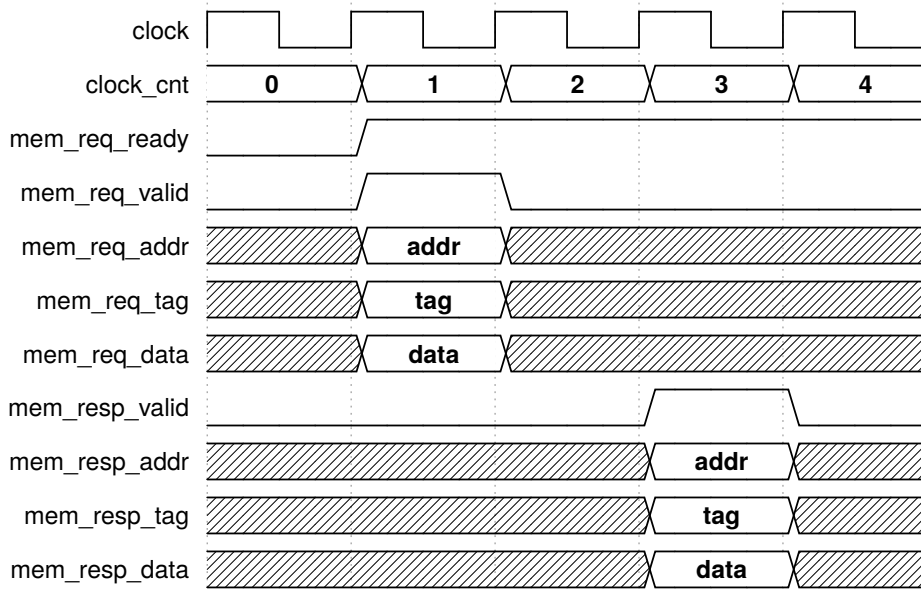


Figure 3.16: Timing diagram for a single RoCC memory request. The diagram demonstrates the two-cycle latency of the memory interface, as the `mem_resp_valid` signal is asserted two clock cycles after the `mem_req_valid` signal. Note the absence of a `mem_resp_ready` signal, confirming the asymmetrical nature of the interface. [2]

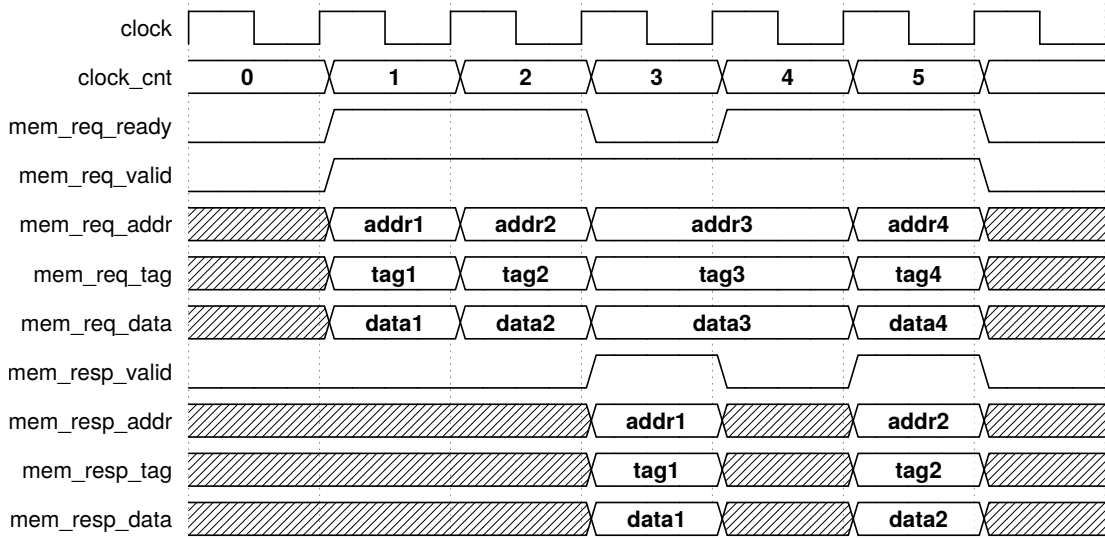


Figure 3.17: Timing diagram for consecutive RoCC memory requests. This illustrates the 50% maximum throughput of the interface. The L1 D-Cache can accept two back-to-back requests but then requires a one-cycle pause before accepting the next two, a critical performance characteristic for data-intensive accelerators. [2]

of Deep Neural Network (DNN) acceleration to understand why such specialized hardware like Gemmini is required in the first place. The complete, detailed definitions of the RoCC hardware interface bundles are provided in Appendix B for reference.

CHAPTER 4: ARCHITECTURAL FOUNDATIONS OF DNN ACCELERATION

The efficient execution of Deep Neural Networks (DNNs) presents one of the most significant challenges and opportunities in modern computer architecture. While today’s general-purpose CPUs are highly optimized for a wide range of tasks, their fundamental design principles are often mismatched with the unique computational and data movement patterns of large-scale neural networks. This chapter will establish the foundational concepts that motivate the need for specialized hardware. We will begin by exploring the core problem—the immense cost of data movement—and then introduce the classic architectural paradigm designed to solve it: the systolic array. Finally, we will examine how this timeless principle is embodied in state-of-the-art industrial accelerators, providing the necessary context to understand the design of the Gemmini accelerator, which is the central subject of this thesis.

4.1 The Data-Compute Divide: The Memory Wall

The performance of any computing system is ultimately limited by two factors: the speed of its computations and the speed at which it can supply data to its computational units. For decades, Moore’s Law drove exponential growth in the number of transistors on a chip, leading to dramatic increases in processing power. However, the speed of off-chip memory systems, such as DRAM, has improved at a much slower pace. This growing disparity between processor speed and memory speed is famously known as the **Memory Wall** [56].

For data-intensive workloads like DNNs, this is the primary performance bottleneck. The problem can be understood with a simple analogy: imagine a master chef (the processor) who can perform culinary tasks with superhuman speed. However, their ingredients (the data) are stored in a large warehouse across the street (DRAM). Even though the chef is exceptionally fast, their overall productivity is dominated by the time spent walking to and from the warehouse to fetch each ingredient.

This bottleneck is not just about latency; it is fundamentally about energy. As quantified by Horowitz [57], the energy required to perform a complex 32-bit floating-point computation is dwarfed by the energy needed to fetch its operands from off-chip DRAM (Figure 4.1). This immense energy cost of data movement means that for DNNs, which perform billions of operations, any viable high-performance hardware

solution must be designed with a primary objective: to minimize data movement.

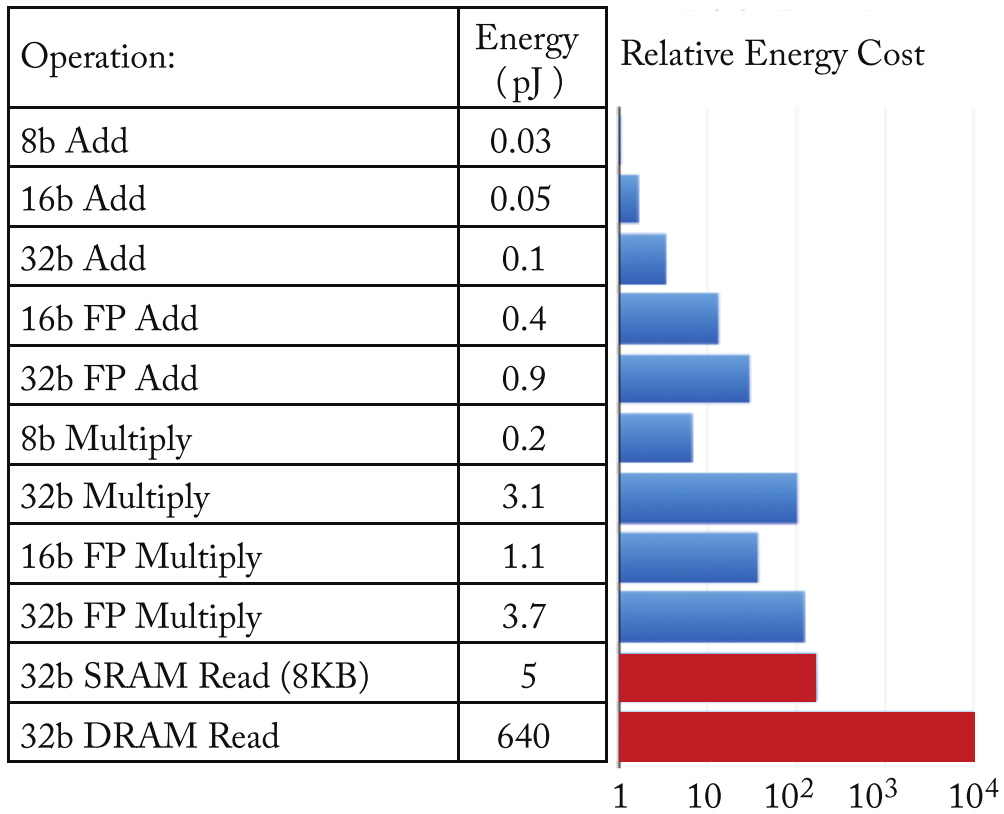


Figure 4.1: The relative energy cost of various arithmetic and memory operations. A 32-bit DRAM read is orders of magnitude more costly than a 32-bit floating-point addition. This disparity is the primary motivation for specialized accelerator architectures. [58, 57].

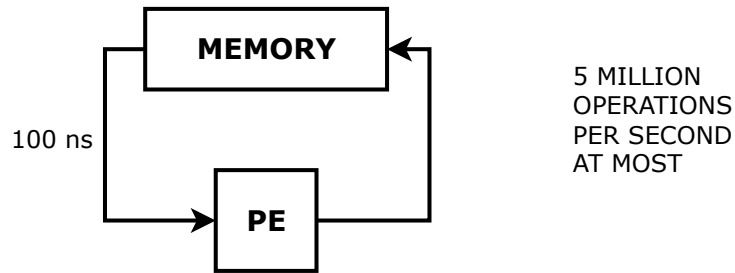
4.2 A Classic Solution: The Systolic Principle

In his seminal 1982 paper, H.T. Kung proposed a novel architectural paradigm designed specifically to address the memory wall: the **systolic array** [59]. The architecture is named by analogy to the human circulatory system. Just as the heart rhythmically pumps blood to the body's cells, in a systolic system, memory "pumps" data through a regular grid of simple Processing Elements (PEs).

The core principle is to **maximize the number of computations performed for each data item fetched from memory**. Figure 4.2 provides a powerful conceptual comparison. In a conventional architecture, each operand must be fetched from memory for a single operation in the PE, making the system bottlenecked by memory bandwidth. In contrast, the systolic approach creates a pipeline of PEs. A single data element is fetched and then passed from one PE to the next, participating

in a new computation at each step. This data reuse allows the systolic architecture to achieve significantly higher throughput with the exact same memory bandwidth.

INSTEAD OF:



WE HAVE:

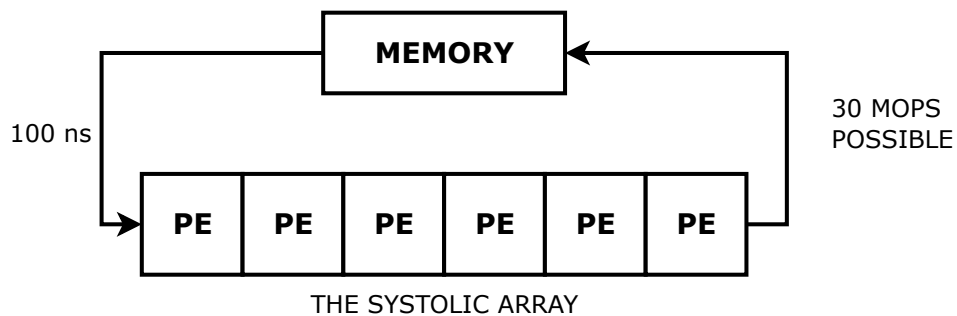


Figure 4.2: A conceptual comparison of a conventional von Neumann architecture and a systolic architecture. The conventional approach (top) is bottlenecked by the memory bus. The systolic approach (bottom) uses an array of PEs to achieve high throughput with the same memory bandwidth by reusing data internally. Adapted from the presentation of Kung’s work [59].

While Kung’s original comparison work laid the theoretical foundation and illustrated the benefit, the mechanism for achieving this data reuse is elegant in its own right. As shown in Figure 4.3, modern systolic arrays implement this principle for matrix multiplication by feeding input operands from two matrices (A and B) into the array from orthogonal sides. The values from matrix A flow horizontally, while the values from matrix B flow vertically. As these data wavefronts intersect at each clock cycle, every PE in the grid performs a Multiply-Accumulate (MAC) operation in parallel. The result is a massively parallel computation structure that amortizes the high energy cost of a single memory access over dozens or even hundreds of useful arithmetic operations, providing a direct architectural solution to the data movement bottleneck.

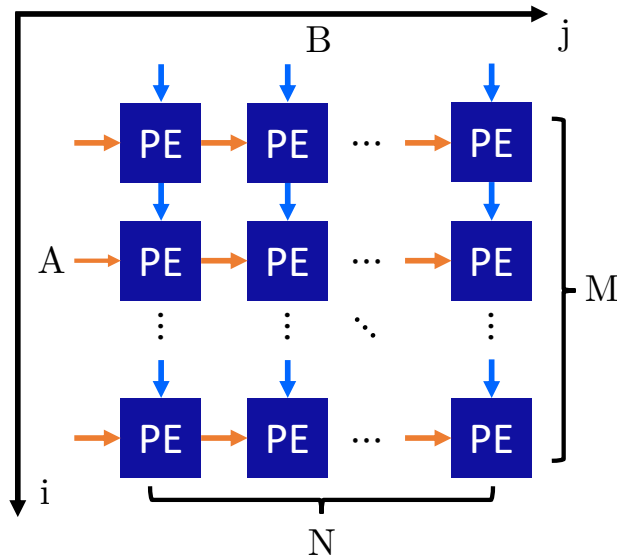


Figure 4.3: The fundamental principle of a systolic array for matrix multiplication. Input matrix A flows horizontally, while input matrix B flows vertically. This rhythmic dataflow allows for massive parallelism and data reuse.

4.3 Case Studies in Modern Systolic Design

While the systolic principle is four decades old, it has become the dominant paradigm for modern high-performance DNN accelerators. We will examine two landmark industrial designs that showcase this principle in practice.

4.3.1 Google’s TPUv1: A Datacenter-Scale Design

Google’s Tensor Processing Unit (TPU) was one of the first large-scale custom ASICs designed specifically for accelerating DNN inference in datacenters [60]. As shown in Figure 4.4, its architecture is a direct and powerful embodiment of the systolic principle. The heart of the TPU is a massive 256×256 Matrix Multiply Unit, a systolic array of 65,536 8-bit multiply-accumulate (MAC) units.

The TPUv1’s design philosophy is a masterclass in optimizing for a specific domain: high-throughput datacenter inference. Several key decisions reflect this focus. First, the use of 8-bit integer arithmetic was a critical trade-off, sacrificing the precision of floating-point numbers for immense gains in area and energy efficiency. This was based on the insight that for inference, the resilience of neural networks to quantization allows for acceptable accuracy with lower-precision data. Second, the enormous on-chip Unified Buffer (a 24 MiB software-managed scratchpad) is essential to "keep the beast fed"—it buffers entire layers of weights and activations on-chip to ensure the

massive systolic array is never starved for data. This design choice implicitly favors large batch sizes, where the cost of loading the model can be amortized over many simultaneous inference requests, prioritizing throughput over single-request latency.

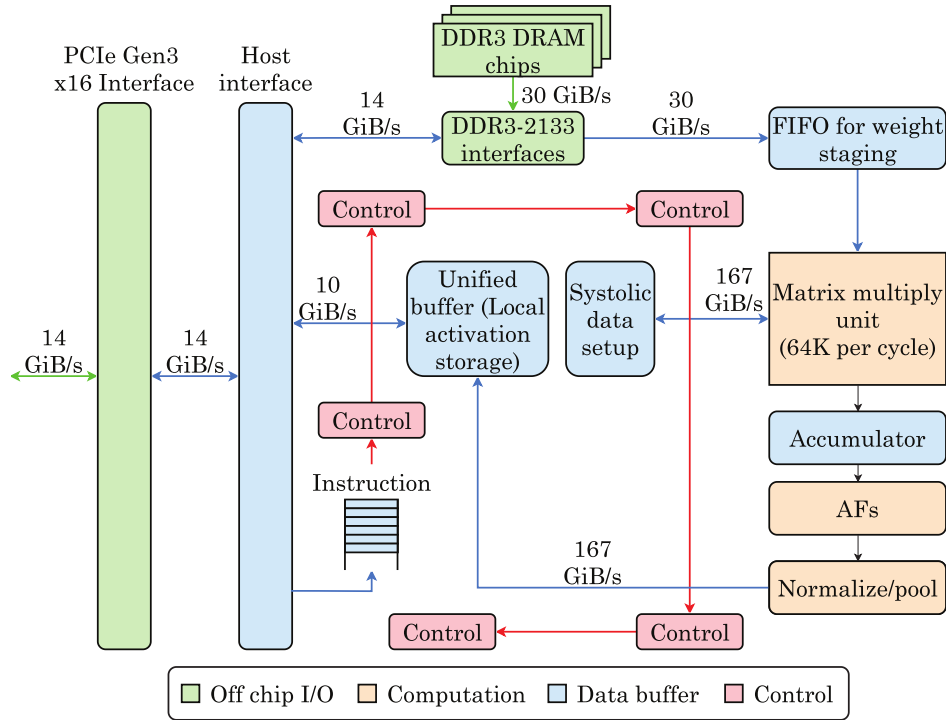


Figure 4.4: A high-level block diagram of the Google TPUv1. The design is dominated by the systolic Matrix Multiply Unit and its large on-chip Unified Buffer. [60, 61].

4.3.2 NVIDIA NVDLA: An Open-Source Accelerator IP

In contrast to the monolithic, datacenter-focused TPU, the NVIDIA Deep Learning Accelerator (NVDLA) is an open-source, configurable IP core designed for integration into a wide range of System-on-Chips (SoCs), particularly for embedded systems [62, 63]. The NVDLA’s design philosophy prioritizes **modularity and integration flexibility** over raw scale.

As shown in Figure 4.5, the architecture is partitioned into distinct, specialized hardware units for convolution, post-processing, and memory management. This modularity is a key feature, allowing an SoC designer to integrate only the necessary components. A significant design choice differentiating the NVDLA from the TPU is its reliance on industry-standard bus protocols like AXI. While this may introduce more protocol overhead than a custom interface, it dramatically simplifies the process of integrating the NVDLA into a third-party SoC from a different vendor. Furthermore, its configurability (e.g., in the number of MAC units from the `nv_small` to

`nv_large` configurations) makes it a versatile solution that can be tailored to the specific power, performance, and area (PPA) constraints of an embedded device.

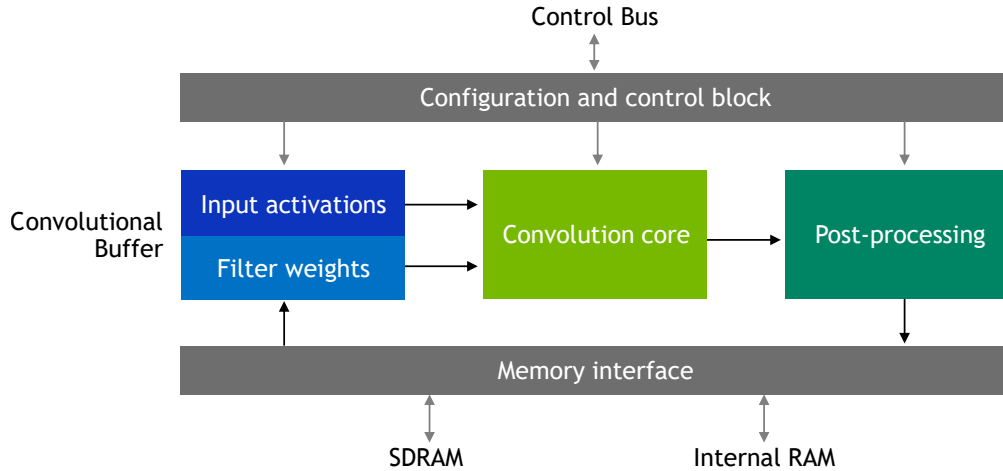


Figure 4.5: The high-level architecture of the NVDLA. The design is partitioned into distinct functional units connected via standard bus interfaces. [62].

4.4 A Formal Framework: Dataflows

The design choices made in architectures like the TPU and NVDLA can be formally described using the concept of a **dataflow**. The dataflow defines the strategy for moving data through the systolic array to maximize reuse. The effectiveness of a given dataflow depends on the structure of the DNN layer being executed. The two most common dataflows, which we will see are directly relevant to Gemmini, are:

- **Weight Stationary (WS):** This dataflow prioritizes weight reuse. Filter weights are pre-loaded into the PEs and remain stationary, while input activations are streamed in. This strategy, used by the Google TPU, is highly efficient for layers with large input feature maps and relatively small filters.
- **Output Stationary (OS):** This dataflow prioritizes minimizing the movement of partial sums. Each PE is responsible for accumulating the final value for a single output activation, which remains stationary in its accumulator. Inputs and weights are streamed to the PE as needed. This can be more efficient for layers where partial sum read/write energy is a dominant cost.

These strategies, visually contrasted in Figure 4.7 and 4.9, represent a fundamental trade-off in accelerator design. The choice of dataflow directly impacts which types of data reuse, shown in Figure 4.6, are most effectively exploited.

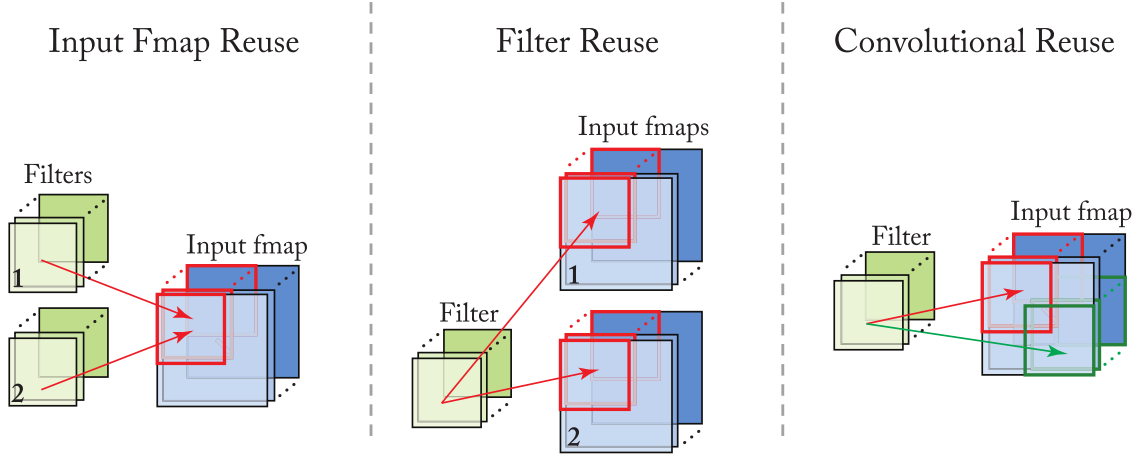


Figure 4.6: The three primary forms of data reuse in convolutional layers: convolutional, filter, and input reuse. [58].

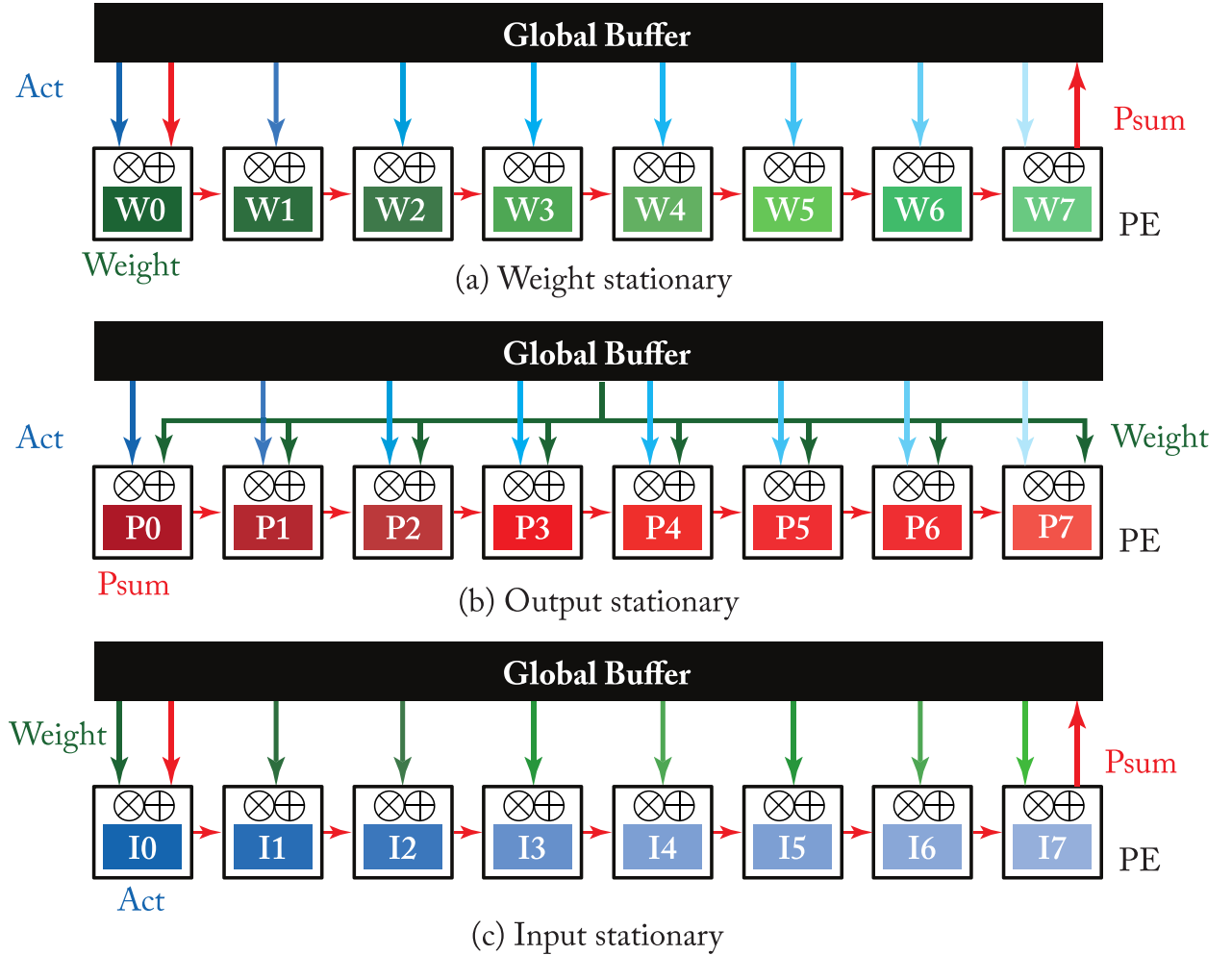


Figure 4.7: A visual comparison of Weight, Output and Input Stationary dataflows. [58].

4.5 The Gemmini Approach: A Full-Stack Research Platform

While the TPU demonstrates the immense potential of large-scale systolic arrays and the NVDLA showcases the value of modular, open-source IP, both present challenges for comprehensive, system-level academic research. The TPU is a proprietary, closed-source design, limiting deep architectural inspection. The NVDLA, while open, is often evaluated as an isolated hardware block, which can obscure the real-world performance bottlenecks that arise from interactions with the host CPU, operating system, and shared memory hierarchy [64]. Alternative approaches to RISC-V-based DNN acceleration have explored different architectural paradigms, such as heterogeneous processor designs with Very Long Instruction Word (VLIW) pipelines and Single Instruction, Multiple Data (SIMD) processing units for edge computing applications [65].

To address these challenges, this thesis utilizes **Gemmini**, an open-source, full-stack DNN accelerator generator from the University of California, Berkeley [64]. Gemmini’s design is founded on the philosophy that to truly understand an accelerator’s performance, it must be evaluated as part of a complete, functioning system. It achieves this by leveraging the Chipyard framework to generate not only the accelerator RTL but also a Linux-capable SoC, enabling the study of critical cross-stack effects such as:

- **SoC Resource Contention:** Competition for shared resources like the L2 cache and the main memory bus between the CPU and the accelerator.
- **Operating System Overhead:** The performance impact of OS services, including context switches and page fault handling.
- **Virtual Memory Interaction:** The latency and performance implications of the accelerator’s DMA engine interacting with the system’s TLBs and Page Table Walker.

Crucially, as a generator written in Chisel, Gemmini is not a single, fixed hardware design. It is a highly flexible template that can be configured at generation time to produce a wide spectrum of systolic architectures. As illustrated in Figure 4.8, by tuning its parameters, Gemmini can generate designs ranging from highly-pipelined, TPU-like structures to more loosely-coupled, NVDLA-like parallel vector engines. This architectural flexibility makes it an exceptionally powerful platform for academic research and design-space exploration.

Therefore, for the objectives of this thesis—to implement and evaluate a custom accelerator within a realistic SoC environment—Gemmini provides the ideal platform. It combines the computational power of a systolic array with the verifiability of a full-stack, open-source framework, enabling a rigorous and holistic analysis.

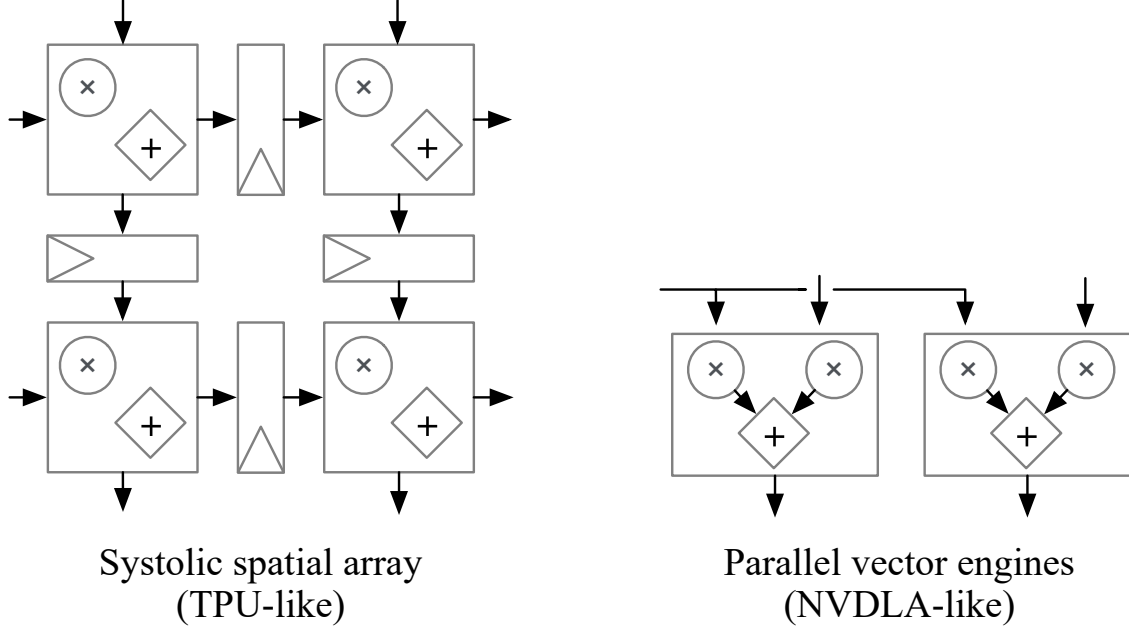


Figure 4.8: The design space enabled by the Gemmini generator, allowing it to produce a wide variety of spatial array structures, from systolic (TPU-like) to parallel vector engines (NVDLA-like). [64].

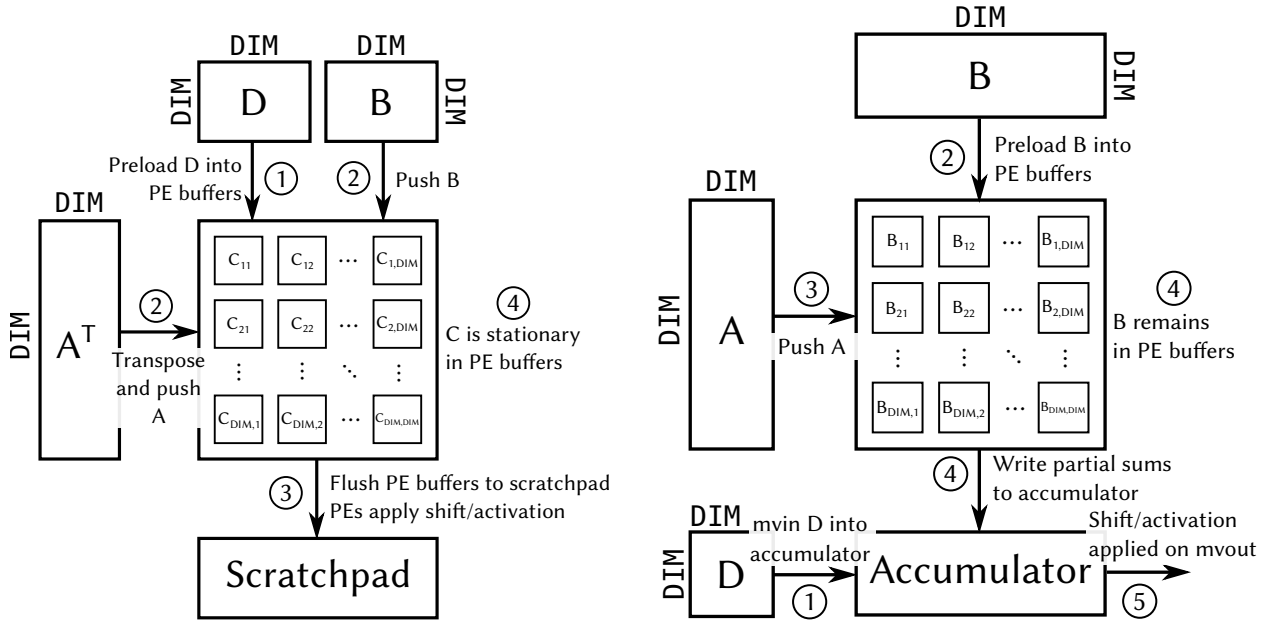


Figure 4.9: A visual comparison of the two primary dataflows supported by Gemini. The choice between these strategies represents a fundamental trade-off in accelerator design. [64].

4.6 Chapter Summary

This chapter has built a foundational argument for the necessity of specialized hardware for DNN acceleration. We began with the core architectural challenge, the Memory Wall, and introduced the systolic array as a classic and enduring solution that minimizes costly data movement by maximizing data reuse. By examining modern industrial accelerators like the Google TPU and NVDLA, we have seen this principle in action. Finally, we formalized these strategies with the concept of dataflows, such as Weight Stationary and Output Stationary. These foundational concepts provide the necessary framework to understand the design and operation of the Gemini accelerator, which will be the focus of the subsequent chapters.

CHAPTER 5: THE GEMMINI DNN ACCELERATOR: AN ARCHITECTURAL OVERVIEW

As established in the previous chapter, the System-on-Chip (SoC) developed in this thesis utilizes a single Rocket Core augmented with a specialized accelerator for deep learning workloads. This chapter delves into the architecture of this accelerator: Gemini, a sophisticated component designed to overcome the fundamental limitations of general-purpose processors for Deep Neural Network (DNN) tasks.



Figure 5.1: Gemini, the DNN Accelerator designed for SoC integration developed by UC Berkeley. [64].

5.1 Top-Level Architectural Blueprint

The Gemini generator can produce a wide range of accelerator instances based on a flexible architectural template. To understand the Gemini architecture, we first examine the high-level template shown in Figure 5.2. This blueprint illustrates the three primary components of the accelerator: the Controller, the on-chip memory (Scratchpad and Accumulator), and the compute core (Spatial Array), and their interface with the host CPU and system DRAM. The following sections provide a detailed analysis of each of these blocks.

5.2 Analysis of Core Architectural Components

5.2.1 The Systolic Compute Engine

The computational heart of Gemini is its systolic array, a highly configurable spatial array that functions as a powerful General Matrix-Matrix Multiplication (GEMM) engine. The array’s microarchitecture, as detailed in Figure 5.3, features a two-level hierarchy composed of Tiles and individual Processing Elements (PEs). Each PE is capable of performing a Multiply-Accumulate (MAC) operation in a single clock cycle. This hierarchical and configurable design allows Gemini to efficiently execute the dense matrix multiplications fundamental to modern Deep Neural Networks (DNNs).

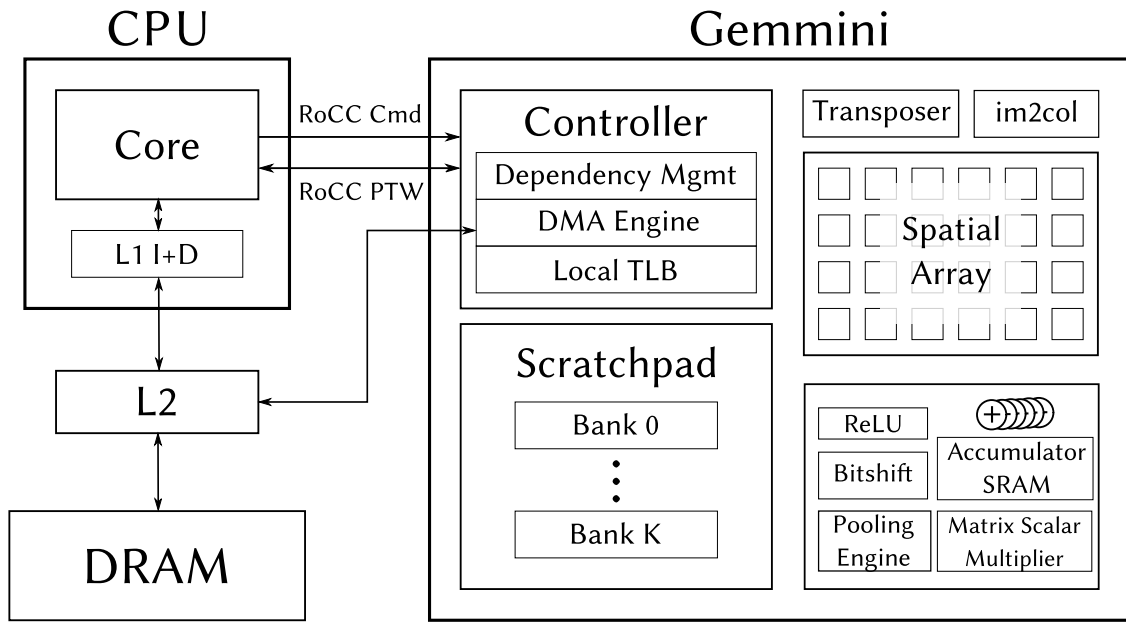


Figure 5.2: High-level overview of the Gemini accelerator template integrated with a host CPU and system memory hierarchy. [64].

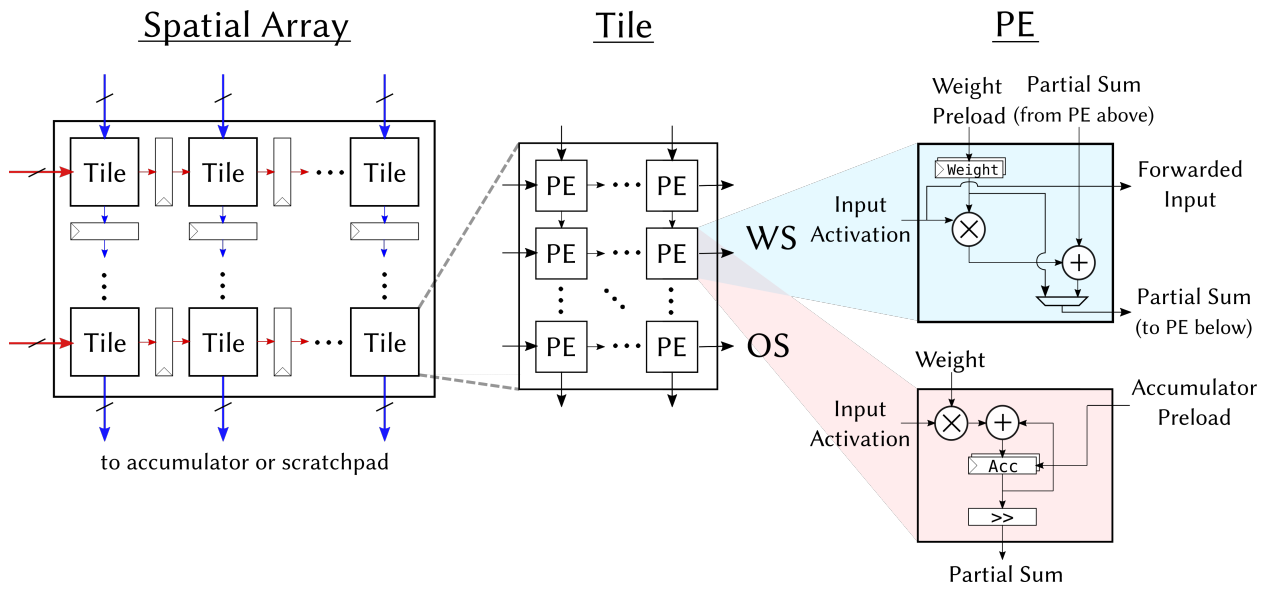


Figure 5.3: The two-level hierarchy of Gemini's spatial array, composed of tiles and Processing Elements (PEs). [64].

The key parameters of the systolic array can be specified at generation time:

- **Dimensions:** The size of the PE grid, structured into a hierarchy of tiles, can be configured (e.g., a 16x16 array composed of 2x2 tiles, each with 8x8 PEs). This allows for a trade-off between hardware area and parallel processing capability.
- **Dataflow:** Gemmini primarily supports two dataflow strategies: Weight Stationary (WS) and Output Stationary (OS).
 - In **WS dataflow**, the DNN weights are pre-loaded into the PEs and remain stationary, maximizing weight reuse as input activations are streamed through.
 - In **OS dataflow**, partial sums of the output activations remain stationary within the PEs, while both inputs and weights are streamed. This is particularly beneficial for layers with low weight reuse.

The choice of dataflow has a significant impact on performance and energy efficiency.

- **Pipelining:** The degree of pipelining within the array and its PEs can be configured, affecting the maximum clock frequency and overall throughput.

5.2.2 Mapping Convolution to Hardware: The `im2col` Transformation

While systolic arrays are fundamentally designed to accelerate General Matrix-Matrix Multiplication (GEMM), they are adept at accelerating 2D convolutions through a crucial data-restructuring technique known as `im2col` ("image-to-column"). This process unrolls the overlapping input image patches from a convolutional layer into the columns of a new, larger matrix. As shown in Figure 5.4, this new matrix can then be multiplied with a corresponding weight matrix, effectively **converting the convolution into a GEMM operation** that the systolic array can process with maximum efficiency.

Recognizing the computational cost of this pre-processing step, Gemmini can be configured with a dedicated hardware unit to perform the `im2col` transformation on-the-fly. This offloads the data restructuring from the host CPU, freeing it for other tasks and streamlining the overall computation pipeline. The decision to include this unit represents a classic area-versus-performance trade-off in the accelerator design.

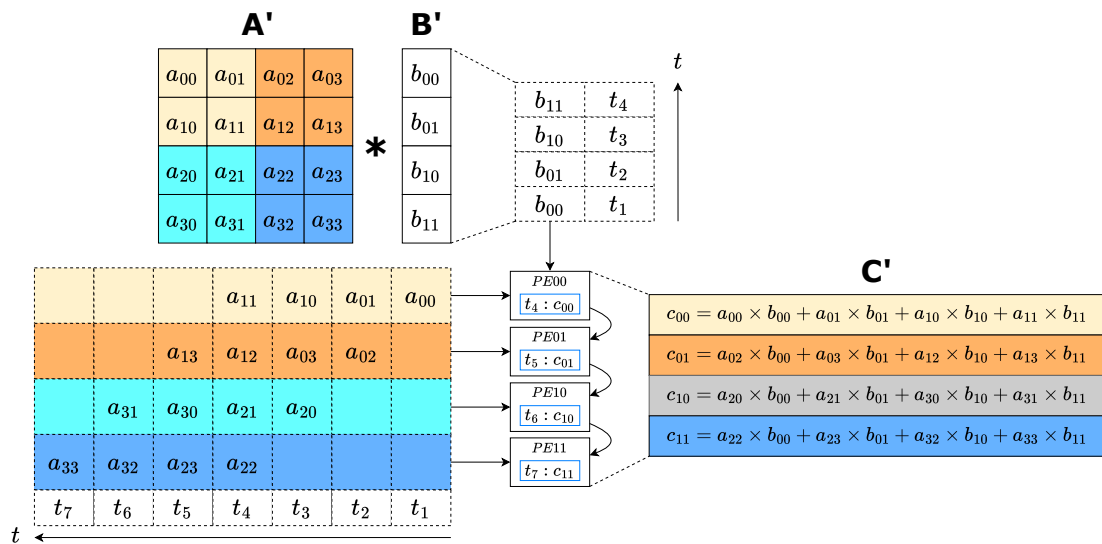
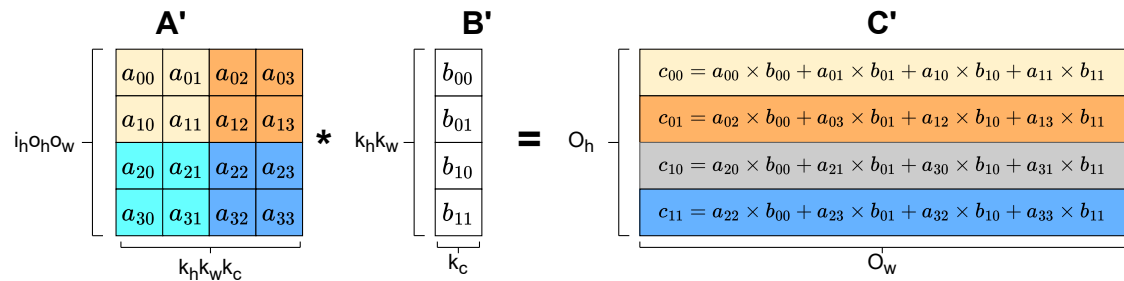
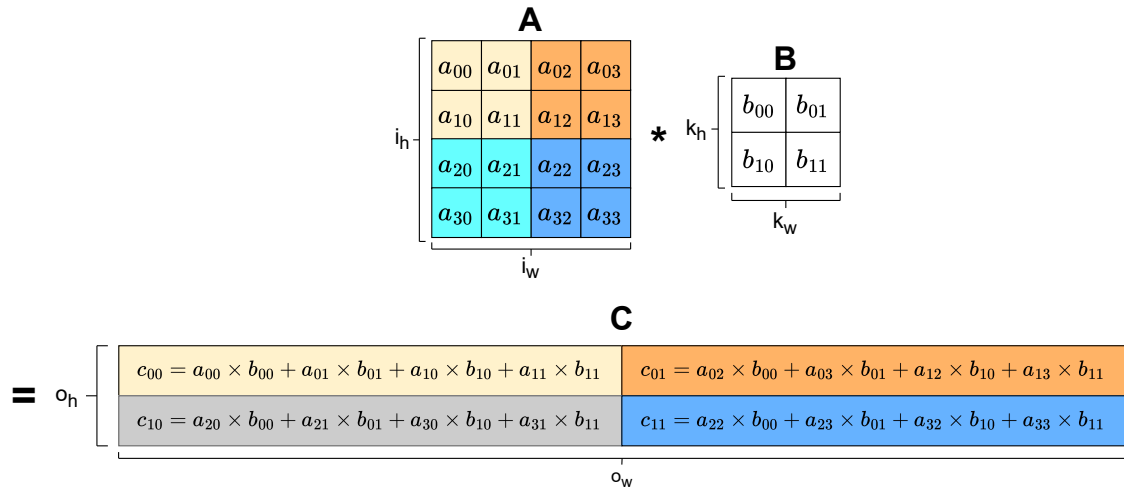


Figure 5.4: Mapping Convolution to Systolic Array Computation via the `im2col` Transformation. Adapted from Gookyi et al. [66].

5.2.3 The On-Chip Memory Subsystem

To feed its high-throughput systolic array, Gemmini includes a specialized on-chip memory system. This design represents a deliberate trade-off between hardware-managed caches and software-managed memories.

- **Scratchpad Memory:** A software-managed SRAM used for staging input activations, weights, and intermediate feature maps close to the systolic array. Its purpose is to reduce the latency and energy consumption associated with fetching data from higher levels of the memory hierarchy, such as the shared L2 cache or main memory (DRAM). Its capacity and banking are configurable.
- **Accumulator Memory:** Separate from the main scratchpad, Gemmini features a dedicated memory to store the partial sums generated by the MAC operations. This memory typically uses a higher precision data type (e.g., 32-bit integers) than the input data (e.g., 8-bit integers) to maintain accuracy during accumulation.

5.3 SoC Integration and System-Level Features

5.3.1 The RoCC Coprocessor Interface

Gemmini integrates into the Chipyard SoC via the **Rocket Custom Coprocessor (RoCC)** interface (Section 3.4.5). As a tightly-coupled coprocessor, it receives custom RISC-V instructions dispatched directly by the host Rocket Core’s pipeline. This allows for low-latency command and control.

This integration uses a decoupled access-execute model, as illustrated conceptually in Figure 5.5. The host CPU issues three main types of commands to Gemmini: load (from main memory to scratchpad), execute (perform computation on data within the scratchpad), and store (from scratchpad to main memory). These commands are placed into separate hardware queues, allowing Gemmini’s internal controller then processes these commands asynchronously, and Gemmini’s DMA engine to handle data movement in parallel with the systolic array’s computations. This overlapping of communication and computation is crucial for hiding memory latency and maximizing the utilization of the systolic array.

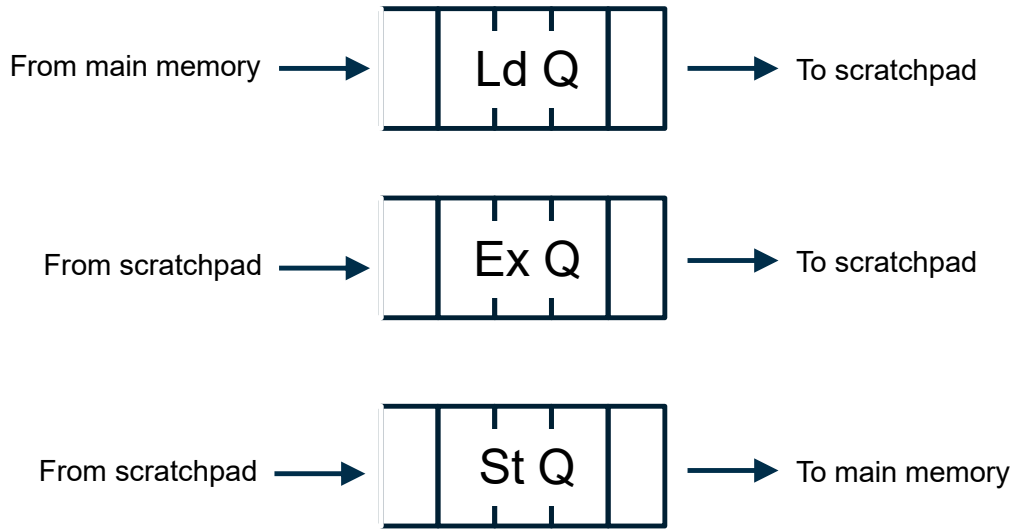


Figure 5.5: Conceptual view of Gemmini’s decoupled access-execute pipelines, managed by separate hardware command queues for Load, Execute, and Store operations. [64].

5.3.2 Virtual Memory Support

To operate correctly under a full-featured operating system like Linux, an accelerator’s DMA engine must be able to translate virtual memory addresses into physical addresses. Gemmini is designed with this "full-stack" capability. It can be configured to share the host CPU’s Page Table Walker (PTW) to perform its own address translations, and it includes its own local Translation Lookaside Buffer (TLB) to cache these translations and reduce latency.

5.4 The Multi-Level Programming Model

To cater to different user needs, from application developers to hardware architects, Gemmini supports a multi-level programming model. This stack provides layers of abstraction, allowing users to work at a level appropriate for their task, from direct hardware control to high-level framework integration.

5.4.1 Low-Level: Gemmini ISA

At the lowest level, Gemmini is controlled by its custom RoCC instruction set. These instructions are typically wrapped in C preprocessor macros for easier use from C/C++ code. The instruction set includes primitive commands like `mv in` (move data in), `mv out` (move data out), `preload` (load data into the PE array), and `compute`. It

also features more "complex" instructions like `loop_matmul` that abstract away the manual tiling and scheduling of primitive commands, offloading the loop control to hardware state machines [67].

5.4.2 Mid-Level: Hand-Tuned C Library

To abstract the complexities of direct ISA programming, Gemmini provides a C library (typically accessed via `gemmini.h`). This library offers functions for common DNN operations, such as `tiled_matmul` and `tiled_conv`. These functions implement optimized heuristics for loop tiling and data movement to maximize scratchpad utilization based on the specific hardware parameters of the generated Gemmini instance.

5.4.3 High-Level: Compiler Support (Conceptual)

The Gemmini ecosystem is designed to support higher-level compilation flows. This involves tools that can take DNN models described in standard frameworks like Open Neural Network Exchange (ONNX) and compile them down to Gemmini instructions, either by calling the mid-level C library or by directly generating low-level ISA calls. This further simplifies the deployment of DNNs on Gemmini-accelerated SoCs.

5.5 Summary

In summary, Gemmini is a sophisticated, highly configurable DNN accelerator generator that embodies the principles of systolic computation. Its key architectural features include a powerful systolic array, a specialized on-chip memory hierarchy, and a tightly-coupled integration with a host processor via the RoCC interface, including support for system-level features like virtual memory. This design enables the high-performance execution of DNN workloads within a full-stack, system-level context.

Having established this architectural overview, the following chapter will now perform a deep dive into the specifics of Gemmini's internal command processing and its multi-level programming model.

CHAPTER 6: GEMMINI'S INTERNAL ARCHITECTURE AND PROGRAMMING MODEL

While Chapter 5 described the conceptual architecture of Gemini, this chapter explores its internal structure and the software interfaces used to control it. This analysis will follow the journey of a single instruction from the host CPU down to the physical hardware, revealing how Gemini's sophisticated design translates high-level commands into high-performance computation. A deeper understanding of these implementation-level details is necessary to effectively program the accelerator and analyze its performance.

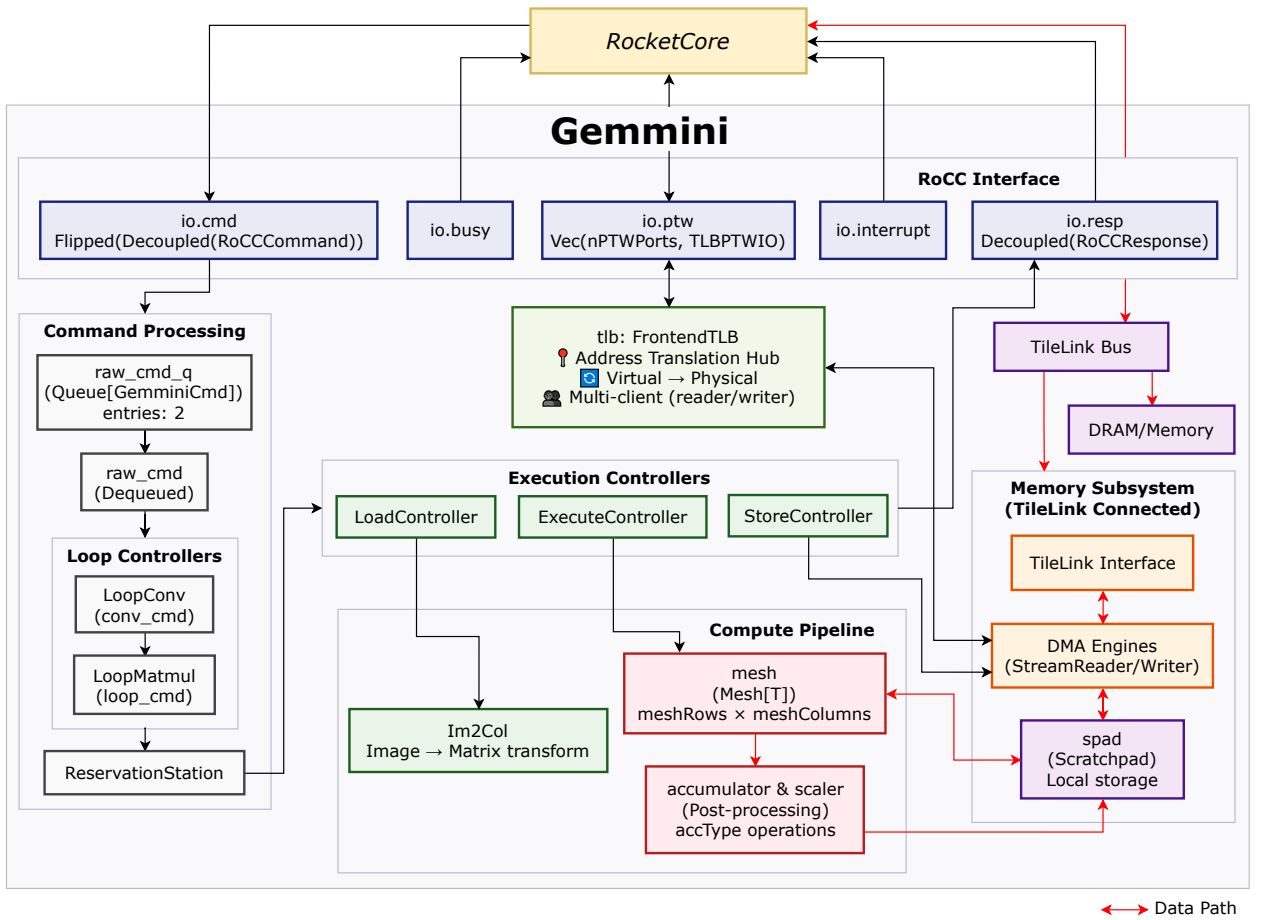


Figure 6.1: Overview of the internal modules in the generated Gemini hardware architecture, showing the flow of commands from the RoCC interface through various queues and controllers.

Figure 6.1 serves as a high-level architectural blueprint for the entire Gemini accelerator. The journey of a command begins when a **RoCCCommand** is dispatched from the **RocketCore** and arrives at the **RoCC Interface**. The command first enters

the **Command Processing** block, where it is enqueued before being passed to the **Loop Controllers**. Here, the two-level command-unrolling hierarchy (**LoopConv** and **LoopMatmul**) autonomously translates the high-level instruction into a stream of simpler, tiled micro-operations. These primitive commands are then staged in the **ReservationStation**, which acts as a central dispatch hub.

From the Reservation Station, commands are issued to the three independent **Execution Controllers**, which manage the core decoupled operations of the accelerator. The **LoadController** and **StoreController** orchestrate all data movement with the **Memory Subsystem**, issuing requests through its **TileLink Interface** to either the on-chip **spad** (Scratchpad) or off-chip DRAM. In parallel, the **ExecuteController** drives the **Compute Pipeline**, managing the optional **Im2Col** unit and issuing commands to the systolic **mesh** and the post-processing **accumulator & scaler**. This entire process is supported by system-level services, such as the **FrontendTLB**, which handles virtual memory address translation via the `io.ptw` port.

This decoupled, multi-stage architecture is the key to Gemmini’s performance, allowing long-latency memory operations to be overlapped with high-throughput computation. The subsequent sections will now explore each of these major blocks in detail.

6.1 The Command-Unrolling Hierarchy

A custom RoCC instruction sent from the Rocket core is not a simple command executed directly by the systolic array. Instead, it is a high-level, almost CISC-like directive, such as `LOOP_WS` or `LOOP_CONV_WS`, that can encapsulate an entire deep learning layer computation. The fundamental design principle of Gemmini is to offload the immense complexity of interpreting this command from the CPU into a dedicated, multi-level hardware hierarchy.

This hardware-based command unrolling is a defining characteristic of a powerful, tightly-integrated co-processor. It autonomously handles tasks that would otherwise require thousands of CPU cycles—such as loop control, address calculation, and data dependency checking—freeing the host processor for other work. This section details the two primary levels of this unrolling hierarchy: **LoopConv** and **LoopMatmul**.

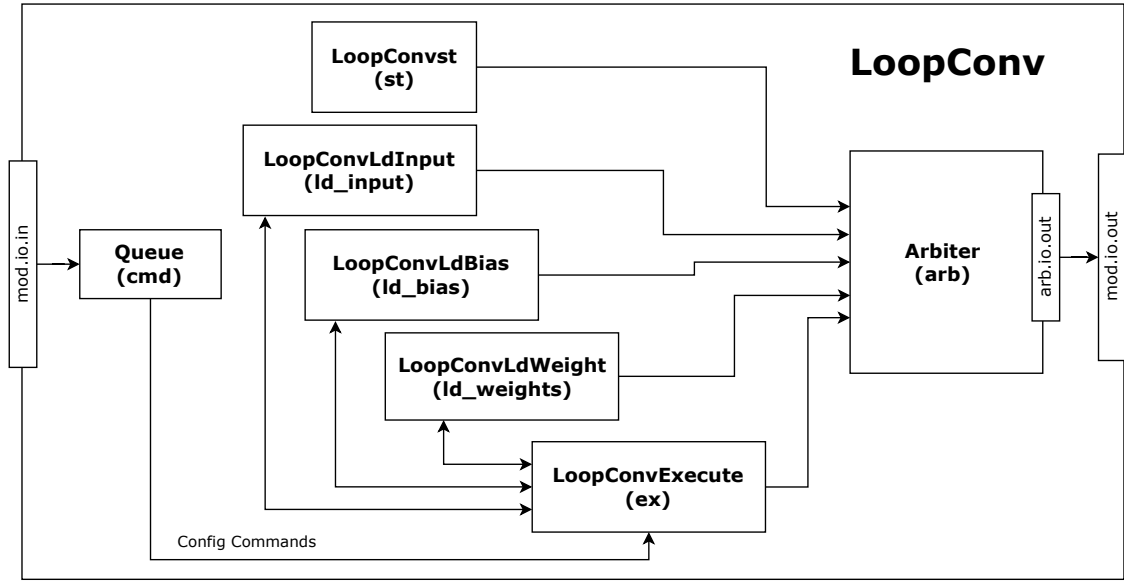


Figure 6.2: The **LoopConv** module. This is the first level of the command-unrolling hierarchy, responsible for decomposing a complex convolution operation into a series of matrix multiplication tasks.

6.1.1 Level 1: Convolution Decomposition (**LoopConv**)

The first stage in processing a high-level command is the **LoopConv** module (Figure 6.2), which functions as a Convolution Loop Unroller. Convolution operations involve complex, nested loops over batch dimensions (N), input/output channels (C), spatial dimensions (H, W), and kernel dimensions (R, S). Without hardware offloading, a programmer would need to manage this 6 to 7-dimensional loop nest, along with data tiling and resource allocation, in software.

The **LoopConv** module automates this entire process. It interprets a single high-level instruction (e.g., `gemmini_loop_conv_ws`) and decomposes the complex convolution into a sequence of simpler matrix multiplication tasks. Its internal architecture is comprised of several parallel hardware units that handle the two distinct phases of the operation: data loading and execution.

First, the data loading phase is handled by three independent, concurrent units:

- **LoopConvLdInput**: Loads input feature maps from memory.
- **LoopConvLdWeight**: Loads convolution kernel weights from memory.
- **LoopConvLdBias**: Loads bias values from memory.

Once the required data has been fetched, the execution phase begins, managed by two final components:

- **LoopConvExecute:** Executes the convolution operation by passing tasks to the **LoopMatmul** module.
- **Arbiter:** A 5-to-1 arbiter that manages access to the memory interface, ensuring that the parallel load and execute units do not create resource contention.

These generated tasks are then passed down to the next level of the command-unrolling hierarchy.

6.1.2 Level 2: Tiled Matrix Multiplication (**LoopMatmul**)

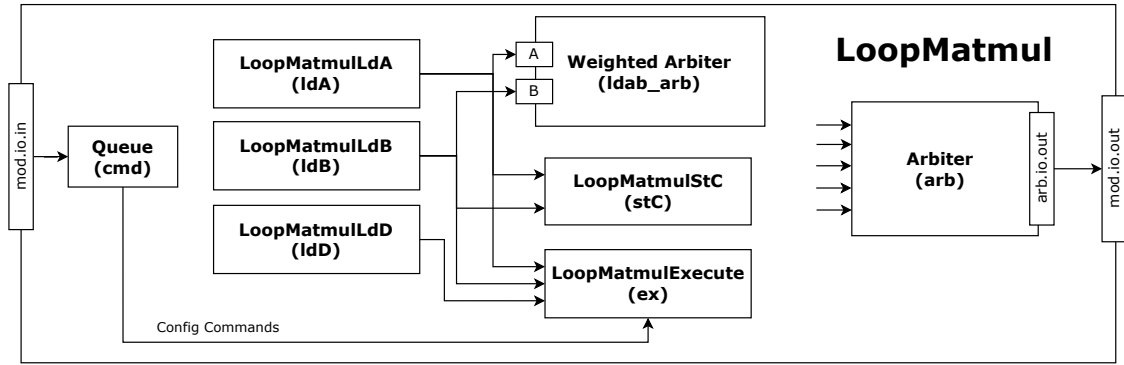


Figure 6.3: The **LoopMatmul** module. This second-level unroller translates matrix multiplication tasks into a stream of smaller, tiled micro-operations for the physical hardware.

The **LoopMatmul** module (Figure 6.3) receives the sequence of matrix multiplication jobs from **LoopConv** and performs the second level of command unrolling. Its primary responsibility is to break these matrix-level operations down into smaller, fixed-size tiles that perfectly match the physical dimensions of the systolic array. This module is effectively a micro-code engine that processes five distinct types of operations—ranging from configuration (**CONFIG_CMD**) and memory transfers (**LOAD3_CMD**, **STORE_CMD**) to the core computational directives (**PRELOAD_CMD**, **COMPUTE_CMD**). This section delves into the internal architecture and policies that allow it to manage this complexity.

6.1.2.1 Internal Architecture and Decoupled Execution

The internal structure of the **LoopMatmul** controller is composed of several independent hardware modules, each responsible for a specific part of the execution pipeline:

- **LoopMatmulLdA**: Loads matrix A from DRAM to scratchpad memory.
- **LoopMatmulLdB**: Loads matrix B (weights) from DRAM to scratchpad memory.
- **LoopMatmulLdD**: Loads bias/accumulator data from DRAM to accumulator memory.
- **LoopMatmulExecute**: Performs the actual matrix multiplication computation.
- **LoopMatmulStC**: Stores results from the accumulator back to DRAM.
- **LoopMatmulStCSpad**: Stores results from the accumulator to scratchpad memory.

While these modules operate independently, they are coordinated through a lightweight handshake mechanism referred to in the design as iterator synchronization. For instance, the **LoopMatmulExecute** unit will not begin computation on a tile of data until it receives a signal that both **LoopMatmulLdA** and **LoopMatmulLdB** have successfully loaded the required operands into the scratchpad. This allows the Load modules to work ahead, fetching data for a future computation (**tile i+1**) while the **Execute** module is still busy processing the current one (**tile i**).

This decoupled architecture is a deliberate and critical design choice for overcoming the "memory wall". The high latency of fetching data from off-chip DRAM is a primary performance bottleneck in any accelerator. By overlapping the long-latency memory operations (Load and Store) with the core computation (Execute), the system effectively hides this latency. This ensures that the systolic array—the most valuable computational resource—is starved for data as infrequently as possible, maximizing its utilization and achieving a throughput which would be unattainable with a simple, rigid pipeline.

6.1.2.2 Resource Management and Arbitration Priority

The parallel operation of these internal modules inevitably leads to contention for shared resources. To manage this, the **LoopMatmul** controller employs a hardware arbiter that enforces a fixed-priority scheduling policy. This policy is not arbitrary; it is carefully designed to maintain pipeline flow and prevent deadlocks. The priority scheme is as follows:

1. **Highest Priority - Store to DRAM:** Commands from the `LoopMatmulStC` module receive top priority. This is essential to prevent accumulator buffer overflow and free up internal resources for subsequent computations.
2. **Execute Operations:** The `LoopMatmulExecute` module’s preload and compute commands receive second priority to keep the systolic array actively computing.
3. **Load Bias/Accumulator:** The `LoopMatmulLdD` module’s commands for loading bias data receive third priority.
4. **Load Input Matrices:** A weighted sub-arbiter manages requests from `LoopMatmulLdA` and `LoopMatmulLdB` modules with fourth priority.
5. **Lowest Priority - Store to Scratchpad:** The `LoopMatmulStCSpad` module receives the lowest priority, as scratchpad stores are internal operations that are less critical to overall pipeline progression than DRAM transfers.

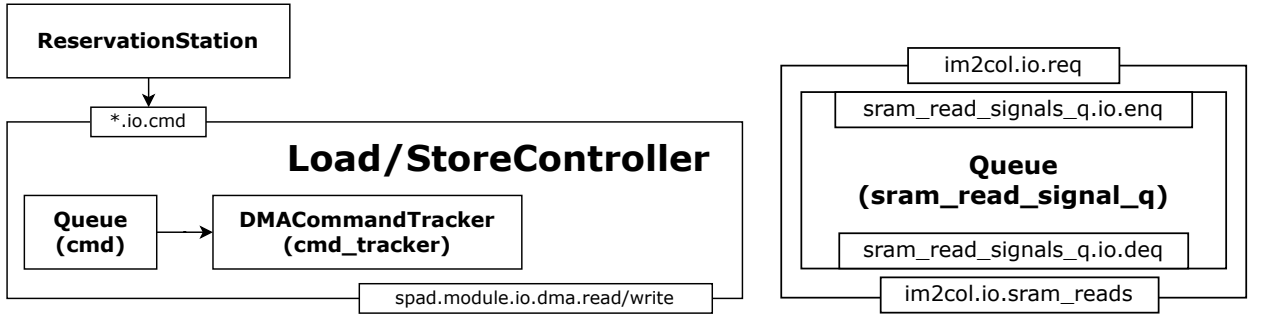
This priority scheme ensures that result data is promptly written to DRAM to free accumulator resources, while maintaining sufficient data throughput to keep the computational units busy. The impressive result of this entire hierarchical process is that the `LoopMatmul` module transforms a single high-level command, like `LOOP_WS(matmul 1024x1024)`, into **thousands** of precisely calculated, optimally ordered, resource-aware hardware operations ready for immediate execution by the back-end controllers.

6.2 Decoupled Execution and Resource Management

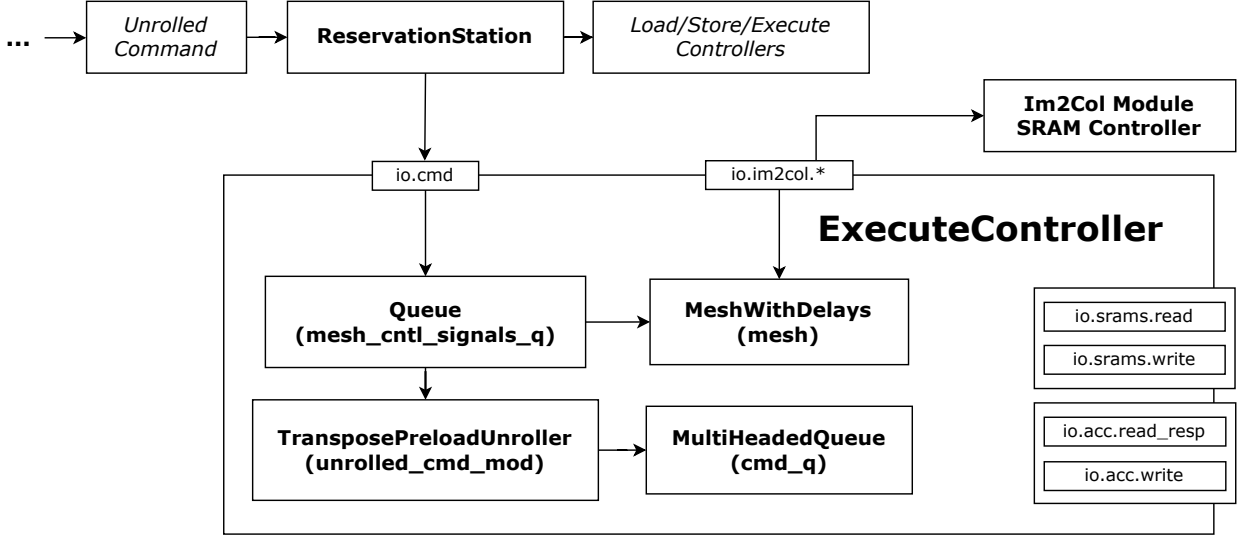
The stream of primitive micro-operations generated by the command-unrolling hierarchy is not fed into a rigid, lock-step pipeline. Instead, Gemmini employs a more sophisticated and powerful paradigm: decoupled execution. This design allows the various stages of a complex operation—loading data, executing computation, and storing results—to proceed concurrently and asynchronously, managed by a central dispatch hub and a set of independent back-end controllers.

6.2.1 The Reservation Station: A Central Dispatch Hub

At the heart of the decoupled back-end is the Reservation Station. It receives a stream of commands from the `LoopMatmul` module that are far more than sim-



(a) The Load and Store Controllers manage data movement between scratchpad and DRAM via a DMA tracker. (b) Im2Col module (an SRAM Controller)



(c) The Execute Controller, which issues commands to the systolic array (Mesh) and im2col units.

Figure 6.4: The primary back-end controllers that receive commands from the Reservation Station.

ple instructions; they are complete, resource-aware directives. These directives include optimally sequenced commands, complete address calculations for DRAM and scratchpad, resource-aware flow control to prevent buffer overflows, and attribution metadata for debugging. The Reservation Station’s primary role is to act as a central orchestrator, managing data dependencies and dispatching these directives to the appropriate hardware units.

This enables a form of out-of-order execution, where a command can be dispatched as soon as its required data and hardware resources are available. The primitive commands it dispatches correspond to the fundamental actions of the accelerator pipeline, including loading data (LOAD_CMD, LOAD2_CMD, LOAD3_CMD), preloading weights into the array (PRELOAD_CMD), initiating the core computation (COMPUTE_AND_FLIP_CMD),

and writing results back to memory (STORE_CMD).

6.2.2 Back-End Controllers: Load, Store, and Execute

From the Reservation Station, primitive commands are dispatched to one of three specialized, independently operating back-end controllers.

- **LoadController**: Handles all data loading operations, fetching data from DRAM to the scratchpad memory.
- **StoreController**: Manages data storage operations, writing results from the scratchpad back to DRAM.
- **ExecuteController**: Orchestrates the actual computation, issuing commands to the systolic array and managing on-the-fly data transformations.

The **LoadController** and **StoreController** (Figure 6.4a) are focused exclusively on managing the high-latency operations of the memory hierarchy. The **ExecuteController** (Figure 6.4c), however, contains a sophisticated collection of its own internal hardware modules, including the systolic array mesh itself, an **im2col** unit (Figure 6.4b), and various queues for command buffering. (A detailed breakdown of the **ExecuteController**'s internal sub-modules is provided in Appendix A.1).

This clear division of labor, orchestrated by the Reservation Station, is the cornerstone of the accelerator's performance. It allows long-latency memory operations and intense computation to overlap, effectively hiding the "memory wall" and ensuring the valuable systolic array is kept maximally utilized.

6.3 The Physical Hardware: Compute and Memory

The control-flow mechanisms described above orchestrate the work performed by the two foundational hardware components of the accelerator: the compute engine that performs the mathematical operations, and the memory subsystem that feeds it.

6.3.1 The Processing Element (PE) and Distributed Arithmetic

The fundamental building block of Gemmini's compute engine is the **Processing Element (PE)**. Each PE is a self-contained unit capable of performing a Multiply-Accumulate (MAC) operation, which is the core computation of a neural network. As shown in Figure 6.5, the PE is designed with the flexibility to support different

dataflow strategies, such as Weight Stationary (WS) and Output Stationary (OS), allowing it to be optimized for different layer types.

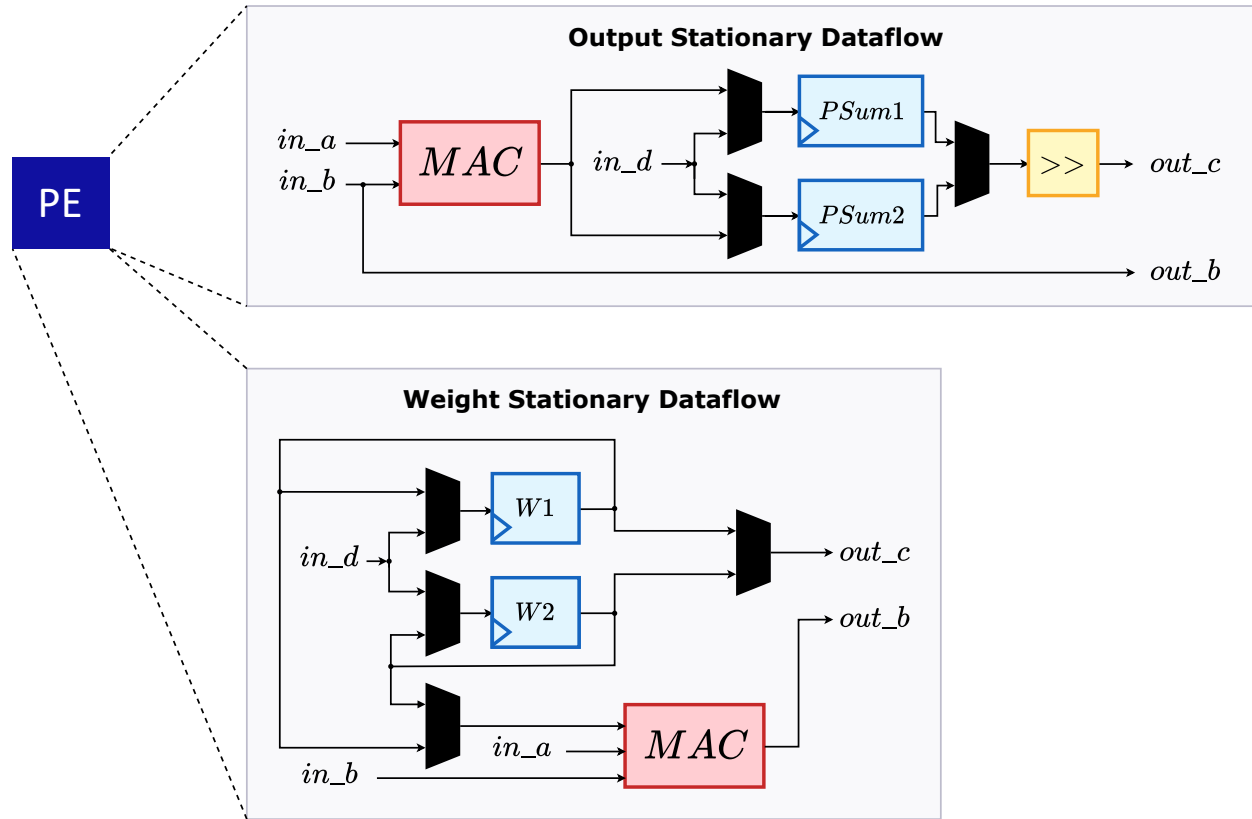


Figure 6.5: A single Processing Element (PE), the fundamental compute unit. It contains a Multiply-Accumulate (MAC) unit and the necessary logic to support multiple dataflows, such as Output Stationary and Weight Stationary.

A critical design choice in Gemmini is its approach to floating-point arithmetic. Gemmini avoids using a traditional, centralized Floating Point Unit (FPU). Instead, it implements a **distributed floating-point architecture** where arithmetic capabilities are embedded within every single PE. This is achieved through a powerful abstraction layer known as the `Arithmetic.scala` typeclass, which provides a full suite of IEEE 754-compliant operations by leveraging the Berkeley HardFloat library. The MAC unit at the heart of each PE (Figure 6.6) utilizes this typeclass, effectively making every PE a specialized FPU. This design transforms the systolic array into a massively parallel floating-point computation engine, tailor-made for the demands of modern DNNs.

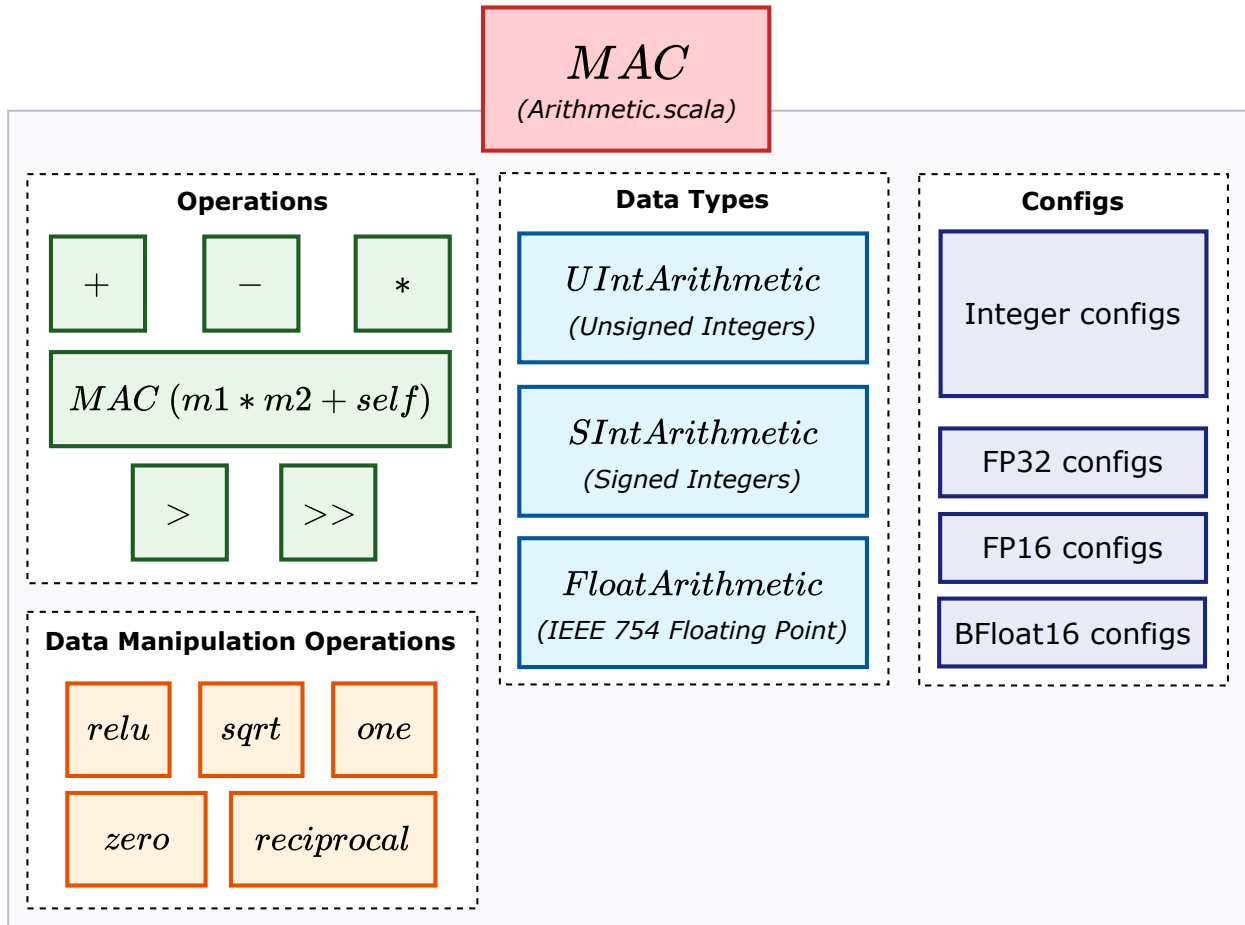


Figure 6.6: The Multiply-Accumulate (MAC) unit architecture. It is implemented using the `Arithmetic.scala` typeclass, which provides a suite of operations and supports multiple data types, including integers and IEEE 754 floating-point numbers.

6.3.2 The Tile and The Mesh: A Tale of Two Hierarchies

Individual PEs are not placed directly into a single large array. Instead, they are organized into a two-level hierarchy of **Tiles** and a **Mesh** (Figure 6.7). The design decision of how registers are used at each level is central to the entire accelerator’s performance.

6.3.2.1 The Tile: A Combinational Block of PEs

A Tile is a two-dimensional array of PEs. The defining characteristic of a Tile is that it is **purely combinational**, meaning there are no pipeline registers between the PEs within a single Tile. This design choice, evident in the hardware generation code where PE inputs are connected directly, is made to minimize local latency and reduce the area and power overhead of registers. Grouping PEs into a combinational

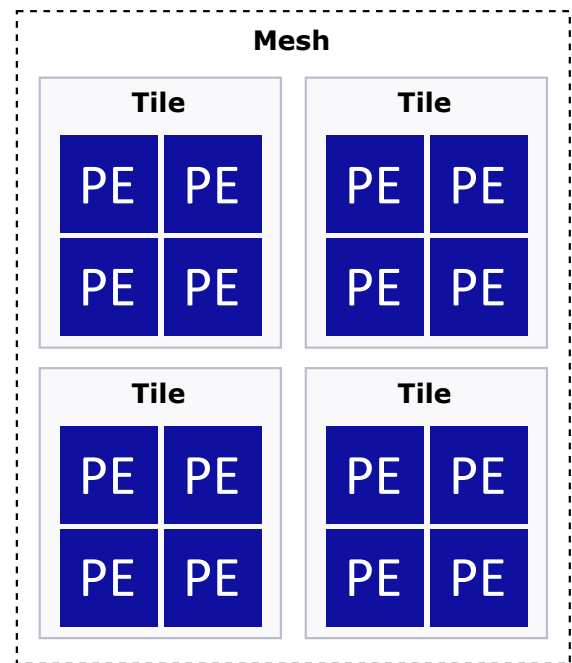
Tile allows for efficient broadcasting of input values to multiple PEs, promoting local data reuse.

6.3.2.2 The Mesh: A Pipelined Systolic Array of Tiles

The Mesh is a two-dimensional array of Tiles. Unlike the combinational logic within a Tile, the Mesh is **explicitly pipelined**, with shift registers placed between each Tile. This is the key that creates the systolic behavior of the entire array. These registers create the "systolic heartbeat," allowing waves of data to flow across the array in a controlled, rhythmic fashion. This pipelining between Tiles is what enables high throughput and scalability, as it breaks up long combinational paths and allows every Tile in the array to be actively computing on a different piece of data simultaneously.



(a) A 4x4 mesh of 1x1-PE tiles. This configuration has 16 tiles and a deep pipeline, favoring a higher clock frequency.



(b) A 2x2 mesh of 2x2-PE tiles. This configuration has only 4 tiles and a shallower pipeline, favoring lower latency at the cost of clock speed.

Figure 6.7: Different Mesh configurations, illustrating the trade-off between the number of tiles and the number of PEs per tile, while maintaining the same total number of 16 PEs.

6.3.3 Architectural Trade-offs in Mesh Configuration

The Gemmini generator allows for configuring the hierarchy of the Mesh, offering a classic architectural trade-off between latency and clock frequency. For a system with a fixed number of PEs (e.g., 256), one can choose between having many small tiles or a few large tiles, as shown in Figure 6.7.

- **Many Small Tiles (e.g., a 16x16 mesh of 1x1-PE tiles):** This configuration maximizes the number of pipeline stages. The combinational logic path within each stage is very short (just a single PE), which allows for a **higher maximum clock frequency**. However, it takes more clock cycles for a single piece of data to traverse the entire array, resulting in **higher pipeline latency**. This configuration is pipeline-optimized.
- **Few Large Tiles (e.g., a 4x4 mesh of 4x4-PE tiles):** This configuration has fewer, deeper pipeline stages. The combinational path within each stage is much longer (passing through multiple PEs), which limits the **maximum clock frequency**. However, an operation can pass through the entire array in fewer clock cycles, leading to **lower pipeline latency**. This configuration is throughput-optimized for batch processing.

The default configurations for Gemmini typically favor smaller (1x1) tiles, representing a design sweet spot that prioritizes high-frequency operation, which is often easier to implement and validate in a final ASIC or FPGA design.

6.3.4 The Scratchpad: An Intelligent Memory Subsystem

The Scratchpad (Figure 6.8) implements Gemmini’s on-chip memory system, serving as the central data hub that bridges the gap between high-latency external DRAM and the high-throughput systolic array. Far more than simple storage, the Scratchpad is a sophisticated data movement engine with integrated processing capabilities, dual data paths, and complex arbitration logic, enabling the high-throughput, low-latency data access patterns essential for efficient matrix computation. Its design demonstrates the critical importance of specialized memory subsystems in achieving peak performance in domain-specific accelerators.

The Scratchpad exposes two primary interfaces to the rest of the system: an **External Memory Interface** that connects to DRAM via the TileLink bus and

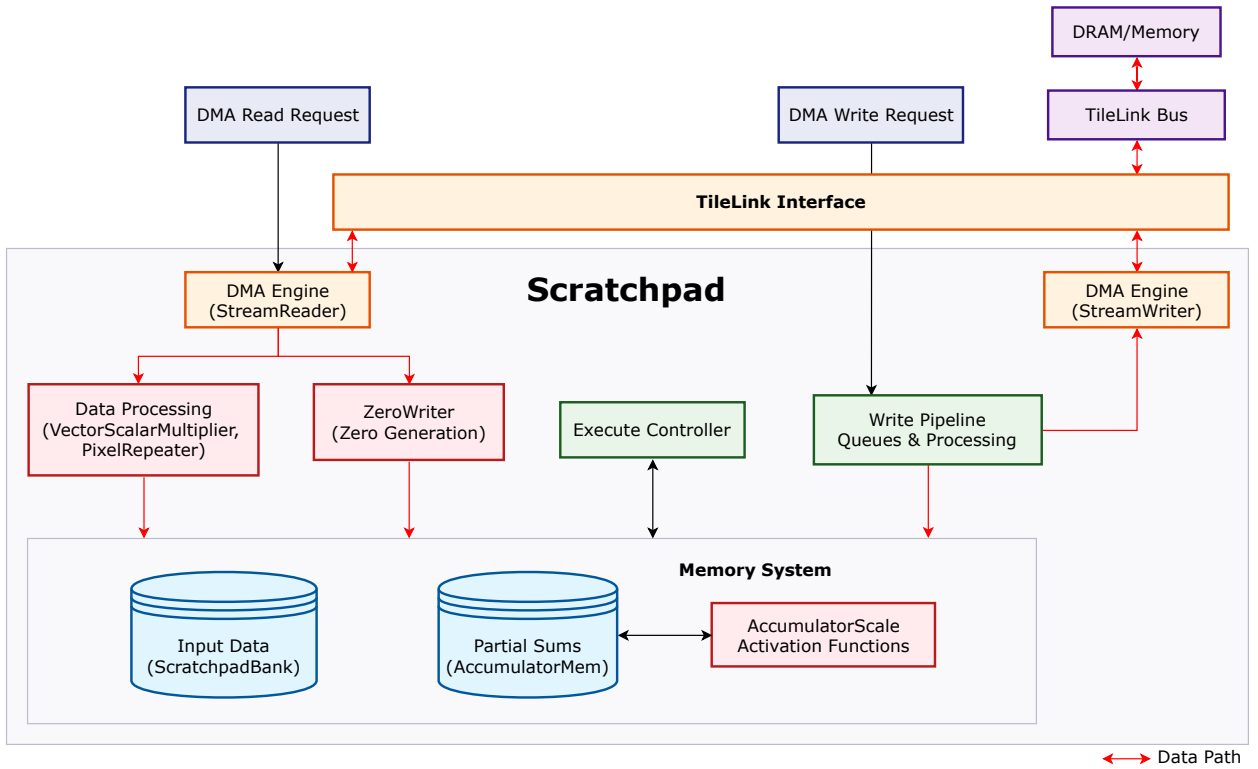


Figure 6.8: The internal structure of the Scratchpad module, an intelligent memory subsystem with integrated DMA engines and data processing units that serves the systolic array.

DMA engines, and a **Compute Unit Interface** that provides direct, low-latency access for the systolic array.

Internally, its architecture is founded on a set of specialized **Memory Banks**. Multiple **ScratchpadBanks** store input data like weights and activations, with their banked structure enabling the parallel access patterns necessary for feeding the array. In parallel, dedicated **AccumulatorMem** banks store the higher-precision partial sums generated during computation, featuring integrated hardware to support activation functions and scaling operations.

Data movement between these internal banks and external memory is managed by dedicated **DMA Engines**, the **StreamReader** and **StreamWriter**. Critically, the Scratchpad contains several **Integrated Data Processing Units** that operate on data as it moves, a key design choice that significantly reduces CPU overhead. These units perform a range of on-the-fly transformations, including scaling for quantization, pixel repetition for convolution optimizations, and applying activation functions. (A detailed breakdown of these processing units is provided in Appendix A.2).

Ultimately, the sophisticated architecture of the Scratchpad—with its combina-

tion of banked memory, integrated data processing, and dedicated DMA engines—is a direct and powerful answer to the "memory wall" problem. It is an active, intelligent subsystem engineered not just to store data, but to transform and deliver it, ensuring the massively parallel Mesh is constantly supplied with operands and enabling the high-throughput computation that is the central goal of this work.

6.4 Chapter Summary

This chapter has provided a deep dive into the internal architecture of the Gemini accelerator, tracing the path of a command from the RoCC interface to the physical compute units. We began by examining the two-level command-unrolling hierarchy, where high-level directives are translated into primitive micro-operations by the `LoopConv` and `LoopMatmul` modules. We then analyzed the decoupled execution model, orchestrated by the central Reservation Station, which allows for the concurrent operation of Load, Store, and Execute controllers to hide memory latency. Finally, we explored the physical hardware, including the distributed arithmetic in the Processing Elements and the configurable tile-and-mesh hierarchy of the systolic array.

This detailed analysis of Gemini’s internal architecture completes the theoretical and design-level exploration of our system. The subsequent chapter transitions from this architectural blueprint to practice, documenting the complete implementation process—from SoC configuration and hardware generation to the final system deployment and testing on the target FPGA platform.

CHAPTER 7: SYSTEM IMPLEMENTATION AND TESTING

This chapter details the implementation of the RISC-V System-on-Chip (SoC) featuring a Gemmini accelerator. The preceding chapters have established the theoretical foundations of the RISC-V ISA, the Chipyard framework, and the architectural principles of DNN acceleration. We now transition from theory to practice, documenting the complete process from hardware configuration and generation to final system validation on an FPGA platform. This chapter will cover the specific SoC configuration, the integration of the Gemmini accelerator as a Rocket Custom Coprocessor (RoCC), the software stack required to boot a full Linux operating system, the automated build process, and finally, the verification of the complete system on a Xilinx VC707 FPGA board. This practical account serves as a tangible demonstration of the maturity of the open-source, full-stack design methodology adopted for this research.

7.1 Hardware Implementation: SoC Configuration

The foundation of our hardware design is the Chipyard SoC generation framework, which provides a flexible and powerful environment for constructing complex RISC-V systems. The result of this process is a complete, Linux-capable SoC, whose top-level architecture is illustrated in Figure 7.1. This diagram encapsulates the full-stack integration central to this thesis, depicting the system from the processor core and accelerator down to the specific peripheral IP cores and their connections to the physical hardware on the FPGA board.

At the heart of the system is the **RocketTile** (Figure 7.1), which contains the 64-bit RISC-V **RocketCore** and its private L1 instruction and data caches. Tightly coupled to the processor pipeline, as discussed in Chapter 5, is the **Gemmini RoCC Accelerator**. This integration via the RoCC interface allows for low-latency command dispatch from the CPU to the accelerator, a critical feature for high-performance operation.

The **RocketTile** communicates with the rest of the system through the coherent **SystemBus**, which is implemented using the **TileLink** protocol. This bus serves as the main interconnect for all major subsystems. The memory hierarchy extends from the L1 caches to a unified L2 cache and finally to the off-chip DDR3 memory. To bridge the on-chip **TileLink** protocol with the industry-standard **AXI** protocol used by the external memory controller IP, a **TileLink to AXI4 bridge** is automatically

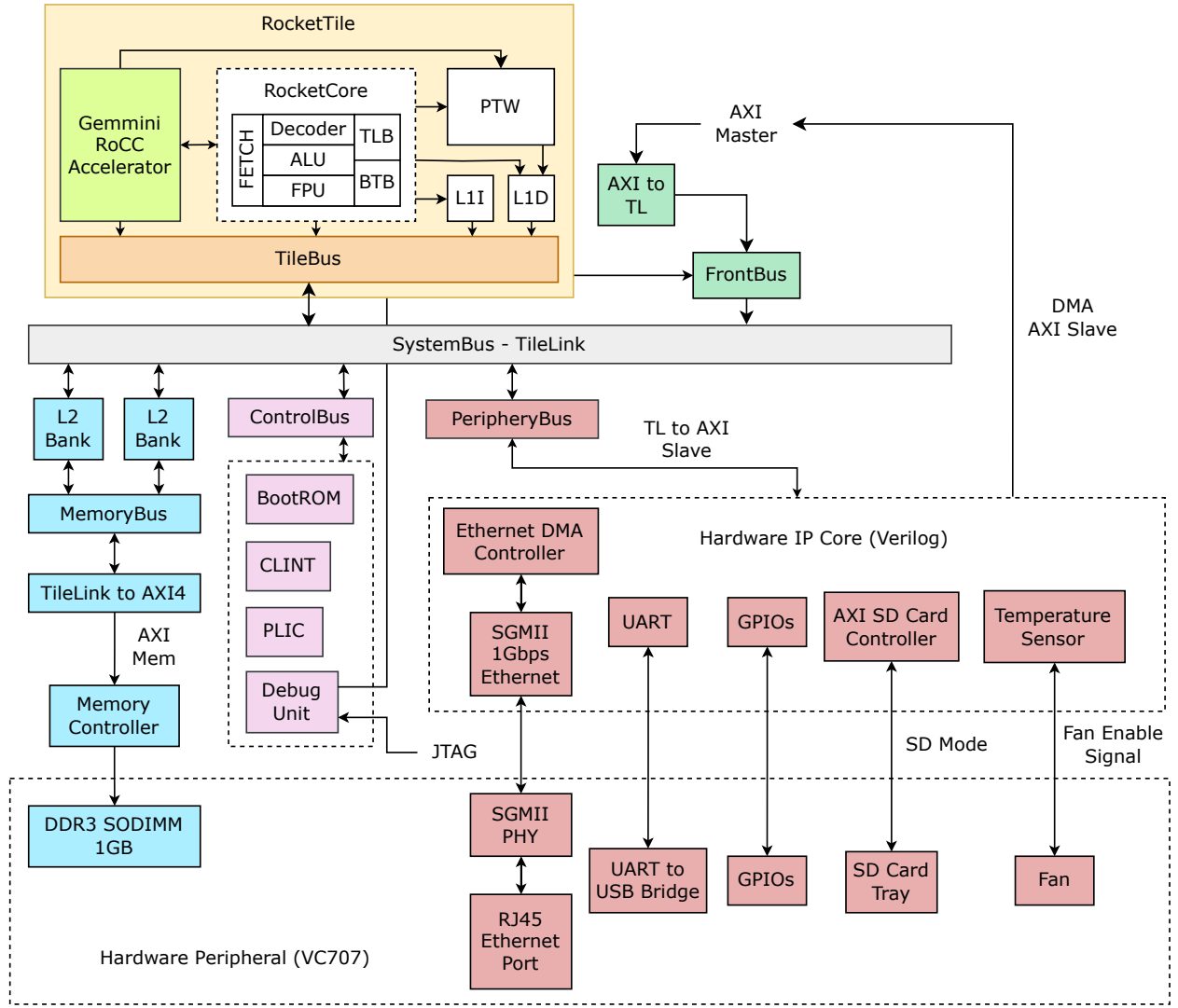


Figure 7.1: The implemented Rocket SoC with Gemini on the VC707 FPGA board. The Rocket Core is connected to a 16x16 Gemini systolic array, and various peripherals are integrated via AXI interfaces.

instantiated by the framework. This demonstrates a practical aspect of modern SoC design where components with disparate interface standards are seamlessly integrated.

To create a fully functional, Linux-capable computing platform, a rich set of peripherals is integrated via the **PeripheryBus**. Similar to the memory interface, a **TL to AXI Slave bridge** connects the **TileLink** bus to a set of **AXI**-based peripheral controllers. These include an **AXI SD Card Controller** to enable booting from an SD card, a **UART** controller for console I/O, and a complete Ethernet subsystem for network connectivity. These controller IPs, written in **Verilog**, are then wired to their corresponding physical counterparts on the VC707 board, such as the SD card tray and the RJ45 Ethernet port.

7.1.1 The Rocket64b1gem16jtag Configuration

The comprehensive architecture shown in (Figure 7.1) is not designed by hand in Verilog, but is programmatically generated from a high-level Scala configuration. As outlined in Chapter 3, this generator-based approach allows for rapid architectural exploration and ensures design consistency.

The specific configuration for our SoC is named `Rocket64b1gem16jtag`. This name concisely describes its key features: a 64-bit Rocket Core (`Rocket64`), one "big" core (`b1`), and a 16x16 Gemmini systolic array (`gem16`). This configuration is defined in Scala using Chipyard's `Config` classes, which employ a `mixin`-based approach to compose system components. This powerful paradigm allows features to be incrementally added to a base configuration through a series of traits, each beginning with the `With...` prefix. The core of the configuration is shown in Listing 7.1.

```
1 class Rocket64b1gem16jtag extends Config(  
2   new WithGemmini(16, 64) ++  
3   new WithInclusiveCache ++  
4   new WithNBreakpoints(8) ++  
5   new WithNBigCores(1) ++  
6   new WithBootROMFile("workspace/bootrom.img") ++  
7   new WithExtMemSize(0x380000000L) ++  
8   new WithNExtTopInterrupts(8) ++  
9   new WithDTS("freechips,rocketchip-vivado", Nil) ++  
10  new WithDebugSBA ++  
11  new WithJtagDTM ++  
12  new WithEdgeDataBits(64) ++  
13  new WithCoherentBusTopology ++  
14  new WithoutTLMonitors ++  
15  new BaseConfig)
```

Listing 7.1: The `Rocket64b1gem16jtag` configuration class.

The `WithGemmini(16, 64)` trait is the most critical for this work. It instructs the generator to instantiate the Gemmini accelerator with a specific set of hardware parameters derived from the `defaultConfig` within the Gemmini generator source code. The generated accelerator is a sophisticated system in its own right, comprising several key hardware blocks:

- `WithNBigCores(1)`: Instantiates a single, high-performance 64-bit Rocket Core. The Rocket Core was selected for its maturity and stability within the open-

source ecosystem, providing a reliable foundation for our system.

- **WithGemmini(16, 64):** This is the crucial trait that integrates the Gemmini DNN accelerator, specifying a 16x16 systolic array mesh and a 64-bit wide DMA bus. This trait instructs the generator to instantiate a complete accelerator system based on its internal `defaultConfig`. The resulting hardware is a sophisticated system-within-a-system, visualized in Figure 7.2, comprising:
 - **A Systolic Array Core:** A 16x16 grid of Processing Elements (PEs), each capable of an 8-bit integer multiply and a 32-bit integer accumulate per cycle. The array supports both Weight Stationary (WS) and Output Stationary (OS) dataflows.
 - **An On-Chip Memory System:** A 256KB, 4-banked scratchpad memory for staging data and a separate 64KB, dual-ported accumulator memory for partial sums.
 - **Control and Execution Logic:** A Reservation Station to enable out-of-order execution of primitive commands, managed by decoupled Load, Store, and Execute controllers.
 - **Specialized Hardware Units:** A hardware Im2Col unit to accelerate convolution preprocessing and a 4-entry Translation Lookaside Buffer (TLB) to support virtual memory.
- **WithInclusiveCache:** Adds a last-level inclusive L2 cache to the memory hierarchy. This component is vital for performance, serving as a large, shared buffer that reduces the frequency of costly off-chip memory accesses, thereby helping to mitigate the "memory wall" problem.
- **WithExtMemSize(0x380000000L):** Configures the size of the external DRAM to 14 GiB. A large main memory is essential for running a full operating system like Debian Linux, which requires significant space for the kernel, user-space applications, and runtime data.
- **WithBootROMFile(...):** Specifies the path to the boot ROM image. This ROM contains the first-stage bootloader code that the processor will execute on reset, initiating the system boot sequence.

This Scala configuration is then consumed by the Chipyard generator-based framework to produce the concrete hardware implementation. The resulting top-level ar-

chitecture of the SoC, as deployed on the Xilinx VC707 FPGA, is illustrated in Figure 7.1.

7.2 Gemmini Accelerator Configuration and Integration

The theoretical architecture of the Gemmini accelerator and its RoCC co-design methodology were detailed in Section 3.4.5, Chapter 5, and 6. This section now documents the practical steps taken within the Chipyard framework to instantiate, configure, and integrate this accelerator into the final SoC design. The process is driven by high-level Scala configuration classes that programmatically generate the final hardware.

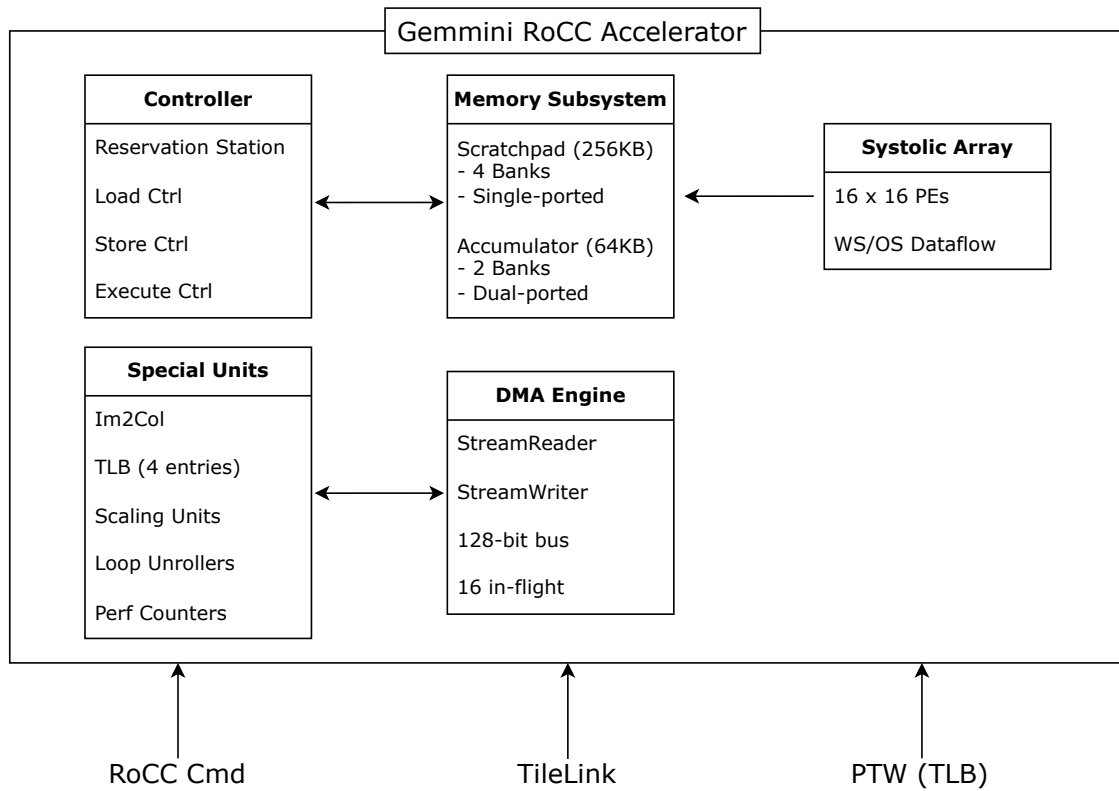


Figure 7.2: Gemmini Configuration on Rocket64b1gem16jtag SoC.

7.2.1 Accelerator Instantiation via the RoCC Interface

The primary mechanism for integrating a tightly-coupled accelerator in Chipyard is through the RoCC interface, as described in Section 3.4.5. In our design, this is achieved by using the `WithGemmini` configuration trait, shown in Listing 7.2. This class instructs the Chipyard generator to perform two critical actions: first, to in-

stantiate the Gemmini accelerator as a `LazyModule`, and second, to connect it as a RoCC device.

```

1 class WithGemmini(mesh_size: Int, bus_bits: Int) extends Config((site,
  here, up) => {
2   case BuildRoCC => up(BuildRoCC) ++ Seq(
3     (p: Parameters) => {
4       implicit val q = p
5       implicit val v = implicitly[ValName]
6       LazyModule(new gemmini.Gemmini(gemmini.GemminiConfigs.
  defaultConfig.copy(
7         meshRows = mesh_size, meshColumns = mesh_size, dma_buswidth =
  bus_bits)))
8     }
9   )
10  case SystemBusKey => up(SystemBusKey).copy(beatBytes = bus_bits/8)
11 })

```

Listing 7.2: The `WithGemmini` configuration trait, which instantiates the accelerator as a RoCC device.

The practical consequence of applying this trait is that the generator assigns the Gemmini accelerator to a reserved block of custom instruction opcodes—specifically `OpcodeSet.custom3` (corresponds to the binary value `0b1111011`). As a result, when the Rocket Core’s instruction decoder encounters an instruction from this opcode space, it does not execute it. Instead, it dispatches the entire instruction, including register operands, directly to the Gemmini unit over the dedicated RoCC command channel that is automatically wired by the framework’s diplomacy system. This automated, configuration-driven process is what establishes the low-latency, co-designed link that is essential for the accelerator’s performance.

7.2.2 Hardware Configuration Parameters

The `WithGemmini` trait not only attaches the accelerator but also passes key parameters that define its generated hardware. For the final `Rocket64b1gem16jtag` configuration, the following parameters were used, resulting in the architecture shown in Figure 7.2:

- **Systolic Array Dimensions (`mesh_size`):** A value of 16 was used, generating a 16×16 grid of Processing Elements. This provides a total of 256 parallel MAC units.

- **DMA Bus Width (`bus_bits`):** A value of 64 was used, matching the data width of the Rocket Core and providing a 64-bit wide data path to the system bus.
- **On-Chip Memory:** The default configuration was used, which instantiates a 256KB, 4-banked scratchpad for staging data, and a separate 64KB, dual-ported accumulator memory for storing partial sums.

These parameters represent a specific design point, balancing high computational throughput with the resource constraints of the target FPGA.

7.3 Peripheral Integration

To create a fully functional, Linux-capable computing platform, the core SoC must be integrated with a rich set of I/O peripherals for storage, networking, and user interaction. This section details the architecture and integration of these essential components. Unlike the Chisel-generated processor core which uses the **TileLink** protocol, most peripherals are integrated as pre-existing, open-source Verilog IP blocks that use the industry-standard AXI4 protocol.

This creates a heterogeneous bus environment, which is managed by the Chipyard framework’s diplomacy-generated protocol bridges. As shown in the overall system diagram (Figure 7.1), peripherals are not attached to the main coherent **SystemBus**. Instead, they connect to a dedicated, AXI4-based **PeripheryBus**. A specialized **TileLink to AXI4 Bridge** is automatically instantiated by the framework to handle all communication between the CPU and this peripheral bus, managing protocol translation, address decoding, and data width conversion.

Each peripheral is managed by a dedicated controller IP that acts as a middleman, translating high-level AXI bus transactions from the Rocket Core into the specific low-level protocols required by the hardware (e.g., SD command sequences or Ethernet MAC framing). For the system software to utilize these components, a corresponding driver must be present in the operating system. This driver interacts with the controller’s memory-mapped registers to initiate data transfers and handle interrupts. The following subsections will briefly describe the implementation of the key peripherals integrated into this design: the SD Card controller, the UART controller, and the Ethernet subsystem, among others.

7.3.1 DDR Memory: High-Bandwidth Main Memory

The SoC's main memory system is built around a high-performance DDR3 SDRAM interface, which is essential for providing the capacity and bandwidth required to run a full Linux operating system and handle the large data sets of DNN workloads. The implementation does not use a custom controller but instead leverages the industry-standard **Xilinx Memory Interface Generator (MIG) 7-Series IP core**. This is a common and robust design practice that ensures reliable, high-frequency memory operation.

A key architectural feature of this design is its scalability. While the processor and system bus are designed with a **34-bit address width, capable of accessing up to 16 GiB of memory**, the current physical implementation on the VC707 board uses a 1 GiB SODIMM. The AXI SmartConnect bridge, instantiated by Chipyard, automatically handles the address translation between the large logical address space and the smaller physical memory map. This demonstrates a forward-looking design, allowing for future memory expansion with minimal changes to the core SoC architecture.

7.3.2 SD Card: AXI SD Card Controller and SD Mode

The SoC includes an AXI SD Card controller, which allows the system to boot from an SD card. This controller is integrated into the system using the Chipyard framework's diplomacy mechanism, which automatically wires up the necessary AXI interfaces.

The SD card is accessed in SD mode, which is a common mode for embedded systems. The controller supports standard SD commands and allows the system to read and write data blocks, making it suitable for booting the operating system and storing user data.

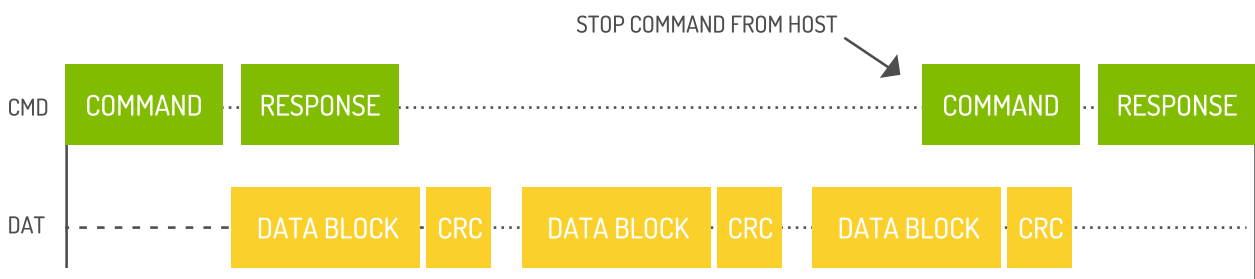


Figure 7.3: SD Mode: Multi-Block Read Operation. [68]

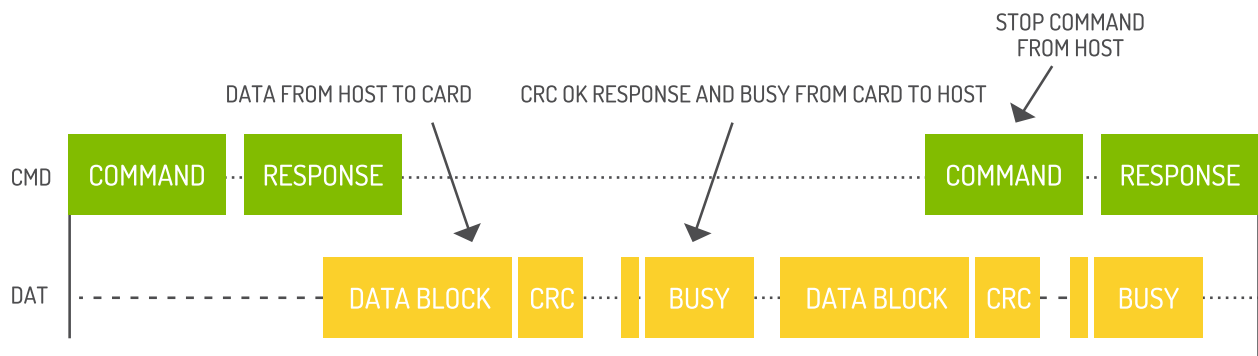


Figure 7.4: SD Mode: Multi-Block Write Operation. [68]

For comparison, here is how the same operation looks in SPI mode, which is another common mode for SD cards. The SPI mode is often used for compatibility with older systems or when a simpler interface is required. The SPI Mode offers lower speed compared to SD mode, but it is simpler to implement and can be used in systems that do not require high-speed data transfer.

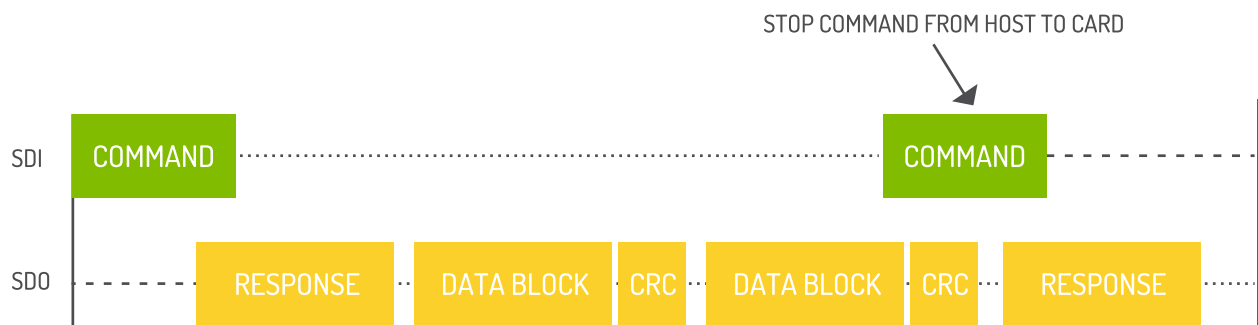


Figure 7.5: SPI Mode: Multi-Block Read Operation. [68]

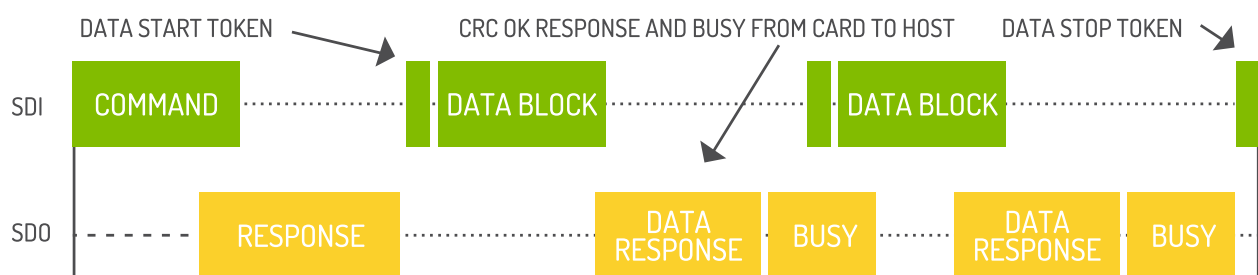


Figure 7.6: SPI Mode: Multi-Block Write Operation. [68]

7.3.3 UART: AXI UART Controller

To provide essential console I/O for debugging and user interaction with the operating system, the SoC integrates a robust UART controller. This component

is not a generic library IP but a custom-designed Verilog module tailored for this system, demonstrating the ability to integrate bespoke hardware into the Chipyard flow.

The controller is connected to the **PeripheryBus** via an AXI4-Lite interface, making it a memory-mapped peripheral accessible by the Rocket Core. Architecturally, it is designed for high-performance, reliable communication and includes several key hardware features:

- **FIFO Buffering:** The controller features independent 16-entry transmit (TX) and receive (RX) FIFOs. These hardware buffers are crucial for preventing data loss at high baud rates, as they decouple the CPU from the byte-by-byte management of the serial link by allowing it to read and write data in bursts.
- **Hardware Flow Control:** Full RTS/CTS hardware flow control is implemented, allowing the UART to automatically signal its readiness to receive data and to pause transmission if the remote device is not ready. This is essential for preventing buffer overruns in high-throughput scenarios.
- **Interrupt Generation:** The controller can generate interrupts to the CPU based on configurable events, such as the receive FIFO reaching a certain threshold. This enables efficient, interrupt-driven I/O handling in the Linux kernel, rather than relying on inefficient polling.

7.3.4 Ethernet: AXI Ethernet Controller

The Ethernet subsystem provides high-speed network connectivity for the RISC-V RocketChip SoC, enabling communication with external systems and networks. The implementation consists of a four-layer architecture that bridges the processor's AXI4 bus interface to the physical Ethernet connection on the VC707 FPGA board.

The Ethernet subsystem is implemented as a hierarchical design with four distinct functional blocks, as illustrated in Figure 7.1.

1. **Ethernet DMA Controller:** serves as the primary interface between the RocketChip SoC and the Ethernet MAC layer. It provides functionality for managing data transfers between the SoC and the Ethernet MAC, including packet buffering, flow control, and interrupt handling.
2. **SGMII 1Gbps Ethernet MAC:** The MAC (Media Access Control) layer is implemented using the open-source `eth_mac_1g_fifo` module from the verilog-

ethernet project [69]. This component provides: IEEE 802.3 Compliance, integrated frame-level FIFO Buffering, Frame Processing (Automatic frame padding, FCS generation and checking, and frame validation), AXI4-Stream Interface, Standard GMII Interface (Gigabit Media Independent Interface).

3. **SGMII PHY Interface:** implemented using Xilinx’s integrated SGMII IP core within the VC707 FPGA. This layer provides: SGMII Protocol (Serial Gigabit Media Independent Interface), Auto-negotiation, Clock Generation (125 MHz clock), Signal Conditioning.
4. **Physical RJ45 Interface:** The physical layer terminates at the RJ45 connector on the VC707 board.

7.3.5 System Monitoring: XADC and Fan Control

To ensure operational stability and manage thermal output, the SoC integrates a dedicated system monitoring and fan control subsystem. This demonstrates a hybrid design approach, combining vendor IP with custom control logic for a robust, real-world solution. The subsystem consists of two primary components:

- **Xilinx XADC IP Core:** The on-chip Xilinx Analog-to-Digital Converter (XADC) is used for real-time monitoring of the FPGA’s internal temperature and critical supply voltages (VCCINT, VCCAUX). This IP is configured to operate in a continuous sampling mode and is connected to the system via an AXI4-Lite interface.
- **Custom Fan Controller:** A custom-designed Verilog module provides PWM-based control for the chassis fan. Crucially, this module is directly connected to the XADC’s hardware thermal alarm pin. This creates an autonomous, hardware-based safety mechanism: if the die temperature exceeds a predefined threshold (e.g., 60°C), the alarm triggers an emergency override, forcing the fan to 100% duty cycle, independent of any software control.

7.4 Development Environment

The project requires a specific development environment to ensure that all tools, compilers, and dependencies are correctly configured. The primary environment is an Ubuntu 20.04 LTS workstation.

The complete project is included with all submodules for the RISC-V ecosystem components (Chipyard, U-Boot, Linux, OpenSBI) which are required for building the SoC and running the operating system.

7.5 Software Stack and Build Process

To run a complete operating system and user applications, a comprehensive software stack is required. This project utilizes a standard boot flow for embedded Linux systems.

7.5.1 Boot Flow: OpenSBI, U-Boot, and Linux

The system boots in several stages:

1. **Boot ROM:** On power-up, the Rocket Core begins execution from an on-chip Boot ROM. This initial code is responsible for basic hardware initialization and loading the next boot stage from an external device, in this case, an SD card.
2. **OpenSBI:** The first piece of software loaded from the SD card is the RISC-V Open Source Supervisor Binary Interface (OpenSBI) [70]. OpenSBI provides a crucial abstraction layer, handling low-level hardware control (like inter-processor interrupts and timers) so that the operating system kernel doesn't need to be machine-specific.
3. **U-Boot:** OpenSBI then loads and jumps to the Das U-Boot bootloader. U-Boot is a more powerful bootloader responsible for further hardware initialization (e.g., DDR memory) and loading the Linux kernel into RAM.
4. **Linux Kernel:** Finally, U-Boot passes control to the Linux kernel. The kernel boots, mounts the root filesystem from the SD card, and starts the user-space services, providing a full Debian environment.

7.5.2 SoC Hardware Generation

The first stage of the build process is to generate the Verilog HDL for the SoC.

1. **Initial Verilog and Device Tree Generation:** The build system invokes the Scala Build Tool (SBT) to compile the Chisel sources using the `Rocket64b1gem16jtag` configuration. This generates the initial Verilog HDL and a

base Device Tree Source (`.dts`) file, which describes the core SoC components as inferred by Chipyard’s diplomacy framework, as described in Chapter 6.

2. **Peripheral Integration:** This generated core is then integrated with several pre-existing, open-source peripheral controllers written in Verilog. These include the AXI UART controller, the AXI SD Card controller, and the AXI Ethernet controller.
3. **Device Tree Customization:** A crucial step is the creation of the final device tree. The base `system.dts` is combined with a board-specific overlay, `board/vc707/bootrom.dts`, which defines the VC707’s unique peripherals like the UART, MMC, and Ethernet controllers at their specific memory-mapped addresses.
4. **Boot ROM Generation:** The customized device tree is then passed to the boot ROM’s own build system, which compiles the C code for the first-stage bootloader and embeds the compiled device tree binary within the final ‘`bootrom.img`’ file.
5. **Final HDL Generation:** With the finalized ‘`bootrom.img`’ created, SBT is invoked a second time. This run uses the new boot ROM image to produce the final, correct Verilog HDL for the SoC, ensuring the processor’s memory map includes the fully configured boot ROM.

7.6 Gemmini Testing

We are going to use the `gemmini-rocc-tests` (<https://github.com/ucb-bar/gemmini-rocc-tests>) to test the Gemmini accelerator. The tests are designed to verify the functionality of the Gemmini RoCC interface and the correctness of the accelerator’s operations. The tests cover a wide range of scenarios, including basic arithmetic operations, matrix multiplication, convolution, and more complex DNN workloads.

With the hardware and software stack fully implemented and the testing methodology defined, the system is now ready for validation and evaluation. The following chapter presents the culminating results of this work, analyzing the hardware resource utilization of the final design, validating the full-stack software deployment, and, most critically, quantifying the performance impact of the Gemmini accelerator.

CHAPTER 8: FPGA IMPLEMENTATION RESULTS AND DISCUSSION

This chapter presents the culminating results of the thesis, transitioning from theoretical design to tangible implementation and empirical validation. The System-on-Chip (SoC), designed programmatically in Chisel, was compiled through the FIRRTL intermediate representation and subsequently implemented using the Xilinx Vivado toolchain for the VC707 FPGA target. This chapter is organized into three main sections. First, it details the hardware implementation results, including the final system architecture and an analysis of on-chip resource utilization and power consumption. Second, it documents the successful deployment of the full software stack, demonstrating a boot-to-Linux environment. Finally, and most critically, it evaluates the performance of the integrated Gemmini DNN accelerator, providing quantitative evidence of its effectiveness.

8.1 Hardware Implementation Analysis

The hardware generation process produces a set of Verilog files that are synthesized, placed, and routed by the Xilinx Vivado Design Suite. The following sections analyze the final hardware implementation from architectural, resource, and power perspectives.

8.1.1 System Block Design and Address Map

The generated system is a complex integration of the RISC-V core, the Gemmini accelerator, memory subsystems, and peripherals. The block diagrams in Figure 8.1, Figure 8.3, and Figure 8.2 provide a visual representation of the final hardware architecture as implemented in Vivado. These diagrams confirm the successful integration of all system components, from the top-level SoC (Figure 8.1) down to the specific connections for DDR memory (Figure 8.3) and I/O peripherals like UART and Ethernet (Figure 8.2).

The system's memory-mapped architecture is confirmed by the Address Editor view in Figure 8.4, which shows the memory ranges assigned to the DDR memory controller, the Gemmini DMA interface, and various peripheral buses. This confirms that the diplomacy framework within Chipyard correctly generated the necessary hardware and address decoding logic.

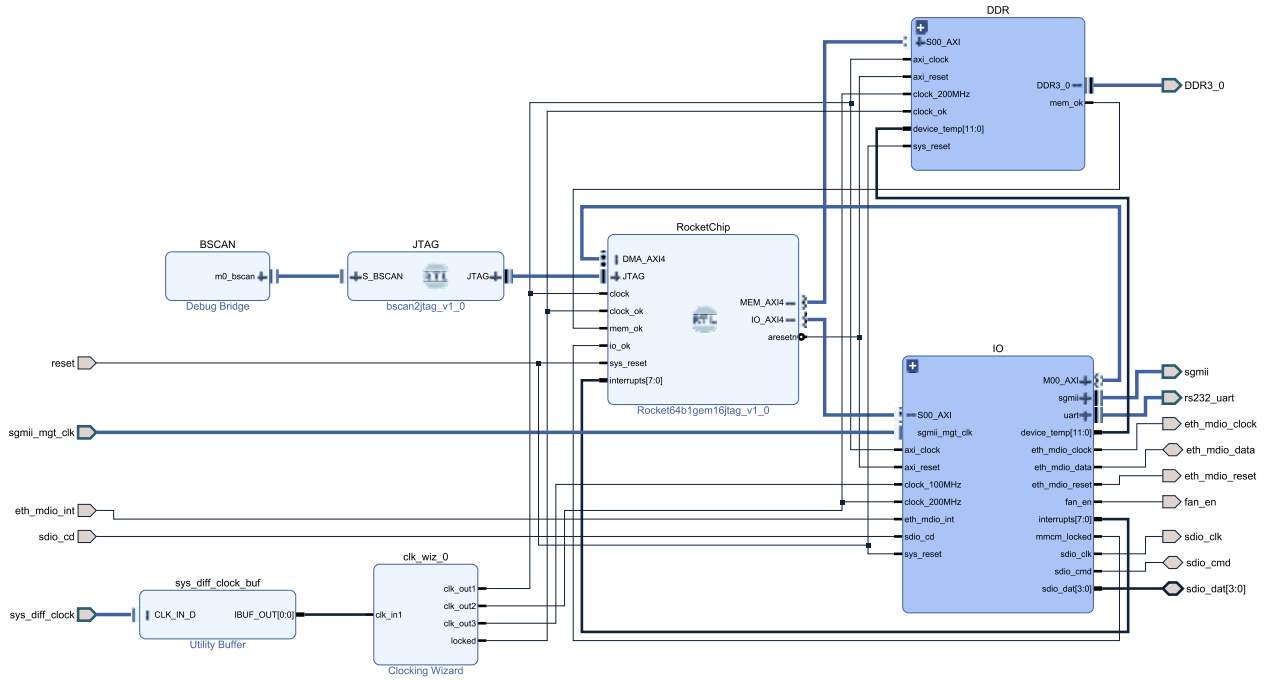


Figure 8.1: Block Design of the System, showing the top-level integration of the RocketChip core, DDR, and IO subsystems.

8.1.2 Vivado Implemented Design Results

8.1.2.1 Device Utilization Analysis

Resource utilization is a critical metric for evaluating the cost and feasibility of an SoC implementation on a given FPGA. The post-implementation utilization report from Vivado provides a detailed accounting of the hardware resources consumed by the design.

As shown in Figure 8.5, the complete, Linux-capable SoC is a substantial design, consuming **80.92%** of the available Look-Up Tables (LUTs), **28.59%** of Block RAM (BRAM), and **8.96%** of DSP slices on the Xilinx VC707's Virtex-7 FPGA. The high LUT utilization is expected given the complexity of a 64-bit processor core with an L2 cache and numerous peripherals. The significant BRAM usage is primarily attributable to the memory-intensive components of the design, namely the L2 cache and the Gemmini accelerator's scratchpad and accumulator memories.

To isolate the resource cost of the Gemmini accelerator itself, a hierarchical utilization analysis was performed. Figure 8.6 shows that the Gemmini module (`riscv_RocketChip_0_Gemmini`) consumes 180,142 LUTs. This represents approximately 73.5% of the logic within the system and underscores the fact that the specialized accelerator is, by far, the largest single component in the processor tile, dwarfing

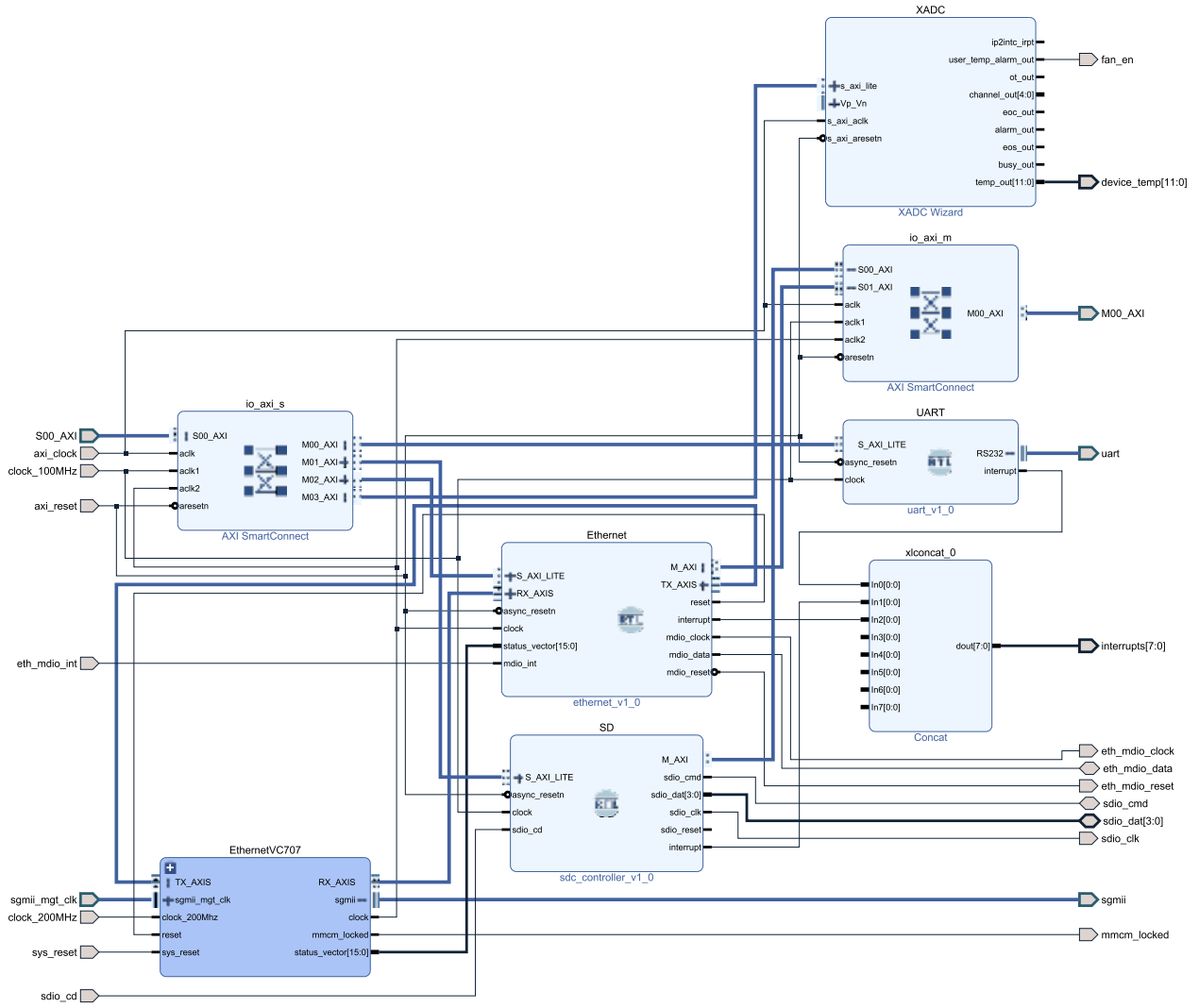


Figure 8.2: Block Design of the IO Subsystem, illustrating the AXI interconnect for peripherals such as Ethernet, SD Card, and UART.

the RISC-V core itself. This trade-off of silicon area for specialized performance is the central premise of hardware acceleration.

8.1.2.2 Power Consumption Analysis

Power analysis, performed by Vivado based on the placed-and-routed design and simulation-derived activity factors, is crucial for understanding the thermal and power-delivery requirements of the system.

The power summary in Figure 8.7 reports a **Total On-Chip Power of 3.856W**. A key observation is the breakdown between static and dynamic power. Dynamic power, consumed by switching logic, accounts for 3.309W (86% of the total), while static power is only 0.237W (6%). This is characteristic of a high-performance digi-

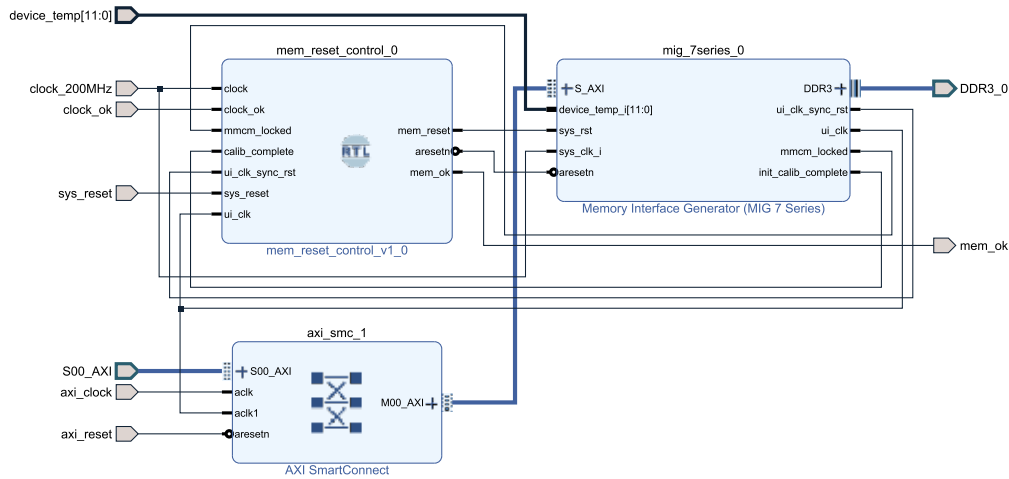


Figure 8.3: Block Design of the DDR Subsystem, detailing the Memory Interface Generator (MIG) and its AXI connection.

Diagram x Address Editor x Address Map x					
☒ Assigned (7) ☐ Unassigned (0) ☐ Excluded (0) Hide All					
Name	Interface	Slave Segment	Master Base Address	Range	Master High Address
Network 0					
/IO/Ethernet					
/IO/Ethernet/M_AXI (34 address bits : 16G)					
/RocketChip/DMA_AXI4	DMA_AXI4	reg0	0x0	16G	0x3_FFFF_FFFF
/IO/SD					
/IO/SD/M_AXI (34 address bits : 16G)					
/RocketChip/DMA_AXI4	DMA_AXI4	reg0	0x0	16G	0x3_FFFF_FFFF
Network 1					
/RocketChip					
/RocketChip/IO_AXI4 (31 address bits : 2G)					
/IO/Ethernet/S_AXI_LITE	S_AXI_LITE	reg0	0x6002_0000	64K	0x6002_FFFF
/IO/SD/S_AXI_LITE	S_AXI_LITE	reg0	0x6000_0000	64K	0x6000_FFFF
/IO/UART/S_AXI_LITE	S_AXI_LITE	reg0	0x6001_0000	64K	0x6001_FFFF
/IO/XADC/s_axi_lite	s_axi_lite	Reg	0x6003_0000	64K	0x6003_FFFF
Network 2					
/RocketChip					
/RocketChip/MEM_AXI4 (34 address bits : 16G)					
/DDR/mig_7series_0/memmap	S_AXI	memaddr	0x0	16G	0x3_FFFF_FFFF

Figure 8.4: Address Editor view, confirming the memory map for the integrated peripherals.

tal design where power consumption is dominated by computational activity rather than leakage. The report identifies that the most significant contributors to dynamic power are Clocks, Signals, and I/O, which is expected for a large, high-frequency SoC with multiple external interfaces. The power hierarchical report (Figure 8.8) further shows that the Gemini module contributes significantly to the overall power budget, consistent with its large area and high level of computational activity.

Summary

Resource	Utilization	Available	Utilization %
LUT	245673	303600	80.92
LUTRAM	9365	130800	7.16
FF	106058	607200	17.47
BRAM	294.50	1030	28.59
DSP	251	2800	8.96
IO	133	700	19.00
GT	1	28	3.57
MMCM	4	14	28.57
PLL	1	14	7.14

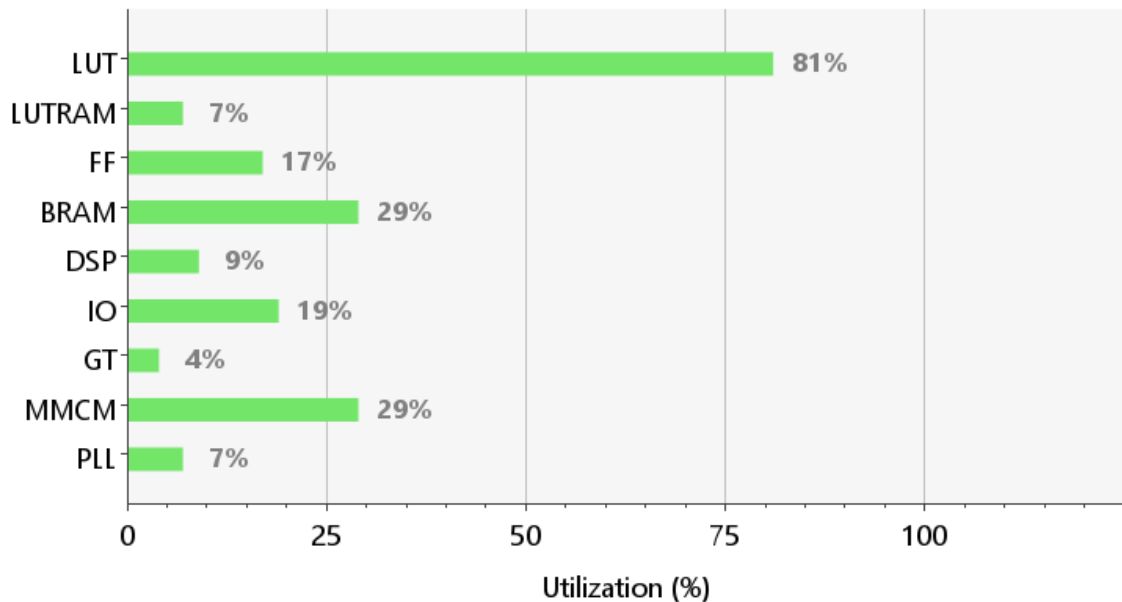


Figure 8.5: Detailed Utilization Summary for the entire SoC on the VC707 FPGA.

Name	Slice LUTs	Slice Registers (607200)	F7 Muxes (151800)	F8 Muxes (75900)	Slice (75900)	LUT as Logic (303600)	LUT as Memory (130800)	Block RAM Tile (1030)	DSPs (2800)
▼ riscv_wrapper	245673	106058	3692	791	71517	236308	9365	294.5	251
▼ riscv_i (riscv)	245673	106058	3692	791	71517	236308	9365	294.5	251
▼ RocketChip (riscv_RocketChip_0)	224082	83768	3299	777	64577	218896	5186	288	251
▼ inst (riscv_RocketChip_0_Rocket64b1gem16jtag)	224082	83750	3299	777	64549	218896	5186	288	251
▼ rocket_system (riscv_RocketChip_0_RocketSy	224075	83716	3299	777	64547	218889	5186	288	251
▼ tile_prci_domain (riscv_RocketChip_0_TileP	206701	77251	1867	222	60131	204046	2655	152	251
▼ tile_reset_domain_tile (riscv_RocketChip	206173	77229	1867	222	59988	203913	2260	152	251
> v (riscv_RocketChip_0_Gemmini)	180142	64040	678	110	52531	178136	2006	128	236

Figure 8.6: Hierarchical Utilization of the Gemmini accelerator module.

Summary

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power: 3.856 W
Design Power Budget: Not Specified
Process: typical
Power Budget Margin: N/A
Junction Temperature: 29.4°C
Thermal Margin: 55.6°C (47.0 W)
Ambient Temperature: 25.0 °C
Effective θ_{JA} : 1.1°C/W
Power supplied to off-chip devices: 0 W
Confidence level: Low
[Launch Power Constraint Advisor](#) to find and fix invalid switching activity

On-Chip Power

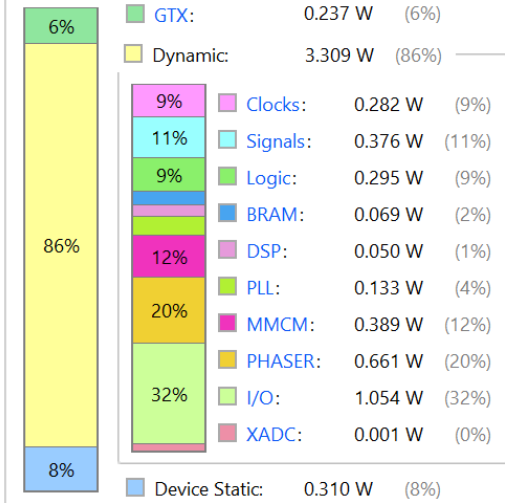


Figure 8.7: Post-implementation on-chip power summary.

Utilization	Name
3.546 W (92% of total)	riscv_wrapper
3.546 W (92% of total)	riscv_i (riscv)
2.228 W (58% of total)	DDR (DDR_imp_10J4PB3)
0.799 W (21% of total)	RocketChip (riscv_RocketChip_0)
0.798 W (21% of total)	inst (riscv_RocketChip_0_Rocket64b1gem16jtag)
0.798 W (21% of total)	rocket_system (riscv_RocketChip_0_RocketSyste
0.751 W (19% of total)	tile_prci_domain (riscv_RocketChip_0_TilePRCIDc
0.749 W (19% of total)	tile_reset_domain_tile (riscv_RocketChip_0_Rock
0.67 W (17% of total)	v (riscv_RocketChip_0_Gemmini)
0.036 W (1% of total)	fpuOpt (riscv_RocketChip_0_FPU)
0.015 W (1% of total)	frontend (riscv_RocketChip_0_Frontend)
0.014 W (1% of total)	core (riscv_RocketChip_0_Rocket)
0.01 W (<1% of total)	dcache (riscv_RocketChip_0_DCACHE)
0.003 W (<1% of total)	ptw (riscv_RocketChip_0_PTW)
<0.001 W (<1% of total)	cmdRouter (riscv_RocketChip_0_RoccCommand
<0.001 W (<1% of total)	respArb_io_in_0_q (riscv_RocketChip_0_Queue_1
<0.001 W (<1% of total)	tlMasterXbar (riscv_RocketChip_0_TLXbar_8)
<0.001 W (<1% of total)	dcacheArb (riscv_RocketChip_0_HellaCacheArbit
0.002 W (<1% of total)	buffer (riscv_RocketChip_0_TLBuffer_16)
<0.001 W (<1% of total)	intsink (riscv_RocketChip_0_IntSyncAsyncCrossi

Figure 8.8: Hierarchical power report for the Gemmini accelerator.

8.2 Software Stack Validation

A primary objective of this thesis was to implement a full-stack, Linux-capable system. This section provides evidence of this successful implementation by documenting the complete boot process, from the initial on-chip Boot ROM to a fully functional Debian Linux user environment.

8.2.1 System Boot Sequence

The system boot process involves a multi-stage software chain, starting with the first-stage bootloader in ROM, which then loads subsequent stages from the SD card. The console output in Listing 8.1 captures this entire sequence.

The process begins with the RISC-V 64, Boot ROM identifying the hardware and proceeding to load the next stage, OpenSBI, which is the `boot.elf` file on the SD card. OpenSBI (Open Source Supervisor Binary Interface) successfully initializes, reporting platform details such as the HART (Hardware Thread) count and timer frequency. It then loads and jumps to the U-Boot bootloader. U-Boot initializes the DRAM and other system hardware before loading the Linux kernel (`6.11.10-dirty (btdat@btdat-hpZ4)`) and the Flattened Device Tree (FDT) blob into memory. Finally, the message `Starting kernel ...` marks the successful handoff to the Linux operating system, which proceeds to boot, initialize its drivers, and present a login prompt.

```
RISC-V 64, Boot ROM V3.8

OpenSBI v1.6

      ----
    /  __ \      /  ----  ----  ----
  | | | | | _ _ _ _ _ _ _ _ | ( _ _ | | _ ) | | | | | |
  | | | | | ' _ \ / _ \ ' _ \ \ _ _ \ | _ < | |
  | | _ | | | _ ) | _ _ / | | | _ _ _ ) | | _ ) | | |
  \ _ _ _ / | . _ _ / \ _ _ | _ | | _ _ _ _ / | _ _ _ / _ _ _ |
      | |
      | _ |

Platform Name           : freechips,rocketchip-vivado
...
(Abridged OpenSBI and U-Boot output for brevity)
```

```
...
Boot HART ID : 0
Boot HART Domain : root
Boot HART Priv Version : v1.12
Boot HART Base ISA : rv64imafdcx
Boot HART ISA Extensions : sdtrig
Boot HART PMP Count : 8
Boot HART PMP Granularity : 2 bits
Boot HART PMP Address Bits : 32
Boot HART MHPM Info : 0 (0x00000000)
Boot HART Debug Triggers : 8 triggers
Boot HART MIDELEG : 0x00000000000000222
Boot HART MEDELEG : 0x0000000000000b109
```

```
U-Boot 2022.01-dirty (Jan 09 2025 - 16:37:27 +0700)
```

```
CPU: rv64imafdczicsr_zifencei_zihpm_xrocket
Model: freechips,rocketchip-vivado
DRAM: 1 GiB
MMC: mmc0@60000000: 0
Loading Environment from nowhere... OK
```

```
...
## Flattened Device Tree blob at 00010080
   Booting using the fdt blob at 0x010080
   Loading Device Tree to 0000000084ffc000, end
   0000000084fffe3f ... OK
```

```
Starting kernel ...
```

```
[ 0.000000] Linux version 6.11.10-dirty (btdat@btdat-hpZ4)
(riscv64-linux-gnu-gcc (Ubuntu 11.4.0-1ubuntu1~22.04)
11.4.0, GNU ld (GNU Binutils for Ubuntu) 2.38) #1 SMP Thu
Jan 9 16:51:21 +07 2025
[ 0.000000] OF: fdt: Ignoring memory range 0x80000000 -
0x80200000
[ 0.000000] Machine model: freechips,rocketchip-vivado
[ 0.000000] SBI specification v2.0 detected
```

Listing 8.1: Console output capturing the full boot sequence from Boot ROM to the Linux kernel.

8.2.2 Debian Linux Environment

After booting, the system provides a standard Debian GNU/Linux environment. Listing 8.2 confirms this by showing a successful login and the output of standard Linux commands. The `uname -a` command verifies the kernel version and the `riscv64` architecture. Critically, the output of `cat /proc/cpuinfo` confirms the processor's identity as `sifive,rocket0` and details its ISA as `rv64imafdc_...`, matching the design specification. This successful boot to a full-featured OS validates the entire hardware and software co-design process.

```
Debian trixie/sid debian ttyAU0

debian login: debian
Password:
Linux debian 6.11.10-dirty #1 SMP Thu Jan  9 16:51:21 +07 2025
    riscv64
...
debian@debian:~$ uname -a
Linux debian 6.11.10-dirty #1 SMP Thu Jan  9 16:51:21 +07 2025
    riscv64 GNU/Linux
debian@debian:~$ cat /proc/cpuinfo
processor       : 0
hart           : 0
isa            : rv64imafdc_zicntr_zicsr_zifencei_zihpm_zca_zcd
mmu            : sv39
uarch          : sifive,rocket0
mvendorid     : 0x0
marchid       : 0x1
mimpid        : 0x20181004
hart isa      : rv64imafdc_zicntr_zicsr_zifencei_zihpm_zca_zcd
```

Listing 8.2: Demonstration of a functional Debian Linux environment on the implemented SoC.

8.3 Gemmini Accelerator Performance Evaluation

With the SoC’s functional correctness and stability validated, the final and most critical evaluation is to quantify the performance benefits of the Gemmini DNN accelerator. The evaluation involved running a comprehensive suite of benchmarks from the `gemmini-rocc-tests` repository, comparing the cycle counts of software-only implementations on the RISC-V Rocket Core against the hardware-accelerated versions. The results provide a definitive measure of the speedup achieved across various workloads.

8.3.1 Functional Correctness

As a prerequisite for performance evaluation, the functional correctness of the accelerator was verified. All 25 test cases within the benchmark suite, spanning matrix multiplications, convolutions, and full network layers, passed successfully. A representative test, `tilled_matmul_ws`, which multiplies two 512×512 matrices, confirmed that the output matrix from the Gemmini accelerator was bit-perfect identical to the one produced by the pure software implementation. This confirms the correctness of the entire hardware/software co-design, from RoCC instruction dispatch to DMA transfers and the systolic array computation.

8.3.2 Performance Speedup and Analysis

The primary motivation for this work was to demonstrate the dramatic performance uplift provided by a specialized hardware accelerator for DNN workloads. The results, summarized in Table 8.1, unequivocally validate this thesis.

The performance gains are profound. While standard convolutions already show a remarkable $\sim 541\times$ average speedup, the most impressive results come from workloads optimized for the hardware’s dataflow. The `conv_trans_weight_0132` test, which leverages weight tensor transformations to maximize data reuse within the systolic array, achieved a staggering **1,308-fold speedup** over the software-only implementation.

$$\text{Speedup} = \frac{\text{CPU Time (ns)}}{\text{Gemmini Time (ns)}} = \frac{169,097,846,400}{129,206,400} \approx 1308.7\times \quad (8.1)$$

This result is a powerful demonstration of the principles discussed in Chapter 4. By architecting the hardware and software to work in concert, we can effectively

Table 8.1: Summary of Gemmini performance speedup across key operational categories. Detailed results for each test case are provided in Appendix D.

Operation Category	Execution Time Range		Speedup Range	Average Speedup
	CPU	Gemmini		
Matrix Multiplication	510ms - 4.3s	12ms - 39ms	$41\times$ - $353\times$	$\sim\mathbf{180\times}$
Standard Convolution	44.7s - 45.4s	81ms - 84ms	$532\times$ - $554\times$	$\sim\mathbf{541\times}$
Transformed Convolution	132s - 169s	129ms - 132ms	$1,007\times$ - $1,309\times$	$\sim\mathbf{1,110\times}$
Depthwise Convolution	1.5s	103ms	$15\times$	$\mathbf{15\times}$

overcome the Memory Wall bottleneck that cripples general-purpose CPUs on these workloads.

8.3.3 Real-World Workload Analysis: MobileNetV1

To evaluate performance on a realistic application, the full MobileNetV1 neural network was executed on the system. The accelerator completed the inference task in **3.81 seconds**. A breakdown of the execution time, shown in Table 8.2, reveals critical insights into the performance characteristics of modern CNNs.

Table 8.2: Cycle breakdown for a single inference pass of MobileNetV1 on the Gemmini accelerator.

MobileNetV1 Component	Execution Time (ns)	Percentage of Total
Depthwise Convolution	3,336,960,000	87.5%
Matrix Multiplication	310,441,600	8.1%
Standard Convolution	74,860,800	2.0%
Other (ResAdd, etc.)	92,643,200	2.4%
Total	3,814,905,600	100%

Notably, the relatively low-arithmetic-intensity **Depthwise Convolution** layers dominate the runtime, consuming 87.5% of the total execution time. This is despite having a lower speedup ($\sim 15\times$) compared to standard convolutions ($\sim 541\times$). This analysis highlights that optimizing for the most common layer types is critical for overall application performance.

8.3.4 Discussion of Results

The empirical data reveals several key characteristics of the implemented system:

- **The Critical Impact of Dataflow:** The highest speedups ($>1000\times$) were achieved in tests (`conv_trans_*`) where the data was explicitly arranged to match the hardware’s preferred weight-stationary dataflow. This confirms that performance is not just about raw compute power, but about the synergy between algorithm and architecture.
- **Computational Intensity Matters:** The comparatively low $15\times$ speedup on depthwise convolutions is expected. These layers have a lower ratio of computation to memory access, making them more memory-bound and limiting the benefits of the systolic array.
- **Hardware Utilization:** A performance analysis test (`tilled_matmul_ws_perf`) revealed that even with high speedup, the peak utilization of the systolic array was only **2%**. This suggests that the system is often limited by data movement (memory bandwidth) rather than by computation. This is a common challenge in accelerator design and presents a clear avenue for future research, such as implementing more advanced data prefetching mechanisms or exploring different scratchpad memory architectures.

These quantitative results serve as the ultimate validation of this thesis. They prove that a tightly-coupled, specialized hardware accelerator, implemented through a full-stack, open-source methodology, provides a dramatic and essential performance uplift for modern computational workloads.

CHAPTER 9: MODERN ALGORITHMS IN ASIC PHYSICAL DESIGN FLOW

The preceding chapters documented the design and FPGA-based validation of a complete RISC-V SoC, culminating in a synthesizable Verilog netlist. This chapter undertakes the subsequent and final step in the journey towards silicon: **physical design and physical verification**. This is the process of transforming that abstract, logical netlist into a concrete, geometric layout of transistors and wires that is ready for manufacturing.

This chapter will firstly cover the physical design and verification flow of VLSI circuits, focusing on the steps that are most relevant to ASIC design. It will then delve into the algorithms and techniques used in each step of the physical design flow, highlighting modern approaches and tools that have been developed to optimize this process.

Although it may be argued that open-source tools for ASIC physical design are not as mature as commercial tools, the open-source community has made significant strides in recent years. Tools like OpenROAD and LibreLane have emerged as powerful alternatives, providing a complete and automated flow for physical design. These tools are designed to be extensible and adaptable, allowing designers to customize the flow to meet their specific needs. That's why this chapter is dedicated to the algorithms and techniques used in ASIC physical design, with a focus on open-source tools that are becoming increasingly relevant in the industry.

9.1 ASIC Physical Design and Verification Workflow

This section provides a comprehensive overview of the ASIC physical design and verification workflow, detailing the transformation process from a logical netlist to a manufacturable layout. The workflow encompasses multiple interconnected stages, each with specific objectives and challenges that must be addressed to achieve a successful silicon implementation. Understanding this workflow is essential for grasping the complexity and precision required in modern ASIC design, where every decision impacts the final chip's performance, power consumption, area utilization, and manufacturing yield.

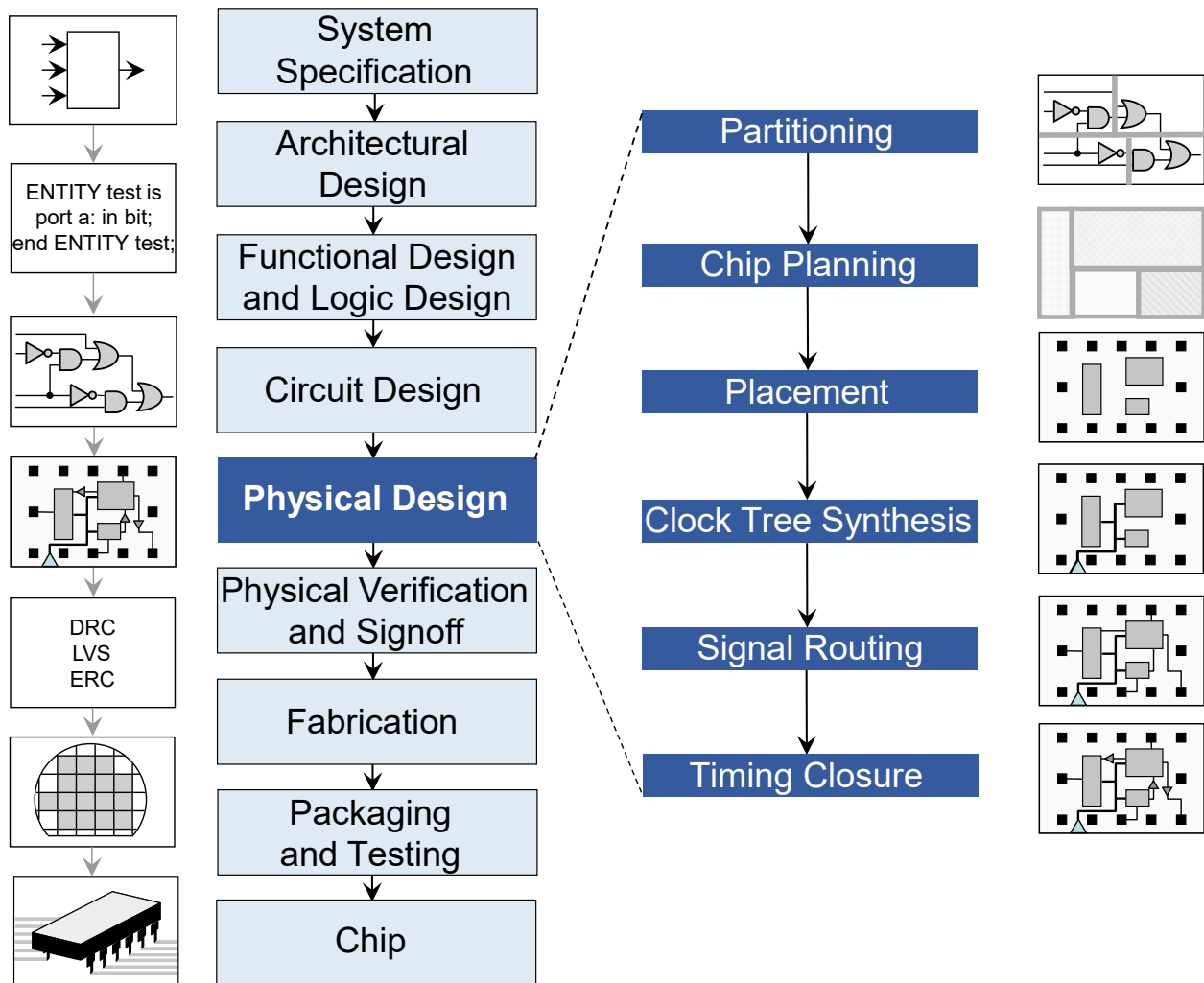


Figure 9.1: The major steps in the VLSI circuit design flow, with a focus on the physical design steps: partitioning, chip planning, placement, clock tree synthesis and routing, and finally timing closure. [71]

9.1.1 Overview of Physical Design

During physical design, all design components are instantiated with their geometric representations. In other words, all macros, cells, gates, transistors, etc., with fixed shapes and sizes per fabrication layer, are assigned spatial locations on the chip (placement) and have appropriate routing connections (routing) completed in metal layers on the upper surface of the chip. The result of physical design is a set of manufacturing specifications that must subsequently be verified.

Physical design is performed with respect to design rules that represent the physical limitations of the fabrication process for a particular manufacturing technology. For instance, all wires must be a prescribed minimum distance apart and have prescribed minimum width. As such, the design layout must be recreated in (migrated

to) each new manufacturing technology.

Physical design directly impacts circuit performance, area, reliability, power, and manufacturing yield. Examples of these impacts include the following:

- **Performance:** long routes have significantly longer signal delays.
- **Area:** placing connected modules far apart results in larger and slower chips.
- **Reliability:** large number of vias can significantly reduce the reliability of the circuit.
- **Power:** transistors with smaller gate lengths achieve greater switching speeds at the cost of higher leakage current and manufacturing variability; larger transistors and longer wires result in greater dynamic power dissipation.
- **Yield:** wires routed too close together may decrease yield due to electrical shorts occurring during manufacturing, but spreading gates too far apart may also undermine yield due to longer wires and a higher probability of opens.

Due to its high complexity, physical design is split into several key steps 9.1:

- **Partitioning (Chap. 2)** breaks up a circuit into smaller subcircuits or modules, which can each be designed or analyzed individually.
- **Floorplanning (Chap. 3)** determines the shapes and arrangement of subcircuits or modules, as well as the locations of external ports and IP or macro blocks.
- **Power and ground routing (Chap. 3)**, often intrinsic to floorplanning, distributes power (VDD) and ground (GND) nets throughout the chip.
- **Placement (Chap. 4)** finds the spatial locations of all cells within each block.
- **Clock network synthesis (Chap. 7)** determines the buffering, gating (e.g., for power management), and routing of the clock signal to meet prescribed skew and delay requirements.
- **Global routing (Chap. 5)** allocates routing resources that are used for connections; example resources include routing tracks in global cells (gcells).
- **Detailed routing (Chap. 6)** assigns routes to specific metal layers and routing tracks within the global routing resources.

- **Timing closure (Chap. 8)** optimizes circuit performance by specialized placement and routing techniques.

After detailed routing, electrically accurate layout optimization is performed at a small scale. Parasitic resistances (R), capacitances (C), and inductances (L) are extracted from the completed layout, and then passed to timing analysis tools to check the functional behavior of the chip. If the analyses reveal erroneous behavior or an insufficient design margin (guard band) against possible manufacturing and environmental variations, then incremental design optimizations are performed.

9.1.2 Physical Verification

After physical design is completed, the layout must be fully verified to ensure correct electrical and logical functionality. Some problems found during physical verification can be tolerated if their impact on chip yield will be negligible during manufacturing. In other cases, the layout must be changed, but these changes must be minimal, and care must be taken not to introduce new problems. Therefore, at this stage, layout changes are usually performed manually by experienced design engineers:

- **Design rule checking (DRC)** verifies that the layout meets all technology-imposed constraints. DRC also verifies layer density for uniform chemical-mechanical polishing (CMP).
- **Layout vs. schematic (LVS) checking** verifies the functionality of the design. To do this, the layout is used to derive (i.e., reverse engineer) a netlist, which is compared with the original netlist produced from logic synthesis or circuit design.
- **Parasitic extraction** derives electrical parameters of the layout elements from their geometric representations; the resulting accurate analog netlist is used to verify the electrical characteristics of the circuit.
- **Antenna rule checking** seeks to prevent antenna effects, which may damage transistor gates during manufacturing plasma-etch steps through the accumulation of excess charge on metal wires that are not connected to PN junction nodes.

- **Electrical rule checking (ERC)** verifies the correctness of power and ground connections, and that signal transition times (slew), capacitive loads, and fanouts are appropriately bounded.

Both synthesis and analysis techniques are integral to the design of VLSI circuits. Synthesis generally precedes analysis in the design flow 9.1. Analysis typically entails the modeling of circuit parameters and signal transitions, and often involves the solution of various equations using established numerical methods. The choice of algorithms for these tasks is relatively straightforward, compared to the vast possibilities for syntheses and optimization. Therefore, this book focuses on optimization algorithms used in IC physical design (which focus on synthesis), and does not cover computational techniques used during physical verification and signoff (where the emphasis is on analysis).

9.2 Open-Source Tools for ASIC Physical Workflow

9.2.1 OpenROAD

OpenROAD is an open-source project that aims to provide a complete, automated flow for the physical design of integrated circuits. It encompasses all stages of the physical design process, from floorplanning to detailed routing, and is designed to be extensible and adaptable to various design requirements. OpenROAD leverages advanced algorithms and techniques to optimize the physical layout of chips, ensuring high performance, low power consumption, and manufacturability.

The OpenROAD (“Foundations and Realization of Open, Accessible Design”) project was launched in June 2018 within the DARPA IDEA program. OpenROAD aims to bring down the barriers of cost, expertise and unpredictability that currently block designers’ access to hardware implementation in advanced technologies. The project team (Qualcomm, Arm and multiple universities and partners, led by UC San Diego) is developing a fully autonomous, open-source tool chain for digital SoC layout generation, focusing on the RTL-to-GDSII phase of system-on-chip design. Thus, OpenROAD holistically attacks the multiple facets of today’s design cost crisis: engineering resources, design tool licenses, project schedule, and risk.

9.2.2 LibreLane

LibreLane is made by The FOSSi Foundation (Free and Open Source Silicon). LibreLane is based off OpenLane2 by Efabless Corporation, which is also based off OpenLane, made by M. Shalan and T. Edwards [72].

LibreLane is a powerful and versatile infrastructure library that enables the construction of digital ASIC physical implementation flows based on open-source and commercial EDA tools. It includes a reference flow (Classic) that is constructed entirely using open-source EDA tools –abstracting their behavior and allowing the user to configure them using a single file (See Figure 1). LibreLane also supports extending or modifying flows using Python scripts and utilities. Here are some of the key benefits of using LibreLane:

- **Flexibility and extensibility:** LibreLane is designed to be flexible and extensible, allowing designers to customize the flow to meet their specific needs by developing Python scripts (plugins) and utilities or by modifying the existing configuration file.
- **Open source:** LibreLane is an open-source project that is freely available to use and modify, which makes it a good choice for designers looking for a transparent, cost-effective solution.
- **Community support:** LibreLane capitalizes on LibreLane’s existing community of users and contributors, which means that a wealth of resources is available to help designers get started and troubleshoot any problems they encounter.

9.3 Scale of Modern Design, and the need for Algorithms

The scale of modern integrated circuit (IC) design has grown exponentially, driven by the demand for higher performance, increased functionality, and lower power consumption. As a result, the design process has become more complex, requiring advanced algorithms and tools to manage the intricacies of modern ICs. The need for efficient algorithms is particularly evident in areas such as placement, routing, and optimization, where the design space is vast and the constraints are numerous.

In other words, the Electronics Design Automation (EDA) industry has been driven by the need to solve increasingly complex problems in IC design.

9.4 Netlist and System Partitioning

Modern integrated circuits (ICs) have become increasingly complex, while the level of integration has reached unprecedented levels. As a result, the design of ICs has become a highly complex and challenging task, requiring sophisticated algorithms and tools to manage the complexity and ensure that the final product meets the desired specifications. One of the key steps in this process is partitioning, which involves breaking down a large circuit into smaller, more manageable subcircuits or modules.

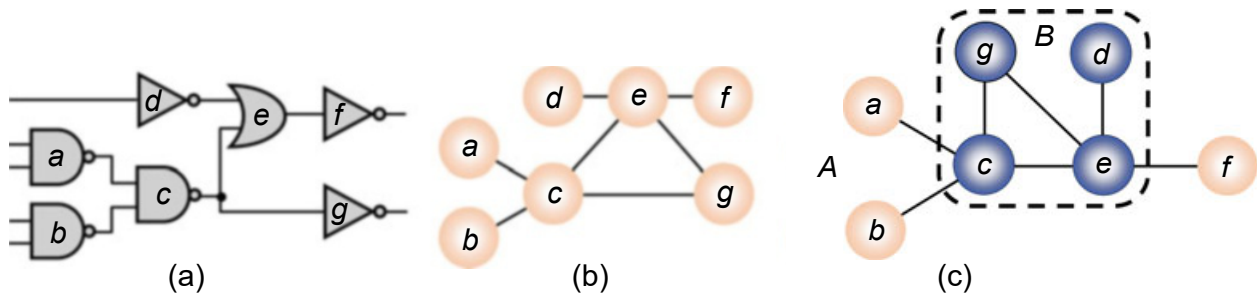


Figure 9.2: An example of a partitioned circuit, where the original circuit is divided into smaller subcircuits or modules. Each module can be designed or analyzed individually, allowing for more efficient design and verification processes.

The partitioner divides the circuit into several subcircuits (partitions or blocks) while minimizing the number of connections between partitions, subject to design constraints such as maximum partition sizes and maximum path delay.

If each block is implemented independently, i.e., without considering other partitions, then connections between these partitions may negatively affect the overall design performance such as increased circuit delay or decreased reliability. Moreover, a large number of connections between partitions may introduce interblock dependencies that hamper design productivity.¹ Therefore, the primary goal of partitioning is to divide the circuit such that the number of connections between subcircuits is minimized (Fig. 2.1). Each partition must also meet all design constraints. For example, the amount of logic in a partition can be limited by the size of an FPGA chip with which it is to be implemented. The number of external connections of a partition may also be limited, e.g., by the number of I/O pins in the chip package.

9.4.1 Partitioning Algorithms, and their Complexity

The partitioning algorithm that was proposed is a framework which combining clustering and Fiduccia-Mattheyses (FM) algorithm for (bi)partitioning (which is a heuristic algorithm for circuit partitioning that is optimized from the low-order polynomial time—the Kernighan-Lin (KL) algorithm).

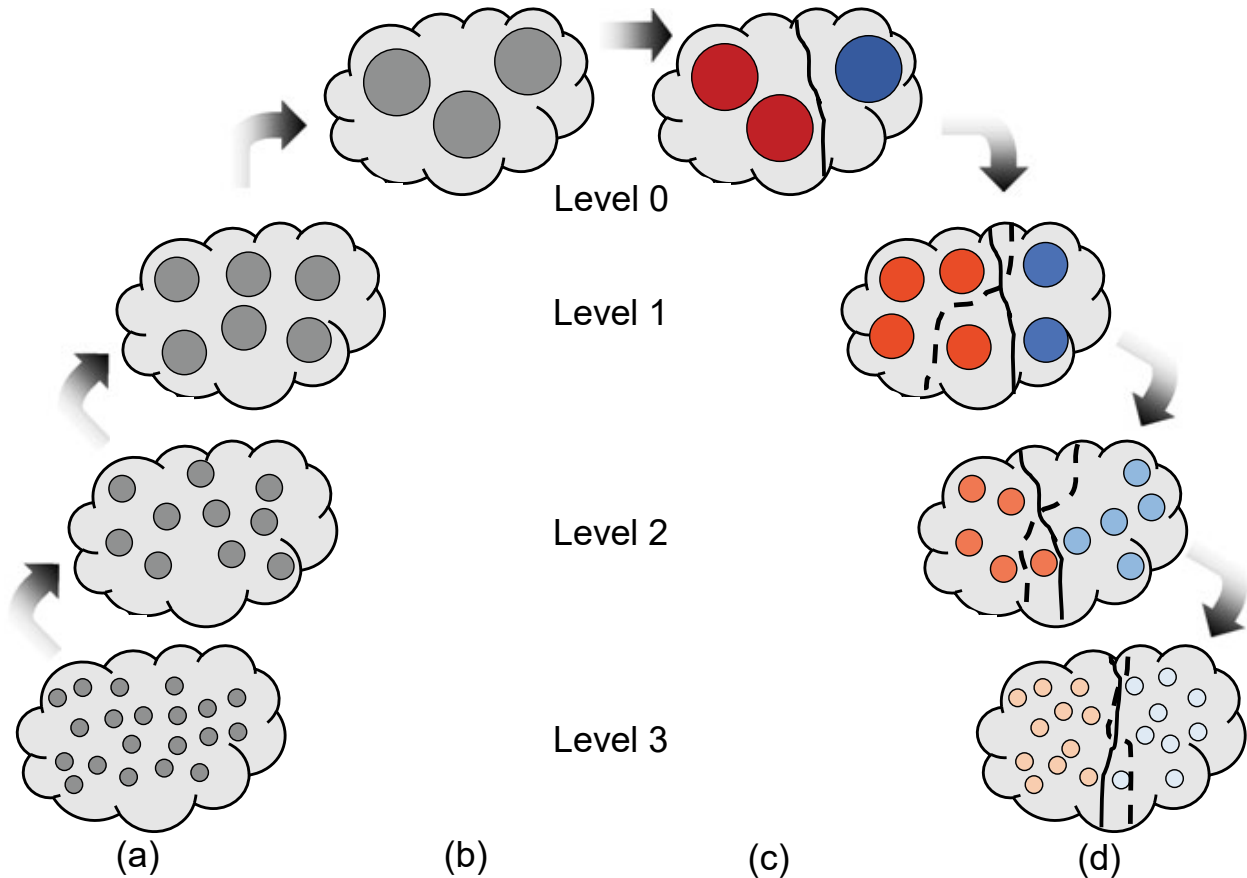


Figure 9.3: An illustration of multilevel partitioning. (a) The original graph is first coarsened through several levels. (b) The graph after coarsening. (c) After coarsening, a heuristic partition is found of the most coarsened graph. (d) That partition is then projected onto the next coarsest graph (dotted line) and then refined (solid line). Projection and refinement continue until a partitioning solution for the original graph is found.

Circuit partitioning, like many other combinatorial optimization problems in VLSI design, is NP-hard. This means that there is no known polynomial-time algorithm that can solve all instances of the problem optimally. However, many heuristic algorithms have been developed to find good approximate solutions in a reasonable amount of time. These algorithms often use techniques such as greedy methods, genetic algorithms, simulated annealing, and spectral methods to achieve effective partitioning.

Circuit partitioning, like many other combinatorial optimization problems discussed in this book, is NP-hard. That is, as the problem size grows linearly, the effort needed to find an optimal solution grows faster than any polynomial function. To date, there is no known polynomial-time, globally optimal algorithm for balance-constrained partitioning (Chap. 1, Sect. 1.6). However, several efficient heuristics were developed in the 1970s and 1980s. These algorithms find high-quality circuit partitioning solutions and in practice are implemented to run in low-order polynomial time—the Kernighan–Lin (KL) algorithm (Sect. 2.4.1), its extensions (Sect. 2.4.2), and the Fiduccia–Mattheyses (FM) algorithm (Sect. 2.4.3). Additionally, optimization by simulated annealing can be used to solve particularly difficult partitioning formulations. In general, stochastic hill-climbing algorithms require more than polynomial time to produce high-quality solutions, but can be accelerated by sacrificing solution quality. In practice, simulated annealing is rarely competitive.

9.4.2 OpenROAD Partitioning Approach: TritonPart

9.4.3 TritonPart: the first open-source, constraints-driven partitioning tool for VLSI physical design

OpenROAD have the partitioning module (par) based on TritonPart, an open-source constraints-driven partitioner. par can be used to partition a hypergraph or a gate-level netlist.

—

Abstract—With the increasing complexity of IC products, large-scale designs must be efficiently partitioned into multiple blocks, tiles, or devices for concurrent backend place-and-route (PR) implementation. State-of-the-art partitioners focus on balanced min-cut without considering constraints such as timing or heterogeneity of resource types. They are thus increasingly unsuitable for current physical design requirements. We introduce TritonPart, the first open-source, constraints-driven partitioning tool for VLSI physical design. TritonPart employs efficient algorithms to handle constraints, including multi-dimensional balance, embedding, and timing constraints. Our experimental work affirms its benefits. For standard min-cut partitioning, TritonPart outperforms hMETIS [17], with improvements of up to 20% on some benchmarks. For embedding-aware partitioning, TritonPart effectively leverages the embeddings generated by SpecPart [4] and improves upon it by 2%. For timing-aware partitioning, TritonPart significantly reduces the number of cuts on

timing-critical paths and prevents timing-noncritical paths from becoming critical (21X, 119X reduction relative to hMETIS and KaHyPar [31], respectively).

Index Terms—hypergraph partitioning, multi-dimensional weights, timing, embedding, VLSI constraints

Ismail Bustany; Grigor Gasparyan; et al.; An Open-Source Constraints-Driven General Partitioning Multi-Tool for VLSI Physical Design <https://ieeexplore.ieee.org/document/>
<https://github.com/ABKGroup/TritonPart> <https://openroad.readthedocs.io/en/latest/main/s>

9.5 Floorplanning, Pin Assignment and Power Routing: Establishing the Chip’s Foundation

9.5.1 Floorplanning

9.5.2 Pin Assignment

9.5.3 Power and Ground Routing

Power and ground routing (also known as Power Planning, Power Distribution Network (PDN) Design, or Power Grid Design) is a critical step in the physical design flow. It ensures that all components of the chip receive stable and sufficient power while minimizing voltage drops and noise. The power grid must be designed to handle the current demands of the circuit, which can vary significantly based on the activity of different components.

9.6 Global and Detailed Placement

9.7 Routing

9.7.1 Global Routing

9.7.2 Detailed Routing

9.8 Specialized Routing and Clock Tree Synthesis

9.9 Timing Closure

9.10 Physical Verification and Signoff

CHAPTER 10: VLSI PHYSICAL IMPLEMENTATION RESULTS

— This chapter is temporarily used as drafting space for the VLSI Physical Design Algorithms chapter. —

The preceding chapters documented the design, integration, and functional verification of a complete RISC-V System-on-Chip (SoC) featuring a Gemmini DNN accelerator, culminating in a successful prototype on an FPGA platform. This chapter undertakes the subsequent and final step in the journey towards silicon: **physical design**. This is the process of transforming the abstract, logical netlist synthesized from Verilog into a concrete, geometric layout of transistors and interconnects, a format known as GDSII, which is ready for fabrication.

The primary objective of this chapter is not merely to generate a final layout, but to conduct a systematic case study that bridges the gap between the foundational theory of VLSI physical design and its modern, open-source implementation. Each stage of the physical design flow presents a set of theoretically difficult, often NP-hard, optimization problems [71]. We will firstly cover the physical design and verification flow of VLSI circuits, focusing on the steps that are most relevant to ASIC design. Then, we will explore the theoretical foundation of each major stage—such as placement, clock tree synthesis, and routing—by drawing upon established academic literature. Subsequently, we will analyze how these challenges are addressed by the heuristic-based, highly-automated algorithms within the **OpenROAD** toolchain, as orchestrated by the **LibreLane** flow. Finally, for each stage, we will present the tangible results of applying this flow to the RISC-V SoC developed in this thesis, effectively documenting the journey from theory to practice.

Although it may be argued that open-source tools for ASIC physical design are not as mature as commercial tools, the open-source community has made significant strides in recent years. Tools like OpenROAD and LibreLane have emerged as powerful alternatives, providing a complete and automated flow for physical design. These tools are designed to be extensible and adaptable, allowing designers to customize the flow to meet their specific needs. That’s why this chapter is dedicated to the algorithms and techniques used in ASIC physical design, with a focus on open-source tools that are becoming increasingly relevant in the industry.

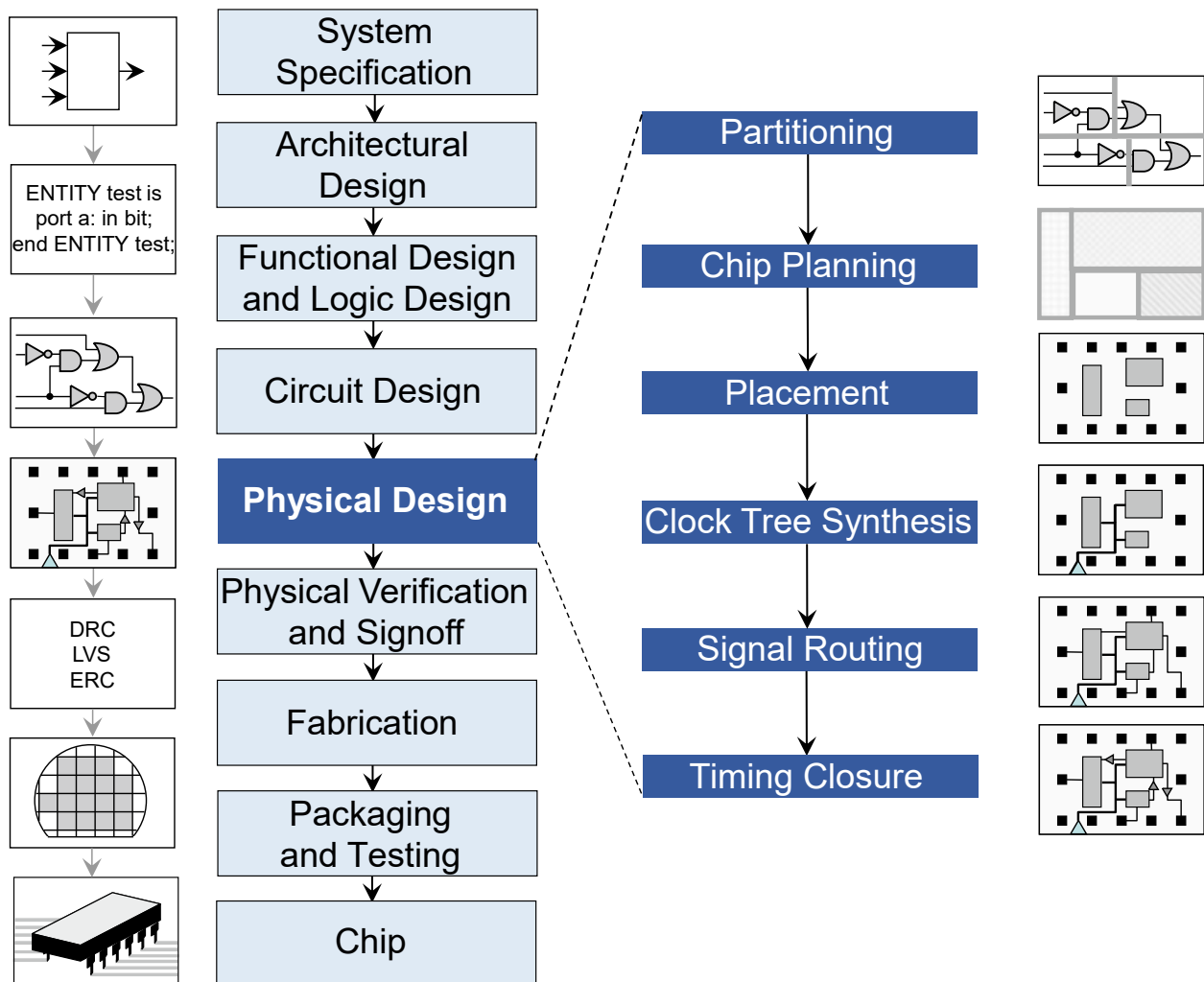


Figure 10.1: The major steps in the VLSI physical design flow, from logic synthesis to final sign-off. This chapter will follow this sequence to analyze the implementation of the RISC-V SoC. Adapted from [71].

10.1 The Open-Source Physical Design Methodology

The physical implementation of the SoC was performed using a fully open-source toolchain, demonstrating a complete "RTL-to-GDSII" flow without reliance on proprietary EDA software. This methodology is centered around two key projects:

The OpenROAD Project An autonomous, open-source tool suite that provides the core engines for every stage of physical design. Initiated by DARPA, the project's goal is to create a no-human-in-the-loop layout generation flow, drastically reducing the cost and expertise barriers to chip design [**OpenROAD-Project**]. The tool performs the heavy lifting of synthesis, placement, routing, and verification.

LibreLane An infrastructure library that orchestrates the sequence of tool invocations required for a complete physical design flow. Based on OpenLane 2, LibreLane provides a structured and configurable environment, using a single configuration file to manage the complex interactions between different EDA tools [**LibreLane-GitHub**]. For this thesis, LibreLane serves as the top-level manager, invoking the necessary OpenROAD tools to process the SoC design. The entire flow is executed using the open-source SkyWater 130nm (**sky130A**) Process Design Kit (PDK).

10.2 Logic Synthesis: From RTL to a Technology-Mapped Netlist

10.2.1 Theoretical Foundation

The first step in the physical design flow is logic synthesis. The goal is to transform a high-level Register-Transfer Level (RTL) description, written in a Hardware Description Language (HDL) such as Verilog, into a gate-level netlist. This process, known as technology mapping, involves converting abstract logic representations (e.g., additions, case statements) into a graph of specific standard cells available in a given Process Design Kit (PDK) [71]. The synthesis engine must perform this mapping while optimizing for the three primary design constraints: Power, Performance, and Area (PPA).

10.2.2 OpenROAD Implementation

Within the LibreLane flow, logic synthesis is performed by **Yosys**, an open-source synthesis suite. Yosys reads the Verilog source files for the SoC and the standard cell

library (‘.lib’ file) from the SkyWater PDK. It performs a series of optimization passes to produce a functionally equivalent, technology-mapped Verilog netlist that serves as the input for floorplanning and placement.

10.2.3 Application to the SoC

The RISC-V SoC, including the Rocket Core, Gemmini accelerator, and all peripherals, was synthesized using Yosys. The process produced a netlist composed entirely of cells from the ‘sky130_fd_sc_hd’ library.

Table 10.1: Post-Synthesis Cell Utilization for the RISC-V SoC.

Cell Type	Instance Count
Combinational Cells (AND, OR, XOR, etc.)	XX,XXX
Sequential Cells (Flip-Flops, Latches)	XX,XXX
Buffer Cells	X,XXX
Total Instance Count	XXX,XXX

10.3 Floorplanning and Power Delivery Network (PDN) Generation

10.3.1 Theoretical Foundation: Establishing the Chip’s Foundation

Floorplanning determines the overall shape and arrangement of the major architectural blocks on the chip, including macros and I/O pads. Its primary objectives are to minimize the total chip area and the estimated wirelength between blocks, which directly impacts performance and power consumption [71]. Concurrently, a robust Power Delivery Network (PDN) must be designed. This involves creating a grid of metal wires for power (VDD) and ground (GND) to ensure that every cell receives a stable voltage with minimal *IR drop*, even during peak computational activity [71].

10.3.2 OpenROAD Implementation

The LibreLane flow automates this stage using several OpenROAD tools. **io-Placer** first determines the positions of the I/O pads around the chip’s periphery. **tapcell** inserts special cells (tap cells, well taps) into the design to prevent latch-up issues. Finally, **pdnGen** generates the power grid based on a configuration file, creating the interlocking mesh of power and ground stripes on the upper metal layers.

For a "flat" design like this SoC without large pre-designed macros, the floorplanning step primarily involves defining the core area dimensions and target utilization.

10.3.3 Application to the SoC

The SoC was floorplanned with a target core utilization of 60% to leave adequate space for routing. A multi-layer power grid was generated to supply the core.

10.4 Placement: Assigning Locations to Standard Cells

10.4.1 Theoretical Foundation

10.4.2 OpenROAD Implementation

10.4.3 Application to the SoC

10.5 Clock Tree Synthesis and Routing

10.5.1 Theoretical Foundation

10.5.2 OpenROAD Implementation

10.5.3 Application to the SoC

10.6 Global and Detailed Routing

10.6.1 Theoretical Foundation

10.6.2 OpenROAD Implementation

10.6.3 Application to the SoC

10.7 GDSII Generation and Final Sign-off Verification

The final stage of the physical design flow is to generate the master layout file in the industry-standard GDSII format, which is the final deliverable for fabrication. Before this, however, a series of critical sign-off checks must be performed to ensure the design is both correct and manufacturable. This process, known as **physical verification**, includes:

- **Design Rule Checking (DRC):** Verifies that the layout adheres to all geometric constraints imposed by the manufacturing process, such as minimum

wire spacing and width [71]. The LibreLane flow uses the Magic tool for this purpose.

- **Layout vs. Schematic (LVS):** Compares the netlist extracted from the final layout against the original synthesized netlist to ensure they are functionally identical. This crucial check confirms that the physical design process did not inadvertently alter the circuit's logic [71]. The Netgen tool is used for LVS.

Upon successfully passing all DRC and LVS checks, the final GDSII file for the RISC-V SoC was generated. This file represents the culmination of the entire design process, containing the precise geometric shapes on every layer required to manufacture the chip.

10.8 Chapter Summary

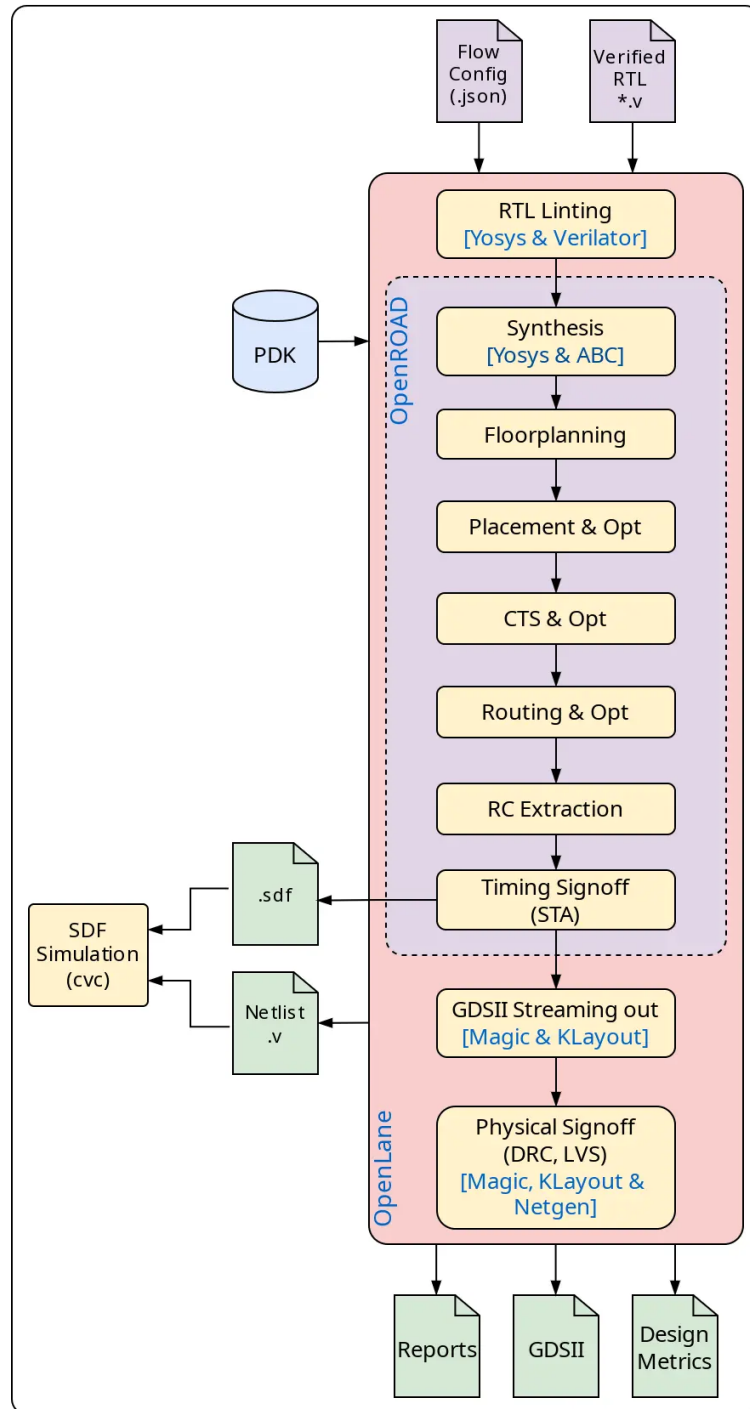


Figure 10.2: The LibreLane flow, showing the sequence of OpenROAD and various tools orchestrated to transform an RTL netlist into a final GDSII layout.



Figure 10.3: Floorplan of the RISC-V SoC showing the core area (center), I/O pads (periphery), and the power delivery network (grid). The target core utilization was 60%.



Figure 10.4: Final GDSII layout of the RISC-V SoC, generated using the open-source LibreLane and OpenROAD toolchain with the SkyWater 130nm PDK. The design passed all sign-off DRC and LVS checks.

CHAPTER 11: CONCLUSION AND FUTURE WORK

This thesis presented a study of advanced hardware acceleration within the open-source RISC-V ecosystem. The investigation centered on the implementation and microarchitectural analysis of the Gemmini DNN accelerator, a state-of-the-art systolic array generator, and its tightly-coupled integration into a Linux-capable System-on-Chip (SoC) via the Rocket Custom Coprocessor (RoCC) interface. The objective was to assess the capabilities of the modern open-source toolchain (Chipyard) for implementing and analyzing such a sophisticated, co-designed system, thereby providing a practical case study on the maturity of the RISC-V paradigm as an alternative to proprietary ISAs. The work culminated in a functional FPGA-based prototype and a quantitative performance analysis, yielding key insights into both the accelerator’s architecture and the effectiveness of the overall open-source workflow.

11.1 Summary of Work and Findings

This thesis fulfilled the objectives laid out in Chapter 1. The scope of the work and its primary findings are summarized as follows:

1. **Architectural Elucidation of Gemmini:** The Gemmini accelerator’s internal architecture was studied and documented in detail, providing one of the first comprehensive, public-facing analyses of its operational flow. Key findings from this study include the elucidation of its two-level command unrolling hierarchy, which translates high-level commands into tiled micro-operations, and its decoupled execution model, a mechanism critical for hiding memory latency.
2. **Analysis of the RoCC Co-Design Methodology:** The RoCC interface was analyzed and implemented as the integration mechanism. The study confirmed its effectiveness as a high-performance co-design methodology, providing a privileged, low-latency communication path that enables the tight coupling necessary for high-throughput accelerators.
3. **Full-Stack System Implementation and Verification:** The architectural study was grounded in a practical, full-stack system implementation. A custom SoC integrating the Rocket Core and Gemmini was successfully deployed on a Xilinx VC707 FPGA. The functional correctness of the entire open-source

toolchain and the hardware/software co-design was verified by the successful boot of a full Debian Linux operating system.

4. **Quantitative Performance Analysis:** A performance analysis was conducted to measure the impact of the tightly-coupled accelerator. The findings show that for workloads co-designed with the hardware's preferred dataflow, the Gemmini accelerator provided a speedup of over **1,300×** compared to a software-only implementation. This result empirically validates the performance potential of leveraging the RISC-V ISA's custom extension capabilities.

11.2 Limitations and Key Insights

The empirical results from Chapter 8 provided critical insights into the system's performance characteristics and limitations. The analysis of a real-world workload, MobileNetV1, revealed that overall performance is not dictated by peak speedup alone. The inference time was dominated by depthwise convolution layers, which, due to their lower computational intensity, achieved a more modest speedup of approximately 15×.

Furthermore, performance analysis revealed that even during high-speedup operations, the systolic array's peak utilization was only 2%. This strongly suggests that the system is primarily limited by memory bandwidth, not computational capacity. This "memory wall" is a common challenge in accelerator design and highlights that simply adding more compute units will not yield better performance without corresponding improvements in the memory subsystem.

11.3 Future Work

The successful implementation and the insights gained from this study open several clear and promising avenues for future research. This work establishes a baseline system upon which the following investigations can be built, categorized into direct architectural enhancements, broader platform development, and long-term strategic goals.

11.3.0.1 Architectural Enhancements to the Gemmini Accelerator

The empirical results highlighted specific performance bottlenecks that can be addressed with targeted architectural improvements.

- **Addressing the Observed Memory Bottlenecks:** Based on the key finding of low systolic array utilization, future work should prioritize improving Gemini’s memory subsystem. This could involve studying the implementation of more sophisticated DMA prefetching mechanisms within the accelerator’s Load/Store controllers or exploring novel scratchpad memory architectures.
- **Architectural Exploration for Common Workloads:** Given that low-intensity layers like depthwise convolutions dominate many modern neural networks, a key research direction is to explore architectural modifications that specifically target these layers. This could involve designing a more flexible compute fabric or a secondary, smaller accelerator optimized for non-GEMM operations.

11.3.0.2 Platform and Ecosystem Development

The validated platform itself can be extended and improved to enable a wider range of future research.

- **Extending the Co-Design with New Accelerators:** The RoCC interface was confirmed to be a highly effective integration methodology. Future research could leverage this platform to design and integrate new, custom RoCC-based accelerators for different workloads (e.g., cryptography, signal processing), further exploring the hardware/software co-design space enabled by RISC-V.
- **Platform Optimization for FPGA-Based Research:** A valuable strategic direction would be to create an FPGA-optimized version of this Chipyard/Gemini platform. This would involve tuning the hardware generators to better map to FPGA primitives, with the goal of reducing resource utilization and improving clock frequency, thereby enabling more complex accelerator research on available hardware.

11.3.0.3 Long-Term Strategic Goal

Finally, the platform enables the pursuit of the ultimate goal of custom silicon design.

- **Transition to ASIC Implementation:** This project has validated the Chipyard framework’s entire FPGA flow. The logical next step is to leverage Chipyard’s integrated VLSI flow (Hammer) to transition this design from an FPGA

prototype to a full ASIC implementation, a critical step toward producing custom silicon.

In conclusion, the detailed study of the Gemmini accelerator and its RoCC integration has shown that the open-source RISC-V ecosystem is a mature and powerful environment for conducting research into state-of-the-art computer architecture and specialized hardware co-design. The findings of this thesis, particularly the elucidation of Gemmini’s internal mechanisms and the quantification of its performance, serve as a strong validation of the open-source paradigm. The resulting system stands as a robust and well-documented foundation for future academic research.

LIST OF PUBLICATIONS

- [P1] Nhu-Hoang Nguyen, Tan-Phat Dang, Tuan-Kiet Tran, Thanh-Dat Bui, Trong-Thuc Hoang, and Huu-Thuan Huynh. “A Configurable 2D-Integer DCT Hardware Accelerator Compatible with H.266 Standard based on RISC-V Architecture”. In: *2024 7th International Seminar on Research of Information Technology and Intelligent Systems (ISRITI)*. 2024, pp. 894–899. DOI: 10.1109/ISRITI64779.2024.10963564.
- [P2] Nhu-Hoang Nguyen, Tan-Phat Dang, Thanh-Dat Bui, Trong-Thuc Hoang, Cong-Kha Pham, and Huu-Thuan Huynh. “Designing and Implementing a 2D Integer DCT Hardware Accelerator Fully Compatible with Versatile Video Coding”. In: *Computational Science and Its Applications – ICCSA 2024 Workshops*. Ed. by Osvaldo Gervasi, Beniamino Murgante, Chiara Garau, David Taniar, Ana Maria A. C. Rocha, and Maria Noelia Faginas Lago. Cham: Springer Nature Switzerland, 2024, pp. 110–121. ISBN: 978-3-031-65154-0. DOI: 10.1007/978-3-031-65154-0_7.
- [P3] Thi-Minh-Tuyen Huynh, Tuan-Kiet Tran, Thanh-Dat Bui, Cong-Kha Pham, and Huu-Thuan Huynh. “An Efficient Algorithm Compatible with Low-performance Hardware for AI Edge Devices in Arrhythmia Prediction”. In: *5th International Conference on Intelligent Systems & Networks (ICISN 2025)*. 2025.
- [P4] Duc-Huy Pham, Thi-Minh-Tuyen Huynh, Tuan-Kiet Tran, Thanh-Dat Bui, Cong-Kha Pham, and Huu-Thuan Huynh. “Efficient AI Model and Hardware Architecture Based on CNN for Arrhythmia Prediction”. In: *2025 10th IEEE International Conference on Integrated Circuits, Design, and Verification (ICDV 2025)*. 2025.

BIBLIOGRAPHY

- [1] Krste Asanović and David A. Patterson. *Instruction Sets Should Be Free: The Case For RISC-V*. Tech. rep. UCB/EECS-2014-146. Aug. 2014. URL: <http://www2.eecs.berkeley.edu/Pubs/TechRpts/2014/EECS-2014-146.html>.
- [2] Davide Pala. “Design and programming of a coprocessor for a RISC-V architecture: Guidelines for embedding computing cores as RISC-V coprocessors”. Supervisor: Prof. Massimo Poncino. Master’s Thesis. Torino, Italy: Politecnico di Torino, 2017. URL: <http://webthesis.biblio.polito.it/id/eprint/6589>.
- [3] Intel Corporation. *Licensing Cores*. Intel Documentation. Accessed: Jul. 26, 2025. 2018. URL: <https://www.intel.com/content/www/us/en/docs/programmable/683502/17-1/licensing-cores.html>.
- [4] Xilinx. *MicroBlaze FAQ*. PDF Document from EETrend. Accessed: Jul. 26, 2025. 2011. URL: https://fpga.eetrend.com/files-eetrend-xilinx/forum/201103/1735-3159-microblaze_faq.pdf.
- [5] Eugene Tarassov. *vivado-risc-v: FPGA based RISC-V Systems Workloads including Linux Debian on an SD Card for Xilinx FPGAs with Vivado and Vitis*. Accessed: November 28, 2023. URL: <https://github.com/eugene-tarassov/vivado-risc-v>.
- [6] Gaisler. *Silicon IP Licensing*. Gaisler Website. Accessed: Jul. 26, 2025. 2025. URL: <https://www.gaisler.com/silicon-ip-licensing>.
- [7] Clifford Wolf. *PicoRV32 - A Size-Optimized RISC-V CPU*. GitHub Repository. Accessed on: July 26, 2025. Available at: <https://github.com/YosysHQ/picorv32>. 2024.
- [8] Charles Papon. *VexRiscv: A FPGA friendly 32 bit RISC-V CPU implementation*. GitHub Repository. Accessed on: July 26, 2025. Available at: <https://github.com/SpinalHDL/VexRiscv>. 2024.
- [9] LiteX-Hub. *Linux on LiteX-VexRiscv*. GitHub Repository. Accessed on: July 26, 2025. Available at: <https://github.com/litex-hub/linux-on-litex-vexriscv>. 2024.

- [10] RIOT OS Forum Community. *RIOT on a VexRiscV and SpinalHDL FPGA Softcore*. RIOT OS Forum. Accessed: Jul. 26, 2025. 2025. URL: <https://forum.riot-os.org/t/riot-on-a-vexriscv-and-spinalhdl-fpga-softcore/3602>.
- [11] lowRISC. *Ibex: A 32-bit RISC-V CPU core*. GitHub Repository. Accessed on: July 26, 2025. Available at: <https://github.com/lowRISC/ibex>. 2024.
- [12] lowRISC. *Ibex Core Documentation: Pipeline Details*. Online Documentation. Accessed on: July 26, 2025. Available at: https://ibex-core.readthedocs.io/en/latest/03_reference/pipeline_details.html. 2024.
- [13] The OpenTitan Project. *OpenTitan: Open source silicon root of trust (RoT)*. Website. Accessed: July 10, 2025. 2025. URL: <https://opentitan.org/>.
- [14] OpenHW Group. *OpenHW Group CORE-V CV32E40P RISC-V IP*. GitHub Repository. Accessed on: July 26, 2025. Available at: <https://github.com/openhwgroup/cv32e40p>. 2024.
- [15] Chips Alliance. *Rocket Chip Generator*. GitHub Repository. Accessed on: July 26, 2025. Available at: <https://github.com/chipsalliance/rocket-chip>. 2024.
- [16] The Berkeley Architecture Research Group. *The Berkeley Out-of-Order RISC-V Processor*. GitHub Repository. Accessed on: July 26, 2025. Available at: <https://github.com/riscv-boom/riscv-boom>. 2024.
- [17] Christopher Celio et al. "BROOM: An Open-Source Out-of-Order Processor With Resilient Low-Voltage Operation in 28-nm CMOS". In: *IEEE Micro* 39.2 (2019), pp. 52–60. DOI: 10.1109/MM.2019.2897782.
- [18] OpenRISC. *or1200*. GitHub. Accessed: Jul. 26, 2025. 2015. URL: <https://github.com/openrisc/or1200>.
- [19] OpenRISC. *OpenRISC Official Website*. Website. Accessed: Jul. 26, 2025. 2025. URL: <https://openrisc.io/>.
- [20] Jeremy Bennett and Simon Cook. "Softcores for FPGA: The Free and Open Source Alternatives". In: *NMI Meeting on "Embedded Processors - you've got the power, but which to choose"*. Published by Embecosm. Presented by Simon Cook. National Microelectronics Institute (NMI). Bristol, UK, Oct. 2013.
- [21] Gaisler. *LEON3 SPARC V8 Processor*. Product Page. Accessed on: July 26, 2025. Available at: <https://www.gaisler.com/products/leon3>. 2024.

- [22] Gaisler. *LEON5 Processor*. Product Page. Accessed: Jul. 26, 2025. Nov. 2019. URL: <https://www.gaisler.com/products/leon5>.
- [23] Gaisler. *NOEL-V Processor*. Product Page. Accessed: Jul. 26, 2025. Jan. 2020. URL: <https://www.gaisler.com/products/noel-v>.
- [24] The SHD Group. *RISC-V Market Analysis 2024: Abridged Report*. Jan. 2024. URL: <https://theshdgroup.com/wp-content/uploads/2024/01/RISC-V-Market-Analysis-2024-Abridged-Report-2.pdf>.
- [25] Rui Jia et al. “A survey of open source processors for FPGAs”. In: *2014 24th International Conference on Field Programmable Logic and Applications (FPL)*. 2014, pp. 1–6. DOI: 10.1109/FPL.2014.6927482.
- [26] Institute of Electronics, Chinese Academy of Sciences. *A Survey of Open Source Processors for FPGAs*. PDF Document from Semantic Scholar. Presentation. Accessed: Jul. 26, 2025. 2025. URL: <https://pdfs.semanticscholar.org/9da2/ef5a0d52d1164f62d1cb4dbd447430bb90f0.pdf>.
- [27] Marcel Baunach. *Soft-Core Processors for Embedded Systems*. Presentation, ISEC Graz. Accessed on: July 26, 2025. Available at: <https://www.isec.tugraz.at/wp-content/uploads/2020/07/softcores.pdf>. 2020.
- [28] Jason G. Tong, Ian D. L. Anderson, and Mohammed A. S. Khalid. “Soft-Core Processors for Embedded Systems”. In: *2006 International Conference on Microelectronics*. 2006, pp. 170–173. DOI: 10.1109/ICM.2006.373294.
- [29] Sagar Karandikar et al. *Full-day Tutorial on FireSim and Chipyard*. Presented at the 50th Annual International Symposium on Computer Architecture (ISCA). Tutorial slides available online. Orlando, FL, USA, June 2023. URL: <https://fires.im/isca-2023-tutorial/>.
- [30] NLnet Foundation. *LiteX: Developer framework for FPGA and ASIC designs. Funded by the NGIO Entrust Fund, a fund established by NLnet with financial support from the European Commission’s Next Generation Internet programme, under the aegis of DG Communications Networks, Content and Technology under grant agreement No 101069594*. 2024. URL: <https://nlnet.nl/project/LiteX/>.
- [31] The PULP Platform Team. *PULP Implementation*. Website. Accessed: July 10, 2025. 2025. URL: <https://pulp-platform.org/implementation.html>.

- [32] OpenTitan Project. *EarlGrey Top-Level Specification*. OpenTitan Documentation. Accessed: Jul. 26, 2025. 2025. URL: https://opentitan.org/book/hw/top_earlgrey/doc/specification.html.
- [33] lowRISC. *lowRISC announces new OpenTitan project partner, ZeroRISC*. lowRISC News. Accessed: Jul. 26, 2025. Oct. 2023. URL: <https://lowrisc.org/news/lowrisc-announces-new-opentitan-project-partner-zerorisc/>.
- [34] Enjoy-Digital. *LiteX: Build your hardware, easily!* GitHub Repository. Accessed: July 10, 2025. 2025. URL: <https://github.com/enjoy-digital/litex>.
- [35] Florent Kermarrec et al. *LiteX: an open-source SoC builder and library based on Migen Python DSL*. 2020. arXiv: 2005.02506 [cs.AR]. URL: <https://arxiv.org/abs/2005.02506>.
- [36] LiteX-Hub. *LiteX-Boards: LiteX boards files*. GitHub Repository. Accessed: July 10, 2025. 2025. URL: <https://github.com/litex-hub/litex-boards>.
- [37] The PULP Platform Team. *PULP Platform*. Website. Accessed: July 10, 2025. 2025. URL: <https://pulp-platform.org/>.
- [38] lowRISC. *OpenTitan: Open source silicon root of trust*. GitHub Repository. Accessed: July 10, 2025. 2025. URL: <https://github.com/lowRISC/opentitan>.
- [39] Krste Asanović et al. *The Rocket Chip Generator*. Tech. rep. UCB/EECS-2016-17. Apr. 2016. URL: <http://www2.eecs.berkeley.edu/Pubs/TechRpts/2016/EECS-2016-17.html>.
- [40] C. Celio, D. Patterson, and K. Asanovic. *The Berkeley Out-of-Order Machine (BOOM) Design Specification*. University of California, Berkeley. 2016. URL: <https://github.com/ccelio/riscv-boom-doc/raw/gh-pages/boom-spec.pdf>.
- [41] Christopher Celio. “A Highly Productive Implementation of an Out-of-Order Processor Generator”. PhD thesis. EECS Department, University of California, Berkeley, Dec. 2018. URL: <http://www2.eecs.berkeley.edu/Pubs/TechRpts/2018/EECS-2018-151.html>.
- [42] Jerry Zhao et al. “SonicBOOM: The 3rd Generation Berkeley Out-of-Order Machine”. In: (May 2020).

- [43] Andrew Waterman and Krste Asanović, eds. *The RISC-V Instruction Set Manual, Volume I: User-Level ISA*. Document Version 20240411. RISC-V Foundation. Apr. 2024. URL: <https://github.com/riscv/riscv-isa-manual/releases/download/20240411/unpriv-isa-asciidoc.pdf>.
- [44] Andrew Waterman et al. *The RISC-V Instruction Set Manual Volume II: Privileged Architecture Version 1.7*. Tech. rep. UCB/EECS-2015-49. May 2015. URL: <http://www2.eecs.berkeley.edu/Pubs/TechRpts/2015/EECS-2015-49.html>.
- [45] RISC-V International. *Members*. Accessed: Jun. 16, 2025. 2025. URL: <https://riscv.org/members/>.
- [46] Ben Keller. *CS250 Laboratory 2: RISC-V, Spike, and the Rocket Core*. Course Handout, CS250: VLSI Systems Design, University of California, Berkeley. Version 091713. Accessed via the Internet Archive. Sept. 2013. URL: <https://web.archive.org/web/20191126182925/http://www-inst.eecs.berkeley.edu/~cs250/fa13/handouts/lab2-riscv.pdf>.
- [47] Ben Keller. *Handouts for CS250: VLSI Systems Design, University of California, Berkeley*. Fall 2013. Accessed via the Internet Archive. Sept. 2013. URL: <https://web.archive.org/web/20200201115643/http://www-inst.eecs.berkeley.edu/~cs250/fa13/handouts/index.html>.
- [48] Christopher Celio, David A. Patterson, and Krste Asanović. *The Berkeley Out-of-Order Machine (BOOM): An Industry-Competitive, Synthesizable, Parameterized RISC-V Processor*. Tech. rep. UCB/EECS-2015-167. June 2015. URL: <http://www2.eecs.berkeley.edu/Pubs/TechRpts/2015/EECS-2015-167.html>.
- [49] J. Zhao. *Chipyard Intro and Fundamentals*. Presentation Slides, ISCA 2021 Tutorial. Accessed: Jun. 6, 2025. 2021. URL: https://fires.im/isca21-slides-pdf/02_chipyard_basics.pdf.
- [50] Sagar Karandikar et al. “FireSim: FPGA-Accelerated Cycle-Exact Scale-Out System Simulation in the Public Cloud”. In: *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. 2018, pp. 29–42. DOI: 10.1109/ISCA.2018.00014.
- [51] SiFive. *A Winning Processor Portfolio*. Accessed: Dec. 30, 2024. 2024. URL: <https://www.sifive.com/risc-v-core-ip>.

- [52] Alon Amid et al. “Chipyard: Integrated Design, Simulation, and Implementation Framework for Custom SoCs”. In: *IEEE Micro* 40.4 (2020), pp. 10–21. ISSN: 1937-4143. DOI: 10.1109/MM.2020.2996616.
- [53] Jonathan Bachrach et al. “Chisel: constructing hardware in a Scala embedded language”. In: *Proceedings of the 49th Annual Design Automation Conference. DAC '12*. San Francisco, California: Association for Computing Machinery, 2012, pp. 1216–1225. ISBN: 9781450311991. DOI: 10.1145/2228360.2228584. URL: <https://doi.org/10.1145/2228360.2228584>.
- [54] Binh Kieu-do-Nguyen et al. “Hardware Software Co-Design for Multi-Threaded Computation on RISC-V-Based Multicore System”. In: *IEEE Access* 12 (2024), pp. 177312–177326. DOI: 10.1109/ACCESS.2024.3505940.
- [55] SiFive Inc. *SiFive TileLink Specification*. Version 1.7.1. Accessed: Jun. 6, 2025. Dec. 2018. URL: https://sifive.cdn.prismic.io/sifive%2F57f93ecf-2c42-46f7-9818-bcdd7d39400a_tilelink-spec-1.7.1.pdf.
- [56] Wm. A. Wulf and Sally A. McKee. “Hitting the memory wall: implications of the obvious”. In: *SIGARCH Comput. Archit. News* 23.1 (Mar. 1995), pp. 20–24. ISSN: 0163-5964. DOI: 10.1145/216585.216588. URL: <https://doi.org/10.1145/216585.216588>.
- [57] Mark Horowitz. “1.1 Computing’s energy problem (and what we can do about it)”. In: *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*. 2014, pp. 10–14. DOI: 10.1109/ISSCC.2014.6757323.
- [58] Vivienne Sze et al. *Efficient processing of deep neural networks*. Jan. 2020. DOI: 10.1007/978-3-031-01766-7. URL: <https://doi.org/10.1007/978-3-031-01766-7>.
- [59] Kung. “Why systolic architectures?” In: *Computer* 15.1 (1982), pp. 37–46. DOI: 10.1109/MC.1982.1653825.
- [60] Norman P. Jouppi et al. “In-Datacenter Performance Analysis of a Tensor Processing Unit”. In: *SIGARCH Comput. Archit. News* 45.2 (June 2017), pp. 1–12. ISSN: 0163-5964. DOI: 10.1145/3140659.3080246. URL: <https://doi.org/10.1145/3140659.3080246>.

- [61] Sparsh Mittal and Sumanth Umesh. “A survey On hardware accelerators and optimization techniques for RNNs”. In: *Journal of Systems Architecture* 112 (2021), p. 101839. ISSN: 1383-7621. DOI: <https://doi.org/10.1016/j.sysarc.2020.101839>. URL: <https://www.sciencedirect.com/science/article/pii/S1383762120301314>.
- [62] Farzad Farshchi, Qijing Huang, and Heechul Yun. “Integrating NVIDIA Deep Learning Accelerator (NVDLA) with RISC-V SoC on FireSim”. In: *2019 2nd Workshop on Energy Efficient Machine Learning and Cognitive Computing for Embedded Applications (EMC2)*. 2019, pp. 21–25. DOI: 10.1109/EMC249363.2019.00012.
- [63] NVIDIA. *NVIDIA Xavier*. Hot Chips 30. Accessed: Jun. 9, 2025. 2018. URL: https://old.hotchips.org/hc30/1conf/1.12_Nvidia_XavierHotchips2018Final_814.pdf.
- [64] Hasan Genc et al. “Gemmini: Enabling Systematic Deep-Learning Architecture Evaluation via Full-Stack Integration”. In: *Proceedings of the 58th Annual ACM/IEEE Design Automation Conference*. IEEE Press, 2022, pp. 769–774. ISBN: 9781665432740. URL: <https://doi.org/10.1109/DAC18074.2021.9586216>.
- [65] Haifeng Zhang et al. “A Heterogeneous RISC-V Processor for Efficient DNN Application in Smart Sensing System”. In: *Sensors* 21.19 (2021). ISSN: 1424-8220. DOI: 10.3390/s21196491. URL: <https://www.mdpi.com/1424-8220/21/19/6491>.
- [66] Dennis Agyemanh Nana Gookyi et al. “Deep Learning Accelerators’ Configuration Space Exploration Effect on Performance and Resource Utilization: A Gemmini Case Study”. In: *Sensors* 23.5 (2023). ISSN: 1424-8220. DOI: 10.3390/s23052380. URL: <https://www.mdpi.com/1424-8220/23/5/2380>.
- [67] Hasan Genc et al. *Gemmini @ MLSys 2022*. Presentation Slides, MLSys 2022 Tutorial. Accessed: [Jul. 26, 2025]. 2022. URL: <https://mlsys.org/virtual/2022/tutorial/18433>.
- [68] Cactus Technologies. *An Introduction To SD Card Interface*. White Paper CTWP009. Suite C, 15/F, Capital Trade Center, 62 Tsun Yip Street, Kwun Tong, Kowloon, Hong Kong: Cactus Technologies Limited. URL: <https://www.cactus-tech.com/wp-content/uploads/2019/03/An-Introduction-To-SD-Card-Interface.pdf>.

- [69] Alex Forencich. *Verilog Ethernet Components*. GitHub Repository. Accessed: July 2025. 2023. URL: <https://github.com/alexforencich/verilog-ethernet>.
- [70] RISC-V International OpenSBI Project Contributors. *OpenSBI: RISC-V Open Source Supervisor Binary Interface*. GitHub repository. Accessed: November 28, 2023. 2023. URL: <https://github.com/riscv-software-src/opensbi>.
- [71] Andrew B. Kahng et al. *VLSI Physical Design: From Graph Partitioning to Timing Closure*. 2nd ed. Springer Cham, June 2022. DOI: 10.1007/978-3-030-96415-3. URL: <https://doi.org/10.1007/978-3-030-96415-3>.
- [72] Mohamed Shalan and Tim Edwards. “Building OpenLANE: A 130nm OpenROAD-based Tapeout- Proven Flow : Invited Paper”. In: *2020 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. 2020, pp. 1–6.

APPENDIX A: DETAILED PROCESSING UNITS

A.1 ExecuteController Sub-Modules

The **ExecuteController** is a complex module that contains several specialized hardware units to manage the core computation and data transformation pipeline. The primary sub-modules include:

- **Im2Col Module:** A specialized SRAM controller for convolution operations that converts image data into column format for efficient convolution. It manages sliding window operations over input data, handles address generation for accessing input feature maps, and provides specialized data access patterns for CNN operations.
- **Queue (sram_read_signals_q):** A pipeline synchronization buffer that buffers control signals for SRAM read operations, provides timing alignment between address generation and data availability, and handles pipeline delays in the Im2Col data path.
- **Queue (mesh_ctrl_signals_q):** A mesh control signal buffer that queues control signals fed into the computational mesh. It synchronizes computation control with data arrival timing and provides pipeline buffering for mesh operations.
- **MultiHeadedQueue:** A multi-port queue for command buffering that allows multiple commands to be read simultaneously. This provides flexible command scheduling and issue capabilities, and is essential for supporting out-of-order execution patterns.
- **TransposePreloadUnroller:** A command transformation unit that unrolls complex operations into simpler primitive operations, handles matrix transpose operations for the weight-stationary dataflow, and manages preload command sequences for optimal data placement.
- **MeshWithDelays (Computational Mesh):** The main systolic array for matrix computations that performs the actual matrix multiplication operations. It implements the systolic array dataflow (input/weight/output stationary), handles multiple simultaneous matrix multiplications, and manages timing and data flow through the PE array.

A.2 Scratchpad Integrated Processing Units

The Gemmini Scratchpad contains several hardware units that process data on-the-fly as it is moved to or from memory, offloading these tasks from the host CPU. The primary units include:

- **VectorScalarMultiplier** (`vsm/vsm_1`): Applies scaling operations to data during movement, with separate scalers for scratchpad and accumulator data paths. This is essential for handling quantized data types.
- **PixelRepeater**: Handles pixel repetition optimizations for first-layer convolution operations, reducing memory bandwidth for specific layer types.
- **AccumulatorScale unit** (`acc_scale_unit`): A critical component that applies scaling and activation functions (e.g., ReLU) to accumulator data before it is written back to memory.
- **Normalizer unit** (`acc_norm_unit`): Performs normalization operations on accumulator data.
- **ZeroWriter**: Generates zero data when the `all_zeros` flag is set, avoiding unnecessary memory reads and saving power.

APPENDIX B: ROCC INTERFACE AND GEMMINI INTEGRATION DETAILS

This appendix provides a detailed technical reference for the Rocket Custom Co-processor (RoCC) interface and its specific implementation for the Gemmini accelerator. The Chisel/Scala code listings below are extracted from the `rocket-chip` and `gemmini` generator source files and serve as the ground-truth definition of the hardware interface and its integration with the CPU pipeline.

B.1 SoC Configuration and Accelerator Instantiation

The integration begins at the SoC configuration level. The Gemmini accelerator is instantiated as a `LazyRoCC` module and is assigned a specific opcode space for its custom instructions.

```
1 // From generators/gemmini/src/main/scala/gemmini/Controller.scala
2 class Gemini[T <: Data : Arithmetic, U <: Data, V <: Data](...)
3   extends LazyRoCC (
4     opcodes = OpcodeSet.custom3, // Assigns Gemmini to the custom3 space
5     nPTWPorts = if (config.use_shared_tlb) 1 else 2
6   ) {
7     override lazy val module = new GeminiModule(this)
8     // ... TileLink node connections
9 }
```

Listing B.1: Gemmini’s extension of the `LazyRoCC` base class, which registers it as a RoCC accelerator within the Chipyard framework.

```
1 // From rocket-chip/src/main/scala/tile/LazyRoCC.scala
2 object OpcodeSet {
3   def custom0 = new OpcodeSet(Seq("b0001011".U))
4   def custom1 = new OpcodeSet(Seq("b0101011".U))
5   def custom2 = new OpcodeSet(Seq("b1011011".U))
6   def custom3 = new OpcodeSet(Seq("b1111011".U)) // Used by Gemmini
7 }
```

Listing B.2: Definition of the RISC-V custom opcode spaces available in Rocket Chip.

B.2 RoCC Hardware Interface Bundles

The following Chisel Bundle classes define the physical wires that constitute the complete RoCC interface between the CPU and the accelerator.

```
1 // Source: rocket-chip/src/main/scala/tile/LazyRoCC.scala
2 class RoCCIO(val nPTWPorts: Int)(implicit p: Parameters)
3   extends Bundle {
4   // Command channel from CPU to Accelerator
5   val cmd = Flipped(Decoupled(new RoCCCommand))
6
7   // Response channel from Accelerator to CPU
8   val resp = Decoupled(new RoCCResponse)
9
10  // Direct memory interface to the L1 Data Cache
11  val mem = new HellaCacheIO
12
13  // Page Table Walker interface for virtual memory
14  val ptw = Vec(nPTWPorts, new TLBPTWIO)
15
16  // Status and Control Signals
17  val busy = Output(Bool())
18  val interrupt = Output(Bool())
19  val exception = Input(Bool()) // Signals an exception from the CPU
20 }
```

Listing B.3: The top-level hardware bundle for the complete RoCC interface.

```
1 // Source: rocket-chip/src/main/scala/tile/LazyRoCC.scala
2 class RoCCCommand(implicit p: Parameters) extends CoreBundle()(p) {
3   val inst = new RoCCInstruction // The decoded instruction
4   val rs1 = Bits(xLen.W) // Data from source register 1
5   val rs2 = Bits(xLen.W) // Data from source register 2
6   val status = new MStatus // Current processor status flags
7 }
```

Listing B.4: The RoCCCommand bundle, defining the payload dispatched from the CPU to the accelerator.

```
1 // Source: rocket-chip/src/main/scala/tile/LazyRoCC.scala
2 class RoCCInstruction extends Bundle {
3   val funct = Bits(7.W) // Function code (specifies Gemmini operation)
4   val rs2 = Bits(5.W) // Source register 2 address
5   val rs1 = Bits(5.W) // Source register 1 address
6 }
```



```

6  val xd = Bool()           // Destination register valid bit
7  val xs1 = Bool()          // Source register 1 valid bit
8  val xs2 = Bool()          // Source register 2 valid bit
9  val rd = Bits(5.W)        // Destination register address
10 val opcode = Bits(7.W)    // Opcode (custom3 for Gemmini)
11 }

```

Listing B.5: The RoCCInstruction bundle, defining the raw instruction format.

B.3 Gemmini Custom Instruction Set

The `funct7` field of a RoCC instruction is passed to the accelerator to specify an operation. Listing B.6 shows the definition of the primary commands used by Gemmini. These are high-level directives that can trigger complex sequences within the accelerator.

```

1 // Source: generators/gemmini/src/main/scala/gemmini/GemminiISA.scala
2 object GemminiISA {
3   // Basic Operations
4   val CONFIG_CMD = 0.U
5   val LOAD_CMD = 2.U
6   val STORE_CMD = 3.U
7   val COMPUTE_AND_STAY_CMD = 5.U
8   val PRELOAD_CMD = 6.U
9   val FLUSH_CMD = 7.U
10
11  // Complex, CISC-like loop commands
12  val LOOP_WS = 8.U
13  val LOOP_CONV_WS = 15.U
14
15  // Configuration sub-commands
16  val CONFIG_EX = 0.U
17  val CONFIG_LOAD = 1.U
18  val CONFIG_STORE = 2.U
19 }

```

Listing B.6: The `GemminiISA` object, defining the function codes for Gemmini’s instructions.

B.4 CPU-Side Pipeline Integration Details

The following code snippets from the `rocket-chip` source demonstrate the deep integration of the RoCC interface within the Rocket Core’s five-stage pipeline.

B.4.1 Command Processing Flow

The CPU pipeline identifies, routes, and dispatches RoCC commands.

```
1 // From rocket-chip/src/main/scala/rocket/IDecode.scala
2 // The decoder table maps the CUSTOM3 opcode to a control signal profile
3 // that uses the register file but not the main ALU.
4 CUSTOM3 -> List(Y,N,Y,N,N,N,N,N,A2_ZERO,A1_RS1,IMM_X,DW_XPR,...) ,
```

Listing B.7: Instruction Decode stage logic for recognizing RoCC opcodes.

```
1 // From rocket-chip/src/main/scala/tile/RocketTile.scala
2 // The Rocket Tile routes the command from the core’s RoCC port
3 // to the command router, which directs it to the correct accelerator.
4 if (outer.roccs.size > 0) {
5   cmdRouter.get.io.in <- core.io.rocc.cmd
6   core.io.rocc.resp <- respArb.get.io.out
7   // ... busy and interrupt signal aggregation
8 }
```

Listing B.8: Command routing from the CPU core to the RoCC accelerator.

B.4.2 Pipeline Stall and Hazard Management

The CPU’s hazard unit is modified to stall the pipeline based on the accelerator’s status, providing physical evidence of the hardware co-design.

```
1 // From rocket-chip/src/main/scala/rocket/RocketCore.scala
2
3 // Stall the Decode stage if RoCC is busy or not ready
4 val rocc_blocked = Reg(Bool())
5 rocc_blocked := !wb_xcpt && !io.rocc.cmd.ready &&
6               (io.rocc.cmd.valid || rocc_blocked)
7 val ctrl_stalld = id_ctrl.rocc && rocc_blocked || ...
8
9 // Replay the instruction in the Writeback stage if the
10 // accelerator is not ready to accept the command.
11 val replay_wb_rocc = wb_reg_valid && wb_ctrl.rocc &&
```

```
12      !io.rocc.cmd.ready
```

Listing B.9: RoCC-specific stall and replay logic from the CPU’s hazard unit.

B.4.3 Response and Register File Write

The RoCC interface has a privileged path to write results directly into the CPU’s register file, bypassing the main memory hierarchy.

```
1 // From rocket-chip/src/main/scala/rocket/RocketCore.scala
2 io.rocc.resp.ready := !wb_wxd // CPU is ready for a response if not
   writing a GPR
3
4 when (io.rocc.resp.fire) {
5   // RoCC response takes priority over other units like the divider
6   div.io.resp.ready := false.B
7
8   // Route RoCC data directly to the register file write port
9   ll_wdata := io.rocc.resp.bits.data
10  ll_waddr := io.rocc.resp.bits.rd
11  ll_wen := true.B
12 }
```

Listing B.10: Response arbitration in the Writeback stage.

B.4.4 Status and Interrupt Handling

The CPU and RoCC accelerator coordinate on status and interrupts.

```
1 // From rocket-chip/src/main/scala/rocket/RocketCore.scala
2 io.rocc.cmd.bits.status := csr.io.status // Pass current CPU status to
   RoCC
3 io.rocc.exception := wb_xcpt // Signal exceptions to RoCC
4
5 // From rocket-chip/src/main/scala/rocket/CSR.scala
6 // RoCC interrupt is wired into the Machine Interrupt Pending (MIP)
   register
7 mip.rocc := io.rocc_interrupt
```

Listing B.11: Status and interrupt coordination logic.

APPENDIX C: TIME MEASURING METHODOLOGY

Due to Linux does not support reading CPU cycles directly using the `rdcycle` instruction, the `clock_gettime` function (`CLOCK_MONOTONIC`) is used to measure elapsed time in nanoseconds. The following C code snippet demonstrates how to read the CPU cycles using this method:

```
1 // From generators/gemmini/software/gemmini-rocc-tests/include/  
   gemmini_testutils.h  
2 static uint64_t read_cycles() {  
3     struct timespec ts;  
4     if (clock_gettime(CLOCK_MONOTONIC, &ts) != 0) {  
5         perror("clock_gettime failed");  
6         return 0;  
7     }  
8     return (uint64_t)ts.tv_sec * 1000000000ULL + ts.tv_nsec;  
9 }
```

Listing C.1: Time Measurement Function Used in Gemmini Tests

Exclusive tests were performed to measure the overhead of this method, which was found to be negligible compared to the overall execution time of the tests.

The result shows that the overhead of using `clock_gettime(CLOCK_MONOTONIC)` is the lowest, around 7.4 us, while `gettimeofday` has a similar overhead of around 7.7 us. The `clock_gettime(CLOCK_PROCESS_CPUTIME_ID)` has a higher overhead of around 26.4 us, which is expected as it provides more detailed CPU time information.

```
1 /*  
2  * timing_test.c – Test program to compare clock_gettime() vs  
   gettimeofday()  
3  *  
4  * This program tests the timing functions available on Linux and  
   compares  
5  * their resolution, accuracy, and overhead.  
6  */  
7  
8 #define _POSIX_C_SOURCE 200809L // Enable POSIX.1–2008 features  
9 #define _DEFAULT_SOURCE         // Enable additional glibc features  
10  
11 #include <stdio.h>  
12 #include <stdlib.h>  
13 #include <time.h>  
14 #include <sys/time.h>
```

```

15 #include <unistd.h>
16 #include <stdint.h>
17
18 // Test how many times we can call each timing function
19 #define NUM_ITERATIONS 10000
20
21 // Function to get time using clock_gettime(CLOCK_MONOTONIC)
22 uint64_t get_monotonic_time_ns() {
23     struct timespec ts;
24     if (clock_gettime(CLOCK_MONOTONIC, &ts) != 0) {
25         perror("clock_gettime failed");
26         return 0;
27     }
28     return (uint64_t)ts.tv_sec * 1000000000ULL + ts.tv_nsec;
29 }
30
31 // Function to get time using gettimeofday()
32 uint64_t get_timeofday_us() {
33     struct timeval tv;
34     if (gettimeofday(&tv, NULL) != 0) {
35         perror("gettimeofday failed");
36         return 0;
37     }
38     return (uint64_t)tv.tv_sec * 1000000ULL + tv.tv_usec;
39 }
40
41 // Function to get process CPU time
42 uint64_t get_process_time_ns() {
43     struct timespec ts;
44     if (clock_gettime(CLOCK_PROCESS_CPUTIME_ID, &ts) != 0) {
45         perror("clock_gettime(CLOCK_PROCESS_CPUTIME_ID) failed");
46         return 0;
47     }
48     return (uint64_t)ts.tv_sec * 1000000000ULL + ts.tv_nsec;
49 }
50
51 void test_timing_resolution() {
52     printf("== Testing Timing Resolution ==\n\n");
53
54     // Test clock_gettime resolution
55     struct timespec res;
56     if (clock_getres(CLOCK_MONOTONIC, &res) == 0) {

```

```

57     printf("CLOCK_MONOTONIC resolution: %ld.%09ld seconds\n",
58           res.tv_sec, res.tv_nsec);
59     printf(" = %.1f nanoseconds\n", res.tv_sec * 1e9 + res.tv_nsec)
60 ;
61 } else {
62     printf("Failed to get CLOCK_MONOTONIC resolution\n");
63 }
64
65 if (clock_getres(CLOCK_PROCESS_CPUTIME_ID, &res) == 0) {
66     printf("CLOCK_PROCESS_CPUTIME_ID resolution: %ld.%09ld seconds\n
67 ",
68           res.tv_sec, res.tv_nsec);
69     printf(" = %.1f nanoseconds\n", res.tv_sec * 1e9 + res.tv_nsec)
70 ;
71 } else {
72     printf("Failed to get CLOCK_PROCESS_CPUTIME_ID resolution\n");
73 }
74
75 printf("gettimeofday() resolution: ~1 microsecond (1000 nanoseconds)
76 \n");
77 printf("\n");
78 }
79
80 void test_timing_accuracy() {
81     printf("== Testing Timing Accuracy (Minimal Operation) ==\n\n");
82
83     uint64_t start, end, elapsed;
84     volatile int dummy = 0;
85
86     // Test clock_gettime(CLOCK_MONOTONIC)
87     printf("Testing clock_gettime(CLOCK_MONOTONIC):\n");
88     uint64_t min_mono = UINT64_MAX, max_mono = 0, total_mono = 0;
89
90     for (int i = 0; i < 1000; i++) {
91         start = get_monotonic_time_ns();
92         dummy = dummy + 1; // Minimal operation
93         end = get_monotonic_time_ns();
94         elapsed = end - start;
95
96         if (elapsed < min_mono) min_mono = elapsed;
97         if (elapsed > max_mono) max_mono = elapsed;
98         total_mono += elapsed;

```

```

95     }
96
97     printf("  Min time: %lu ns\n", min_mono);
98     printf("  Max time: %lu ns\n", max_mono);
99     printf("  Avg time: %.1f ns\n", (double)total_mono / 1000.0);
100    printf("  Effective resolution: ~%lu ns\n", min_mono);
101
102    // Test gettimeofday()
103    printf("\nTesting gettimeofday():\n");
104    uint64_t min_gtod = UINT64_MAX, max_gtod = 0, total_gtod = 0;
105
106    for (int i = 0; i < 1000; i++) {
107        start = get_timeofday_us();
108        dummy = dummy + 1;  // Minimal operation
109        end = get_timeofday_us();
110        elapsed = (end - start) * 1000;  // Convert to nanoseconds
111
112        if (elapsed < min_gtod) min_gtod = elapsed;
113        if (elapsed > max_gtod) max_gtod = elapsed;
114        total_gtod += elapsed;
115    }
116
117    printf("  Min time: %lu ns\n", min_gtod);
118    printf("  Max time: %lu ns\n", max_gtod);
119    printf("  Avg time: %.1f ns\n", (double)total_gtod / 1000.0);
120    printf("  Effective resolution: ~%lu ns\n", min_gtod);
121
122    printf("\n");
123 }
124
125 void test_timing_overhead() {
126     printf("== Testing Timing Function Overhead ==\n\n");
127
128     uint64_t start, end;
129     uint64_t overhead_mono = 0, overhead_gtod = 0, overhead_proc = 0;
130
131     // Test clock_gettime(CLOCK_MONOTONIC) overhead
132     start = get_monotonic_time_ns();
133     for (int i = 0; i < NUM_ITERATIONS; i++) {
134         get_monotonic_time_ns();
135     }
136     end = get_monotonic_time_ns();

```

```

137     overhead_mono = (end - start) / NUM_ITERATIONS;
138
139     // Test gettimeofday() overhead
140     start = get_monotonic_time_ns();
141     for (int i = 0; i < NUM_ITERATIONS; i++) {
142         get_timeofday_us();
143     }
144     end = get_monotonic_time_ns();
145     overhead_gtod = (end - start) / NUM_ITERATIONS;
146
147     // Test clock_gettime(CLOCK_PROCESS_CPUTIME_ID) overhead
148     start = get_monotonic_time_ns();
149     for (int i = 0; i < NUM_ITERATIONS; i++) {
150         get_process_time_ns();
151     }
152     end = get_monotonic_time_ns();
153     overhead_proc = (end - start) / NUM_ITERATIONS;
154
155     printf("Function call overhead (average over %d calls):\n",
NUM_ITERATIONS);
156     printf("    clock_gettime(CLOCK_MONOTONIC):      %lu ns\n",
overhead_mono);
157     printf("    gettimeofday():                          %lu ns\n",
overhead_gtod);
158     printf("    clock_gettime(CLOCK_PROCESS_CPUTIME): %lu ns\n",
overhead_proc);
159     printf("\n");
160 }
161
162 void test_timing_demo() {
163     printf("==== Practical Timing Example ==== \n\n");
164
165     uint64_t start_mono, end_mono;
166     uint64_t start_gtod, end_gtod;
167     uint64_t start_proc, end_proc;
168
169     printf("Timing a simple computation (1000 additions):\n");
170
171     // Time with clock_gettime(CLOCK_MONOTONIC)
172     start_mono = get_monotonic_time_ns();
173     volatile long sum = 0;
174     for (int i = 0; i < 1000; i++) {

```



```

175         sum += i;
176     }
177     end_mono = get_monotonic_time_ns();
178
179     // Time with gettimeofday()
180     start_gtod = get_timeofday_us();
181     sum = 0;
182     for (int i = 0; i < 1000; i++) {
183         sum += i;
184     }
185     end_gtod = get_timeofday_us();
186
187     // Time with process CPU time
188     start_proc = get_process_time_ns();
189     sum = 0;
190     for (int i = 0; i < 1000; i++) {
191         sum += i;
192     }
193     end_proc = get_process_time_ns();
194
195     printf("Results:\n");
196     printf("    clock_gettime(CLOCK_MONOTONIC):    %lu ns (%.2f us)\n",
197           end_mono - start_mono, (end_mono - start_mono) / 1000.0);
198     printf("    gettimeofday():                      %lu us (%.2f us)\n",
199           end_gtod - start_gtod, (double)(end_gtod - start_gtod));
200     printf("    clock_gettime(CLOCK_PROCESS_CPUTIME): %lu ns (%.2f us)\n",
201           end_proc - start_proc, (end_proc - start_proc) / 1000.0);
202     printf("    Sum result: %ld\n", sum);
203     printf("\n");
204 }
205
206 void show_current_time() {
207     printf("== Current Time Values ==\n\n");
208
209     struct timespec ts;
210     struct timeval tv;
211
212     // Show CLOCK_MONOTONIC
213     if (clock_gettime(CLOCK_MONOTONIC, &ts) == 0) {
214         printf("CLOCK_MONOTONIC: %ld.%09ld seconds\n", ts.tv_sec, ts.
215 tv_nsec);
216     }

```

```

216
217 // Show gettimeofday
218 if (gettimeofday(&tv, NULL) == 0) {
219     printf("gettimeofday(): %ld.%06ld seconds\n", tv.tv_sec, tv.
tv_usec);
220 }
221
222 // Show CLOCK_PROCESS_CPUTIME_ID
223 if (clock_gettime(CLOCK_PROCESS_CPUTIME_ID, &ts) == 0) {
224     printf("PROCESS_CPUTIME: %ld.%09ld seconds\n", ts.tv_sec, ts.
tv_nsec);
225 }
226
227 printf("\n");
228 }
229
230 int main() {
231     printf("Linux Timing Functions Test Program\n");
232     printf("=====\n\n");
233
234     printf("This program tests the timing functions used in the Gemini\n
n");
235     printf("Linux port to replace the rdcycle instruction.\n\n");
236
237     show_current_time();
238     test_timing_resolution();
239     test_timing_accuracy();
240     test_timing_overhead();
241     test_timing_demo();
242
243     return 0;
244 }

```

Listing C.2: C Code for Timing Test

```

1 debian@debian:~$ ./timing_test
2 Linux Timing Functions Test Program
3 =====
4
5 This program tests the timing functions used in the Gemini
6 Linux port to replace the rdcycle instruction.
7
8 == Current Time Values ==

```

```

9
10 CLOCK_MONOTONIC: 288.119331000 seconds
11 gettimeofday(): 1710323247.363538 seconds
12 PROCESS_CPUTIME: 0.039620000 seconds
13
14 == Testing Timing Resolution ==
15
16 CLOCK_MONOTONIC resolution: 0.000000001 seconds
17 = 1.0 nanoseconds
18 CLOCK_PROCESS_CPUTIME_ID resolution: 0.000000001 seconds
19 = 1.0 nanoseconds
20 gettimeofday() resolution: ~1 microsecond (1000 nanoseconds)
21
22 == Testing Timing Accuracy (Minimal Operation) ==
23
24 Testing clock_gettime(CLOCK_MONOTONIC):
25   Min time: 6000 ns
26   Max time: 133000 ns
27   Avg time: 7505.0 ns
28   Effective resolution: ~6000 ns
29
30 Testing gettimeofday():
31   Min time: 7000 ns
32   Max time: 133000 ns
33   Avg time: 8165.0 ns
34   Effective resolution: ~7000 ns
35
36 == Testing Timing Function Overhead ==
37
38 Function call overhead (average over 10000 calls):
39   clock_gettime(CLOCK_MONOTONIC): 7393 ns
40   gettimeofday(): 7666 ns
41   clock_gettime(CLOCK_PROCESS_CPUTIME): 26380 ns
42
43 == Practical Timing Example ==
44
45 Timing a simple computation (1000 additions):
46 Results:
47   clock_gettime(CLOCK_MONOTONIC): 69000 ns (69.00 us)
48   gettimeofday(): 69 us (69.00 us)
49   clock_gettime(CLOCK_PROCESS_CPUTIME): 128000 ns (128.00 us)

```

```
Sum result : 499500
```

Listing C.3: Output of Timing Test

APPENDIX D: DETAILED PERFORMANCE EVALUATION RESULTS

This appendix provides the comprehensive, unabridged results of the performance evaluation conducted on the implemented SoC. The data was generated using the `gemmini-rocc-tests` benchmark suite on the Xilinx VC707 FPGA platform.

D.1 Performance Benchmark Results

Table D.1: Detailed Matrix Multiplication Performance

Test: tiled_matmul_*	Matrix Size	Execution Time (ms)		Speedup
		CPU	Gemmini	
*_os	-	2,036.450	38.781	52.5×
*_ws	512×512×512	2,488.828	20.438	121.8×
*_ws_At	-	4,113.003	12.445	330.5×
*_ws_Bt	-	510.343	12.362	41.3×
*_ws_full_C	-	4,064.579	22.064	184.2×
*_ws_low_D	-	4,264.698	12.096	352.6×

Table D.2: Detailed Convolution Performance

Test: conv_*	Input Size	Output Size	Execution Time (ms)		Speedup
			CPU	Gemmini	
*	224×224	112×112	44,931.245	83.296	539.3×
*_first_layer	224×224	112×112	44,727.939	84.102	531.8×
*_perf	224×224 (4 batch, 3→32 ch)	112×112	-	84.032	-
*_dw	112×112	56×56	1,536.294	102.560	15.0×
*_dw_perf	112×112 (3 batch, 17 ch)	56×56	-	103.971	-
*_rect	224×224	112×112	44,761.962	83.120	538.4×
*_stride	224×224	-	45,357.882	81.936	553.7×
*_trans_output_1203	-	-	131,830.362	130.941	1006.6×
*_trans_weight_0132	-	-	169,097.846	129.206	1308.7×
*_trans_weight_1203	-	-	134,060.240	132.128	1014.6×
*_with_rot180	-	-	93,918.979	146.602	640.5×

Table D.3: Detailed Convolution with Pooling Performance

Test: conv	Input Size	Output Size	Execution Time				Speedup
			CPU (sec)			Gemmini (ms)	
			Conv	Pool	Total		
*_with_ pool	224×224	112×112 →56×56	44,810	4,024	48,834	97	503.2×
*_rect_ pool	224×224	112×112 →56×56	44,755	4,014	48,769	98	494.6×

Table D.4: Detailed MLP Model Performance

Model	Layer Distribution	Execution Time (ms)
mlp1_32	6 layers (22.4M, 77.1M, 46.1M, 22.5M, 6.2M, 0.5M)	174.870
mlp1	6 layers (22.4M, 77.3M, 45.7M, 18.7M, 6.2M, 0.5M)	170.637
mlp4	2 layers (215.6M, 217.2M)	432.838

D.2 Test Program Documentation and Configurations

This section documents the specific configuration of each test program used in the evaluation, ensuring full reproducibility of the results.

D.2.1 Matrix Multiplication Tests (bareMetalC/)

D.2.1.1 Basic Matrix Multiplication Tests

- `tiled_matmul_os`: Output-stationary matrix multiplication
 - Uses Gemmini’s output-stationary dataflow
 - Matrix dimensions: 512×512×512 (non-baremetal)
- `tiled_matmul_ws`: Weight-stationary matrix multiplication
 - Uses Gemmini’s weight-stationary dataflow (most efficient)
 - Matrix dimensions: 512×512×512 (I×J×K)

D.2.1.2 Matrix Multiplication Variants

- `tiled_matmul_ws_At`: Weight-stationary with A-transpose
 - Same as `tiled_matmul_ws` but with matrix A transposed

- Matrix dimensions: $500 \times 300 \times 412$ (I×J×K)
- `tilled_matmul_ws_Bt`: Weight-stationary with B-transpose
 - Same as `tilled_matmul_ws` but with matrix B transposed
- `tilled_matmul_ws_full_C`: Weight-stationary with full accumulator width
 - Uses full-width accumulator for output matrix C
- `tilled_matmul_ws_low_D`: Weight-stationary with bias matrix
 - Includes bias matrix D in computation ($C = A \times B + D$)

D.2.1.3 Performance Analysis Tests

- `tilled_matmul_ws_perf`: Weight-stationary performance analysis
 - Matrix dimensions: $128 \times 256 \times 512$ (I×J×K)
 - **Verified metrics:**
 - * Total MACs: 16,777,216
 - * Time Execution (ms): 2.426 | **Utilization: 2%**
 - * Memory traffic: 2.16MB RDMA reads, 32KB WDMA writes

D.2.2 Convolution Tests (bareMetalC/)

D.2.2.1 Standard Convolution Tests

- `conv`: Basic 2D convolution operation
 - Input: $224 \times 224 \times 3$ (batch=4), Output: $112 \times 112 \times 32$
 - Kernel: 3×3 , Stride: 2, Padding: 1
- `conv_first_layer`: First layer convolution (typical CNN entry)
 - Same configuration as `conv` but optimized for first layer
 - Input: $224 \times 224 \times 3$ (batch=4), Output: $112 \times 112 \times 32$
- `conv_rect`: Rectangular convolution
 - Tests non-square convolution operations

- Input: $224 \times 224 \times 3$ (batch=4), Output: $112 \times 112 \times 32$
- `conv_stride`: Strided convolution
 - Tests convolution with stride=2
 - Input: $224 \times 224 \times 3$ (batch=4), Output: $112 \times 112 \times 32$

D.2.2.2 Depthwise Convolution Tests

- `conv_dw`: Depthwise convolution
 - Input: $112 \times 112 \times 17$ (batch=3), Output: $56 \times 56 \times 17$
 - Lower computational intensity than standard convolution

D.2.2.3 Transformed Convolution Tests

- `conv_trans_output_1203`: Output transformation (1,2,0,3)
 - Tests convolution with output tensor transformation
 - Input: $224 \times 224 \times 17$ (batch=4), Output: $112 \times 112 \times 32$
- `conv_trans_weight_0132`: Weight transformation (0,1,3,2)
 - Tests convolution with weight tensor transformation
 - Input: $224 \times 224 \times 17$ (batch=4), Output: $112 \times 112 \times 32$
- `conv_trans_weight_1203`: Weight transformation (1,2,0,3)
 - Alternative weight tensor transformation
 - Input: $224 \times 224 \times 17$ (batch=4), Output: $112 \times 112 \times 32$

D.2.2.4 Convolution with Pooling Tests

- `conv_with_pool`: Convolution followed by pooling
 - Input: $224 \times 224 \times 3$ (batch=4), Conv output: $112 \times 112 \times 32$, Pool output: $56 \times 56 \times 32$
- `conv_rect_pool`: Rectangular convolution with pooling
 - Same configuration as `conv_with_pool`

D.2.2.5 Special Convolution Tests

- `conv_with_rot180`: Convolution with 180° rotation
 - Tests convolution with rotated input
 - Input: $224 \times 224 \times 3$ (batch=4), Output: $224 \times 224 \times 17$ (stride=1, no size reduction)

D.2.3 Neural Network Models (imagenet/)

- `mobilenet`: Complete MobileNet v1 inference
 - Architecture: Standard convolution + 13 depthwise separable convolution blocks
 - Input: $224 \times 224 \times 3$ ImageNet images

D.2.4 Multi-Layer Perceptron Models (mlps/)

- `mlp1`: 6-layer MLP with RELU activation
 - Architecture: $832 \rightarrow 2560 \rightarrow 2048 \rightarrow 1536 \rightarrow 1024 \rightarrow 512 \rightarrow 64$
 - Original dimensions: $784 \rightarrow 2500 \rightarrow 2000 \rightarrow 1500 \rightarrow 1000 \rightarrow 500 \rightarrow 10$ (zero-padded)
 - Batch size: 64
 - **Total time execution (ms)**: 170.637
 - **Layer breakdown**: 22.4M, 77.3M, 45.7M, 18.7M, 6.2M, 0.5M (ms)
- `mlp1_32`: 32-bit version of MLP1
 - Same architecture as `mlp1` but with 32-bit precision
 - **Total Time Execution (ms)**: 174.870
 - **Layer breakdown**: 22.4M, 77.1M, 46.1M, 22.5M, 6.2M, 0.5M (ms)
- `mlp4`: 2-layer MLP with RELU activation
 - Architecture: $3072 \rightarrow 4608 \rightarrow 3072$
 - Original dimensions: $3036 \rightarrow 4554 \rightarrow 3036$ (zero-padded)
 - Batch size: 64
 - **Total Time Execution (ms)**: 432.838
 - **Layer breakdown**: 215.6M, 217.2M (ms)

D.2.5 Additional Tests

- **resadd**: Residual addition operations
 - Tests element-wise addition for residual connections
 - **Time Execution (ms)**: 1.382 (128×512 matrix)
- **gemmini_counter**: Hardware counter validation
 - Tests Gemmini’s performance counter functionality
 - Used for performance monitoring and debugging